

TALUS: Threshold ML-DSA with One-Round Online Signing via Boundary Clearance and Carry Elimination

Leo Kao 

Codebat Technologies Inc., Taipei, Taiwan

Abstract. Deploying ML-DSA (FIPS 204) in threshold settings has remained an open problem: the scheme’s inherently non-linear rounding step defeats the additive share techniques that underpin practical threshold schemes for elliptic-curve signatures such as FROST. We present TALUS, the first threshold ML-DSA construction that achieves *one-round online signing* with $> 99\%$ online success, while producing standard signatures verifiable by any unmodified ML-DSA verifier. We formalise this as the *Lattice Threshold Trilemma*, proving that no group homomorphism from the ML-DSA nonce space into any abelian group can simultaneously be hiding and binding, ruling out all possible homomorphic commitment schemes.

TALUS overcomes this barrier with two techniques. The *Boundary Clearance Condition* (BCC) identifies nonces whose rounding residuals lie far enough from modular boundaries that the secret key component s_2 has no effect on the signature; such nonces ($\approx 31.7\%$ of attempts) are filtered during offline preprocessing. The *Carry Elimination Framework* (CEF) then enables parties to compute the commitment hash input distributedly, without reconstructing the full nonce product. Together, BCC and CEF reduce online signing to a single broadcast round: each party sends one message and the coordinator assembles a valid FIPS 204 signature.

We instantiate TALUS in two deployment profiles: TALUS-TEE (trusted execution environment, T -of- N) and TALUS-MPC (fully distributed, malicious security with identifiable abort for $T \geq 2$). Security of both variants reduces to ML-DSA EUF-CMA. A Rust implementation across all three FIPS 204 security levels (ML-DSA-44, ML-DSA-65, ML-DSA-87) shows that TALUS-TEE completes a signing operation in 0.62–1.94 ms and TALUS-MPC in 2.27–5.02 ms (amortised, $T=3$), competitive with the fastest concurrent threshold ML-DSA proposals.

Keywords: threshold signatures · ML-DSA · FIPS 204 · post-quantum cryptography · lattice-based cryptography · boundary clearance · carry elimination

Contents

1	Introduction	3
1.1	Core Innovations	4
1.2	Contributions	5
1.3	Comparison with Prior Work	5
1.4	Technical Overview	5
1.5	Related Work	7
1.6	Paper Organization	8
2	Preliminaries	8
2.1	Notation	8
2.2	ML-DSA (FIPS 204)	8
2.3	Shamir Secret Sharing	9
2.4	Shamir Nonce DKG	9

E-mail: leo@codebat.ai (Leo Kao)

2.5	Security Definitions	10
2.6	Beaver Triples and MPC Primitives	10
3	The Lattice Threshold Trilemma	11
3.1	DL vs. Lattice Commitments	11
3.2	The Trilemma Theorem	11
3.3	Consequences for Protocol Design	12
4	Boundary Clearance Condition	12
4.1	Why cs_2 Appears Necessary	13
4.2	Boundary Distance and the BCC	13
4.3	Safety Theorem	13
4.4	Corollaries	15
4.5	Per-Attempt BCC Pass Probability	15
4.6	Why This Wasn't Found Before	16
5	Carry Elimination Framework	17
5.1	The Distributed w_1 Problem	17
5.2	The Key Identity	18
5.3	Carry Elimination Protocol	18
5.4	Secondary Carry Analysis (Lagrange Case)	18
5.5	Communication Cost	19
5.6	CEF in TALUS-MPC: Masked Broadcast	19
6	TALUS-TEE	21
6.1	Key Generation	21
6.2	Offline Preprocessing	21
6.3	Online Signing	22
6.4	Blame Protocol	23
6.5	Key Refresh	23
7	TALUS-MPC	23
7.1	System Model	23
7.2	Key Generation (Phase 0)	24
7.3	CarryCompare: Distributed Carry Comparison	25
7.4	Offline Preprocessing (Phase 1)	27
7.5	Offline Round Count	29
7.6	Beaver Triple Construction from Pairwise Seeds	30
7.7	Online Signing (Phase 2)	31
7.8	Correctness and Success Rate	32
7.9	Shift-Invariance and Security Loss	32
7.10	Communication Analysis	34
7.11	Blame Protocol (Identifiable Abort)	34
7.12	Security	35
8	Security	36
8.1	Security of TALUS-TEE	36
8.2	Privacy of TALUS-TEE	38
8.3	Security of TALUS-MPC	38
8.4	Combiner Privacy (M-LWR Reduction)	42
8.5	Carry Leakage Analysis	42

9	Implementation	43
9.1	Experiments	43
9.2	Performance	44
9.3	Communication Cost Breakdown	46
9.4	Computation Costs	47
9.5	Optimization Roadmap	47
10	Discussion	48
10.1	FROST Structural Isomorphism	48
10.2	Deployment Trade-offs	48
10.3	Implementation Considerations	50
10.4	Future Directions	52
11	Conclusion	53
A	Supplementary Game-Hopping Proofs	53
A.1	Proof of Theorem 23 (TALUS-TEE EUF-CMA)	53
A.2	Proof of Theorem 24	54
A.3	TALUS-MPC Security Proof	54
B	UC Security	55
B.1	UC Security of TALUS-TEE	55
B.2	Ideal Functionalities	56
B.3	The TALUS-TEE Simulator	56
B.4	BCC Coupling Lemma	57
B.5	Indistinguishability Argument	58
B.6	Feldman-VSS Commitment for Blame	58
B.7	UC Security of TALUS-MPC	59
C	Proactive Key Refresh	61
C.1	Motivation and Erasure Model	61
C.2	Refresh Protocol	61
C.3	Security Analysis	62
D	Shift-Invariance Analysis	63
D.1	Shift-Invariance Probability vs. Shift Magnitude	63
D.2	Carry Probability for Distributed $\mathbf{w}_1$	63
E	TALUS for All ML-DSA Parameter Sets	64
F	FIPS 204 Hint Encoding	64
G	Trilemma Proof and Carry–Privacy Duality	65
G.1	Proof of Theorem 1	65
G.2	Carry–Privacy Duality	66
	References	67

1 Introduction

Threshold signatures allow a group of N parties to collaboratively sign messages such that any T of them can produce a valid signature, but fewer than T learn nothing about the signing key. With the NIST standardisation of post-quantum cryptography [Nat24b, Nat24a], threshold variants of these standards are urgently needed for enterprise key management, hardware security modules (HSMs), and multi-party custody systems.

The flagship post-quantum signature standard ML-DSA (FIPS 204, based on CRYSTALS-Dilithium [DKL⁺18]) presents a specific challenge for thresholdization that classical schemes do not: the *r0-check*. In single-party ML-DSA, after computing the response $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$, the signer checks that a secondary residual $\mathbf{r}_0 - c\mathbf{s}_2$ is small. In the threshold setting, $\mathbf{s}_2$ is a secret key component that must be either: (a) shared with MPC (expensive), or (b) held by a trusted coordinator (limits trust model), or (c) incorporated via a noise-flooding mechanism (degrades parameters). Every prior threshold ML-DSA construction [CdPE⁺26, BdCE⁺26, DKLS25] (and lattice-based threshold signature construction [BCdP⁺26]) handles the r0-check in one of these ways, accepting the overhead as fundamental.

This paper. We show the r0-check overhead is *not fundamental*: it can be eliminated with probability $\approx 31.7\%$ per signing attempt using a simple structural observation about the ML-DSA commitment geometry. The result is **TALUS** (Threshold Agile Lattice Unified Signatures), the first threshold ML-DSA construction that simultaneously achieves: (i) full **FIPS 204 compatibility** (signatures verify with any unmodified ML-DSA verifier), (ii) **arbitrary threshold** (T -of- N for any $1 \leq T \leq N$, with signing set $|S| = T$ exactly; TALUS-TEE requires no honest-majority condition; TALUS-MPC requires $N \geq 2T - 1$ for $T \geq 3$ by the Dishonest Majority Barrier (Theorem 27), and no restriction for $T = 2$), and (iii) **one online round** (parties send $\mathbf{z}_i$; coordinator assembles σ), matching the latency of single-party signing. TALUS-TEE achieves this with no restrictions; TALUS-MPC achieves it after $\max(3, \lceil \log_2(N/2) \rceil + 2)$ offline preprocessing rounds (Lemma 6), which are fully amortisable and independent of any online message. Prior FIPS-compatible schemes achieve (i) and (ii) but not (iii): Mithril [CdPE⁺26] requires 3 online rounds; Quorus [BdCE⁺26] requires 2 abstract rounds (16–29 actual broadcast rounds when instantiated [BdCE⁺26]). TALUS is the first to achieve all three simultaneously.

1.1 Core Innovations

1.1.1 Boundary Clearance Condition (BCC)

The signing nonce commitment $\mathbf{w} = \mathbf{A}\mathbf{y}$ decomposes as $(\mathbf{w}_1, \mathbf{r}_0) = \text{Decompose}(\mathbf{w}, 2\gamma_2)$. When every coefficient of $\mathbf{r}_0$ is bounded away from $\pm\gamma_2$ by more than $\beta = \tau\eta$, the secret-dependent shift $c\mathbf{s}_2$ cannot cause any coefficient to cross a modular boundary. This means:

- the r0-check passes automatically (no computation of $c\mathbf{s}_2$ needed);
- the hint $\mathbf{h}$, which corrects for ct_0 low-bit carries, is computable entirely from public values $\mathbf{r} = \mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d$ and $\mathbf{w}_1$, with expected weight ≈ 38 and $\|\mathbf{h}\|_1 \leq \omega = 55$;
- no MPC on $\mathbf{s}_2$ is required;
- the signature $\sigma = (\tilde{c}, \mathbf{z}, \mathbf{h})$ requires only the response $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$ from parties.

We call this the *Boundary Clearance Condition* (BCC), formally defined in Section 4 (Theorem 2). BCC holds with probability $\approx 31.7\%$ per nonce attempt (Theorem 3).

1.1.2 Carry Elimination Framework

The commitment hash input $\mathbf{w}_1 = \text{HighBits}(\mathbf{A}\mathbf{y}, 2\gamma_2)$ requires the aggregated nonce $\mathbf{w} = \mathbf{A}\mathbf{y}$. In a threshold setting, each party holds $\mathbf{y}_i$ (not $\mathbf{y}$ directly), so parties cannot independently compute $\mathbf{w}_1$.

We exploit the identity $\alpha^{-1} \equiv -16 \pmod{q}$ (where $\alpha = 2\gamma_2 = (q-1)/16$) to develop a distributed protocol for computing $\mathbf{w}_1$ without reconstructing $\mathbf{w}$. In TALUS-MPC, each party h uses its own additive nonce piece $\hat{\mathbf{w}}_h = \mathbf{A}\hat{\mathbf{y}}_h$ (unit coefficients), broadcasts a masked decomposition $(\tilde{H}_h, \tilde{b}_h)$, and the coordinator assembles the true $\mathbf{w}_1$ using the algebraic identity and a single carry bit from a distributed comparison (CSCP). We call this the *Carry Elimination Framework* (Section 5).

Carry identity $\mathbf{w}_1$ matches direct computation exactly for all $T \in \{2, 3, 4, 5, 8\}$ (100% agreement over 10^5 trials); see `experiments/carry_identity.py`.

1.2 Contributions

1. **Lattice Threshold Trilemma (Theorem 1):** A proof that no group homomorphism from the nonce space $(R_q^\ell, +)$ into *any* finite abelian group can simultaneously be hiding and binding, and that HighBits is not a group homomorphism (since $\mathbb{Z}_q$ is simple). This establishes the carry cost as a *mathematical necessity* for any FIPS-compatible lattice-based threshold scheme, not a protocol artifact. The carry-privacy duality (Proposition 3) formalises why the carry must be computed without revealing the LowBits remainder.
2. **Boundary Clearance Condition (Theorem 2):** When $\|\mathbf{r}_0\|_\infty < \gamma_2 - \beta$, the secret-dependent shift $\mathbf{cs}_2$ cannot cross any rounding boundary, so $\mathbf{w}_1$ is unchanged and the hint $\mathbf{h}$ is publicly computable. *Prior work treated the r0-check as an unavoidable online predicate requiring MPC on $\mathbf{s}_2$. The key insight is that BCC depends only on $\mathbf{r}_0 = \text{LowBits}(\mathbf{Ay})$, a public function of the nonce, and can therefore be evaluated offline.* BCC holds with probability $\approx 31.7\%$ per nonce attempt (Theorem 3), yielding expected $3.15\times$ preprocessing overhead.
3. **Carry Elimination Framework (Section 5):** A distributed protocol for computing $\mathbf{w}_1 = \text{HighBits}(\mathbf{Ay}, 2\gamma_2)$ without reconstructing $\mathbf{y}$. The identity $\alpha^{-1} \equiv -16 \pmod{q}$ reduces the carry computation to a single secure comparison via the Carry-Safe Comparison Protocol (CSCP), computable in $\max(3, \lceil \log_2(N/2) \rceil + 2)$ offline rounds. When $T = 2$, CarryCompare instantiates as a Distributed Comparison Function (DCF) [BGI15, BGI16] (Section 7.3.1), reducing offline communication from ≈ 350 KB to ≈ 5 KB per batch ($\sim 70\times$ improvement).
4. **Protocol instantiation (Sections 6–7):** Two deployment profiles: TALUS-TEE (T -of- N , trusted TEE coordinator; signers may be malicious with identifiable abort) and TALUS-MPC (fully distributed, malicious security with identifiable abort for $T \geq 2$). Both achieve one online round and produce standard FIPS 204 signatures verifiable by unmodified implementations. Security of both variants reduces to ML-DSA EUF-CMA (Theorems 23 and 25).
5. **Implementation (Section 9):** A Rust reference implementation across all three FIPS 204 security levels with Criterion benchmarks. At $T = 3$ (ML-DSA-65), online signing costs 0.29 ms (TEE) and 0.68 ms (MPC); amortised offline preprocessing costs 1.23 ms and 3.41 ms per accepted nonce.

1.3 Comparison with Prior Work

Table 1 compares TALUS with existing threshold ML-DSA and related schemes.

TALUS-TEE achieves one online round, which is minimal in the sense that each party must send at least one message (their response share $\mathbf{z}_i$) before the coordinator can assemble the signature, while maintaining FIPS compatibility. This is achieved by the combination of BCC (eliminating $\mathbf{s}_2$ computation) and the Shamir Nonce DKG [Kao26] (enabling T -of- N with $|S| = T$, no two-honest requirement). TALUS-MPC also uses one broadcast round per signing *attempt*. However, unlike TALUS-TEE (which pre-filters nonces offline via BCC before any signing session starts), TALUS-MPC cannot pre-check BCC in preprocessing: the coordinator learns $\mathbf{w}_1$ but not $\mathbf{w}_0 = \mathbf{w} \bmod \alpha$, so the BCC test $\|\mathbf{w}_0\|_\infty < \gamma_2 - \beta$ cannot be evaluated offline. Each online attempt therefore succeeds with probability $p_{\text{BCC}} \approx 31.7\%$, requiring ≈ 3.15 attempts (and preprocessing sessions) per successful signature on average. Figure 1 illustrates the architectural difference between the two variants.

1.4 Technical Overview

Why the r0-check is the bottleneck. In standard ML-DSA, the r0-check at step 6 is a cheap local operation. In the threshold setting it requires evaluating $\mathbf{cs}_2$ across shares. Without BCC, this requires an MPC multiplication of the challenge c (public) with the secret $\mathbf{s}_2$: in practice a Beaver-triple multiplication or a TEE evaluation, adding at least one online round.

Table 1: Comparison of threshold ML-DSA constructions. “OR” = online rounds *in which signers broadcast* (coordinator’s challenge broadcast is not counted; FROST [KG21]’s 2 rounds = nonce-commitment + response, both signer-initiated). “Comm” = online communication per party ($O(n_\ell)$ hides constant factors; TALUS-MPC Round 1 broadcasts ≈ 26.2 KB at $T = 3$, see Table 7). “FIPS” = produces FIPS 204-valid signatures. “IA” = identifiable aborts. “KR” = key refresh. “Trust” = trust model. *Security model caveat:* Trilithium achieves UC security; all other schemes (including TALUS) achieve game-based EUF-CMA; these are not directly comparable security guarantees.

Scheme	OR	Comm/party	FIPS	OR-Setup	IA	KR	Determ. ^b	Trust
Mithril [CdPE ⁺ 26]	3	$O(n_\ell)$	Yes	1	No	No	No	MPC
Hermine [BCdP ⁺ 26]	1	$O(n_\ell)$	No	1	Yes	Yes	Yes	MPC
Quorus [BdCE ⁺ 26]	2	$O(n_\ell)$	Yes	1	No	No	Yes	MPC [#]
Trilithium [DKLS25]	14 [*]	$O(n_\ell)$	Yes	1 [*]	No	No	No	2PC [*]
TALUS-TEE (ours)	1	$O(n_\ell)$	Yes	1	Yes	Yes	Yes	TEE [§]
TALUS-MPC (ours)	1[‡]	$O(n_\ell)$	Yes	$O(\log N)$	Yes [†]	Yes [‡]	No [‡]	MPC [¶]

[†]Optimistic IA: blame via Feldman VSS check (online) or reveal-on-failure (offline); see Section 7.11.

[‡]Per attempt ($p_{\text{BCC}} \approx 31.7\%$ success; ≈ 3.15 rounds/signature): BCC untestable offline in MPC.

[§]TALUS-TEE: any $N \geq T$; no honest-majority requirement (TEE coordinator evaluates carry locally).

[¶]TALUS-MPC: $N \geq 2T-1$ required for $T \geq 3$ (Dishonest Majority Barrier); $T=2$ has no restriction.

^{||}Quorus reports 2 ideal-functionality calls (F_Open); actual MPC instantiation requires 16–29 broadcast rounds (Table 3 of [BdCE⁺26]).

[#]Quorus requires honest majority ($N \geq 2T-1$).

^{*}Trilithium is a 2-party (Phone+Server) protocol requiring a trusted Correlated Randomness Provider; 14 online rounds per attempt (≈ 60 expected with ML-DSA rejection retries).

^b“Determ.” = the online signing phase never aborts and always produces a valid signature in exactly one round (given BCC-filtered nonces); Mithril and Trilithium may abort-and-retry online.

[‡]TALUS-MPC key refresh: one-round proactive protocol (Algorithm TALUS-MPC.Refresh, Appendix C) achieving mobile adversary security [HJKY95] with erasures.

BCC eliminates the bottleneck. When $\mathbf{r}_0$ is “interior” (BCC holds), the subtraction of any $c\mathbf{s}_2$ (with $|c\mathbf{s}_2| \leq \beta$) cannot change the high-order part of $\mathbf{w}$. No MPC on $\mathbf{s}_2$ is needed: the hint $\mathbf{h}$ is computable from public values $\mathbf{r} = \mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d$ and $\mathbf{w}_1$. Parties send only $\mathbf{z}_i = \mathbf{y}_i + c\mathbf{s}_{1,i}$; the coordinator assembles $\mathbf{z}$ and computes $\mathbf{h}$ locally.

The trilemma. The Lattice Threshold Trilemma (Section 3) establishes that the carry cost is fundamental to any FIPS-compatible lattice-based threshold scheme: no group homomorphism from the nonce space $(R_q^\ell, +)$ into any finite abelian group can simultaneously be hiding and binding, and HighBits is not a group homomorphism.

Carry elimination for distributed $\mathbf{w}_1$. The challenge c depends on $\mathbf{w}_1 = \text{HighBits}(\mathbf{A}\mathbf{y})$. In TALUS-TEE, the coordinator holds $\mathbf{y}$ (via TEE) and computes $\mathbf{w}_1$ directly. In TALUS-MPC, parties hold shares $\mathbf{y}_i$ and must compute $\mathbf{w}_1$ without reconstructing $\mathbf{w}$. The carry elimination framework handles the “carries” introduced by the Lagrange combination: using $\alpha^{-1} \equiv -16 \pmod{q}$, the coordinator can correct the contribution of each party’s $\text{HighBits}(\mathbf{A}\mathbf{y}_i)$ to recover the true $\mathbf{w}_1$.

Success rate. TALUS-TEE succeeds when BCC holds ($\approx 31.7\%$), the z-bound passes (empirically $\approx 100\%$), and $\text{wt}(\mathbf{h}) \leq \omega$ ($\approx 99.7\%$). Combined success rate $\approx 31.5\%$ per preprocessed nonce (confirmed experimentally for $|S| \in \{2, 3, 5\}$; see `experiments/talus_tee.py`). Failed attempts consume preprocessed nonces; the coordinator simply advances to the next preprocessed nonce.

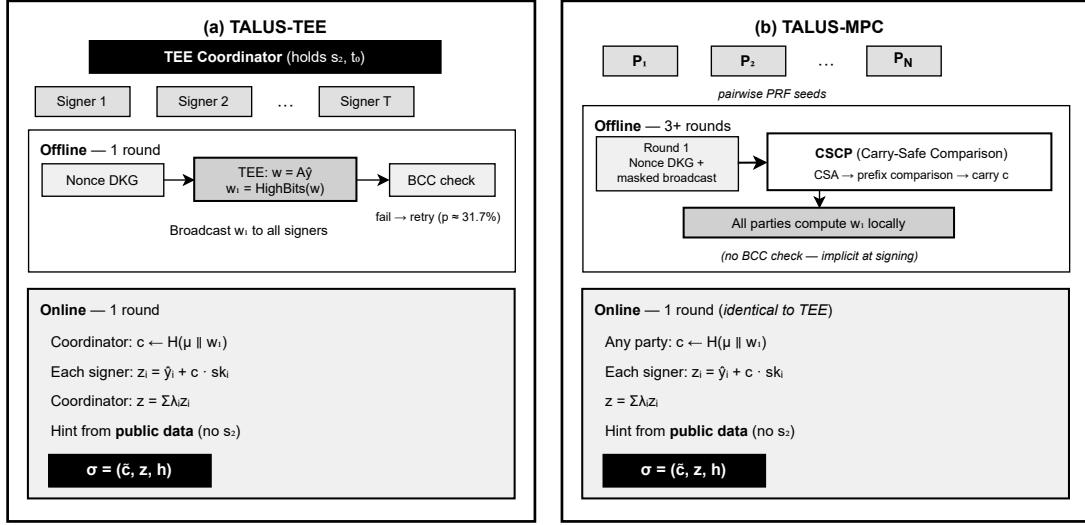


Figure 1: High-level architecture comparison. **(a)** TALUS-TEE: a trusted coordinator holds s_2 and t_0 , computes $w_1 = \text{HighBits}(A\hat{y})$ centrally, and pre-filters nonces offline via BCC. **(b)** TALUS-MPC: all N parties are equal; w_1 is computed distributedly via the Carry-Safe Comparison Protocol (CSCP) in 3+ offline rounds. Both variants share an identical 1-round online phase producing a FIPS 204-valid signature $\sigma = (\tilde{c}, z, h)$.

1.5 Related Work

Lattice-based threshold signatures. Raccoon [dPKPR24] achieves threshold signing for a custom lattice scheme using masking techniques, but is not FIPS compatible. Flood-and-Submerge [ENP24] uses noise flooding and random submersions for Plover/Pelican (hash-and-sign lattice), achieving robustness but with ≈ 13 KB signatures and non-standard parameters. del Pino and Niot [dPN25] construct compact threshold signatures for small thresholds ($T \leq 8$) using replicated secret sharing with short shares; signatures are ≈ 2.7 KB (close to single-party Dilithium) but the scheme is Dilithium-based, not FIPS 204 compatible. Ringtail [BKL⁺25] achieves two-round threshold signing (1 online round after preprocessing) from standard LWE and SIS assumptions, but produces non-standard signatures (≈ 13 KB, not FIPS 204 compatible).

Concurrent ML-DSA threshold work. Mithril [CdPE⁺26] (USENIX Security 2026) achieves FIPS-compatible threshold ML-DSA via replicated secret sharing in 3 online rounds. Hermine [BCdP⁺26] provides lattice-based FROST-like threshold signatures with non-interactive identifiable aborts and proactive security in **1 online round**; it is not FIPS 204 compatible (Raccoon-based, ≈ 11 – 15 KB signatures). Quorus [BdCE⁺26] achieves 2 online rounds (16–29 concrete broadcast rounds) with no identifiable aborts. Trilithium [DKLS25] provides UC-style (AP-secure) distributed signing for two parties (Phone + Server) using a trusted correlated randomness provider; 14 online rounds per attempt (≈ 60 expected with rejection retries). *TALUS improves upon all of these simultaneously: TALUS-TEE achieves 1 online round, full FIPS 204 compatibility, and arbitrary T -of- N with no honest-majority requirement. TALUS-MPC achieves the same online-round and FIPS goals; the Dishonest Majority Barrier (Theorem 27) shows that $N \geq 2T - 1$ is necessary for any single-round Shamir-based carry protocol when $T \geq 3$, justifying the honest-majority condition as a structural constraint rather than a design limitation. No prior FIPS-compatible scheme matches the round count of either variant.*

Non-lattice threshold signatures. FROST [KG21] achieves 2-round threshold Schnorr; Lindell’s protocol [Lin21] achieves 2-round threshold ECDSA. TALUS matches the best classical round complexity while producing standard FIPS 204 output; structurally it is the lattice analogue

of FROST (Table 14 in §10.1), with offline carry resolution as the sole algebraic price of the lattice setting.

Security models. We use the game-based EUF-CMA model following [KRT24]. A UC proof for TALUS-TEE in the $(\mathcal{F}_{\text{TEE}}, \mathcal{F}_{\text{RO}})$ -hybrid model [Kat07] is given in Appendix B; it achieves statistical distance 0 against static adversaries (malicious signers reducible to semi-honest via identifiable abort; Theorem 31). A UC proof for TALUS-MPC in the $(\mathcal{F}_{\text{PRF}}, \mathcal{F}_{\text{RO}})$ -hybrid model is given in the same appendix (Theorem 33).

1.6 Paper Organization

Preliminaries (Section 2) fix notation, review ML-DSA, and define the threat models and security goals used throughout. *Lattice Threshold Trilemma* (Section 3) proves that the carry cost is a mathematical necessity for any FIPS-compatible lattice-based threshold scheme. *Boundary Clearance Condition* (Section 4) formalises when the r0-check can be eliminated offline. *Carry Elimination Framework* (Section 5) develops the distributed protocol for computing $\mathbf{w}_1$ without reconstructing $\mathbf{w}$. *Protocol instantiations* (Sections 6–7) present TALUS-TEE and TALUS-MPC; each section is self-contained and may be read independently. *Security* (Section 8) proves EUF-CMA and privacy for both variants; full MPC proofs are in Appendix A. *Implementation and evaluation* (Sections 9–10) provide Rust benchmarks, GCP network experiments, and a discussion of design trade-offs and open problems.

2 Preliminaries

2.1 Notation

We use bold lowercase letters (e.g., $\mathbf{s}$) for vectors and bold uppercase letters (e.g., $\mathbf{A}$) for matrices. For an integer q , write $\mathbb{Z}_q = \mathbb{Z}/q\mathbb{Z}$. The polynomial ring is $R_q = \mathbb{Z}_q[X]/(X^n + 1)$ with $n = 256$. Write $n_\ell = n \cdot \ell = 1280$ and $nk = n \cdot k = 1536$ for the nonce-vector and commitment-vector dimensions (ML-DSA-65: $\ell = 5$, $k = 6$).

For $f \in R_q$, write $\|f\|_\infty = \max_i |f_i|$ with coefficients centred in $(-q/2, q/2]$. For $\mathbf{v} \in R_q^k$, $\|\mathbf{v}\|_\infty = \max_i \|v_i\|_\infty$.

We write $a \xleftarrow{\$} S$ for uniform sampling from a finite set S and $\text{negl}(\kappa)$ for a negligible function in security parameter κ . Throughout, *all modular arithmetic is centred*, i.e., representatives are chosen in $(-q/2, q/2]$ unless stated otherwise.

2.2 ML-DSA (FIPS 204)

ML-DSA [Nat24b] is a lattice-based signature scheme derived from CRYSTALS-Dilithium [DKL⁺18], using the Fiat–Shamir with Aborts paradigm [Lyu12]. Security in the (quantum) random oracle model follows from [KLS18].

Parameters (ML-DSA-65).

- $n = 256$, $q = 8,380,417 = 2^{23} - 2^{13} + 1$
- $k = 6$, $\ell = 5$: matrix dimensions
- $\eta = 4$: secret key coefficient bound
- $\gamma_1 = 2^{19}$: nonce range
- $\gamma_2 = (q - 1)/32 = 261,888$: rounding granularity
- $\alpha = 2\gamma_2 = 523,776 = (q - 1)/16$: stripe width
- $\tau = 49$: challenge weight ($|c|_0 = \tau$, $c_i \in \{0, \pm 1\}$)
- $\beta = \tau\eta = 196$: response norm bound
- $\omega = 55$: maximum hint weight
- $d = 13$: public key rounding ($\mathbf{t} = \mathbf{t}_1 \cdot 2^d + \mathbf{t}_0$)

Key Generation. Sample $\rho \xleftarrow{\$} \{0, 1\}^{256}$, expand $\mathbf{A} \in R_q^{k \times \ell}$ from ρ . Sample $\mathbf{s}_1 \xleftarrow{\$} \chi_\eta^\ell$, $\mathbf{s}_2 \xleftarrow{\$} \chi_\eta^k$ with $\|\cdot\|_\infty \leq \eta$. Set $\mathbf{t} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2$, decompose $\mathbf{t} = \mathbf{t}_1 \cdot 2^d + \mathbf{t}_0$. Output $\mathbf{pk} = (\rho, \mathbf{t}_1)$, $\mathbf{sk} = (\rho, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_0)$.

Signing (simplified). Sample nonce $\mathbf{y} \xleftarrow{\$} [-\gamma_1 + 1, \gamma_1]^{n_\ell}$. Compute $\mathbf{w} = \mathbf{A}\mathbf{y}$, decompose $(\mathbf{w}_1, \mathbf{r}_0) = \text{Decompose}(\mathbf{w}, 2\gamma_2)$. Derive challenge hash $\tilde{c} \leftarrow H(\mu\|\mathbf{w}_1)$, $c \leftarrow \text{SampleInBall}(\tilde{c})$. Set $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$. Abort (restart) if $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta$ or $\|\mathbf{r}_0 - c\mathbf{s}_2\|_\infty \geq \gamma_2 - \beta$. Compute hint $\mathbf{h} = \text{MakeHint}(-c\mathbf{s}_2, \mathbf{w} - c\mathbf{s}_2 + c\mathbf{t}_0, 2\gamma_2)$. Output $\sigma = (\tilde{c}, \mathbf{z}, \mathbf{h})$.

Verification. Expand c from $\tilde{c}$. Compute $\mathbf{w}'_1 = \text{UseHint}(\mathbf{h}, \mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d, 2\gamma_2)$. Accept iff $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$ and $\tilde{c} = H(\mu\|\mathbf{w}'_1)$.

Key identity. The correctness of hint-based verification follows from:

$$\mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d = \mathbf{w} - c\mathbf{s}_2 + c\mathbf{t}_0,$$

since $\mathbf{t}_1 \cdot 2^d = \mathbf{t} - \mathbf{t}_0 = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2 - \mathbf{t}_0$.

Key identity for α . The modulus $q = 2^{23} - 2^{13} + 1$ and $\alpha = (q - 1)/16$ satisfy:

$$\alpha \cdot 16 = q - 1 \equiv -1 \pmod{q}, \quad \text{hence} \quad \alpha^{-1} \equiv -16 \pmod{q}.$$

This identity is central to the Carry Elimination Framework (Section 5).

2.3 Shamir Secret Sharing

Shamir's (T, N) -threshold secret sharing [Sha79] distributes a secret $s \in \mathbb{Z}_q$ among N parties so that any T can reconstruct s but any $T - 1$ learn nothing. For vectors $\mathbf{s} \in R_q^\ell$, sharing proceeds coefficient-by-coefficient.

Sharing. Choose random $f(X) = s + r_1X + \dots + r_{T-1}X^{T-1} \in \mathbb{Z}_q[X]$ and give share $s_i = f(i)$ to party i .

Reconstruction. For signing set S with $|S| \geq T$: $s = \sum_{i \in S} \lambda_i s_i \pmod{q}$ where the Lagrange coefficients are $\lambda_i = \prod_{j \in S, j \neq i} j(j - i)^{-1} \pmod{q}$.

2.4 Shamir Nonce DKG

The Shamir Nonce DKG [Kao26] generates the signing nonce $\mathbf{y}$ as a Shamir sharing, matching the structure of the long-term key $\mathbf{s}_1$.

Offline preprocessing. Each party $h \in S$ samples a degree- $(T - 1)$ polynomial $f_h(X) = y_h + a_{h,1}X + \dots + a_{h,T-1}X^{T-1}$ with $y_h \xleftarrow{\$} [-\gamma_1/|S| + 1, \gamma_1/|S|]^{n_\ell}$ and $a_{h,j} \xleftarrow{\$} \mathbb{Z}_q^{n_\ell}$ for $j \geq 1$. Party h sends the evaluation $f_h(i)$ to party i over a secure channel.

Nonce share. Party i 's nonce share is $\mathbf{y}_i = \sum_{h \in S} f_h(i) \pmod{q}$.

Reconstruction. The aggregate nonce is $\mathbf{y} = \sum_{i \in S} \lambda_i \mathbf{y}_i = F(0)$ where $F = \sum_h f_h$ has degree $T - 1$ and constant term $\mathbf{y} = \sum_h y_h$.

Privacy. Adversary with $T - 1$ corrupted parties holds $T - 1$ evaluations of a degree- $(T - 1)$ polynomial, leaving one degree of freedom. The constant term $\mathbf{y}$ is bounded in $[-\gamma_1 + 1, \gamma_1]^{n_\ell}$ but the higher coefficients $a_{h,j}$ are uniform over $\mathbb{Z}_q^{n_\ell}$. This gives the honest party's nonce share $\mathbf{y}_{i^*} = F(i^*)$ high conditional min-entropy (bounded away from $\mathbb{Z}_q^{n_\ell}$ -uniform only by the bounded constant term) [Kao26].

Irwin-Hall distribution. The aggregate nonce $\mathbf{y} = \sum_{h \in S} y_h$ is a sum of $|S|$ independent bounded uniform draws and follows an Irwin-Hall distribution $\text{IH}(\gamma_1, |S|)$ per coefficient. This has greater density near zero than a single uniform draw, which (for $|S| \leq 128$) increases the z-bound acceptance rate slightly [Kao26].

2.5 Security Definitions

Definition 1 ((T, N) -Threshold Signature). A (T, N) -threshold signature scheme Π with algorithms (KeyGen, Sign, Verify):

- $\text{KeyGen}(1^\kappa, N, T) \rightarrow (\text{pk}, \{\text{sk}_i\}_{i \in [N]})$
- $\text{Sign}(\{\text{sk}_i\}_{i \in S}, \mu) \rightarrow \sigma$ for $|S| \geq T$
- $\text{Verify}(\text{pk}, \mu, \sigma) \rightarrow \{0, 1\}$

Definition 2 (EUF-CMA). Π is EUF-CMA secure following [KRT24]: for any PPT adversary $\mathcal{A}$ that (i) receives pk , (ii) adaptively corrupts up to $|C| < T$ parties, and (iii) queries a signing oracle $\mathcal{O}_{\text{Sign}}$ polynomially many times (each on a chosen message m_j with a chosen signing set $S_j \ni C$) and a random oracle H :

$$\Pr[\mathcal{A}^{\mathcal{O}_{\text{Sign}}, H} \rightarrow (m^*, \sigma^*) : m^* \notin \{m_j\} \wedge \text{Verify}(\text{pk}, m^*, \sigma^*) = 1] \leq \text{negl}(\kappa).$$

Definition 3 (Privacy). Π provides *statistical privacy* (resp. *computational privacy*) if for any $|C| < T$ corrupted parties there exists a PPT simulator Sim such that the statistical distance (resp. computational indistinguishability) between the real view $\text{View}_C(\text{Sign}(\{\text{sk}_i\}_{i \in S}, \mu))$ and the simulated view $\text{Sim}(\text{pk}, \sigma)$ is 0 (resp. $\text{negl}(\kappa)$), where σ is the output signature. TALUS-TEE achieves statistical privacy (SD = 0; Theorem 24). TALUS-MPC achieves computational privacy under PRF security (Theorem 26).

2.6 Beaver Triples and MPC Primitives

For TALUS-MPC (Section 7), parties use *Beaver multiplication triples* [Bea92] to evaluate bilinear functions of their secret shares. A Beaver triple (a, b, c) with $c = a \cdot b$ allows two parties to compute a product xy using one round of interaction, given pre-shared triples generated offline.

A *pseudorandom function* (PRF) family $\{F_k\}_{k \in \{0,1\}^\kappa}$ is (t, ε) -secure if for all t -time distinguishers D , $|\Pr[D^{F_k(\cdot)} = 1] - \Pr[D^{f(\cdot)} = 1]| \leq \varepsilon$, where $k \leftarrow \{0,1\}^\kappa$ is uniformly random and f is a uniformly random function. We write $\text{Adv}_F^{\text{PRF}}(\mathcal{B})$ for the PRF advantage of a PPT adversary $\mathcal{B}$.

We generate Beaver triples from correlated pseudorandom seeds (CPRF) following the approach of [BCGI18], which reduces offline communication to $O(1)$ seeds per triple.

Threat models. We use three standard adversarial models, listed from weakest to strongest:

- *Semi-honest*: corrupted parties follow the protocol but inspect their local view; security holds against passive eavesdropping.
- *Malicious*: corrupted parties may deviate arbitrarily.
- *Malicious + Identifiable Abort (IA)*: malicious parties may deviate or abort, but any deviation causing a protocol failure triggers a blame mechanism that publicly identifies the deviating party.

TALUS-TEE is proven secure against malicious signers (the TEE coordinator enforces correct protocol execution; the Blame protocol identifies deviators with probability 1). TALUS-MPC achieves malicious-IA for $T \geq 2$.

Remark 1 (Security Model). TALUS-MPC (Section 7) is proven secure in the *malicious + identifiable-abort* model for $T \geq 2$ (Theorems 25 and 26). Malicious deviations are identified via the Blame protocol (Theorem 20); no SPDZ-style MACs are required.

Table 2: Commitment candidates for lattice-based threshold signing. No single candidate achieves all three properties.

Commitment	Homomorphic	Hiding	Binding
$\mathbf{A}\mathbf{y}$	✓	×	✓
$\text{HighBits}(\mathbf{A}\mathbf{y})$	×	✓	✓
$\mathbf{A}'\mathbf{y}, m < \ell$	✓	✓	×
$DL: g^k$	✓	✓	✓

3 The Lattice Threshold Trilemma

In FROST [KG21], threshold signing reduces to Shamir interpolation of nonce shares followed by a single group exponentiation. The map $k \mapsto g^k$ is a group isomorphism of order q , simultaneously homomorphic, hiding, and binding. This section proves that no analogous map exists in the lattice setting: *no group homomorphism whatsoever* from the nonce space $(R_q^\ell, +)$ into any finite abelian group can simultaneously be hiding and binding. This makes the carry cost (Sections 5–7) a mathematical necessity rather than a protocol artifact.

3.1 DL vs. Lattice Commitments

In Schnorr-type threshold signatures, the nonce commitment $R = g^k$ enjoys three properties simultaneously: (i) homomorphic ($g^{k_1} \cdot g^{k_2} = g^{k_1+k_2}$), (ii) hiding (DDH), (iii) binding (DL). In ML-DSA, the natural commitment is $\mathbf{w} = \mathbf{A}\mathbf{y} \in R_q^k$. Table 2 evaluates three candidates.

Candidate 1: $f(\mathbf{y}) = \mathbf{A}\mathbf{y}$ is additive and binding (M-SIS), but when $k \geq \ell$ the left inverse $\mathbf{A}^+ = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T$ recovers $\mathbf{y}$ exactly, violating hiding. **Candidate 2:** $\text{HighBits}(\mathbf{A}\mathbf{y})$ is hiding (M-LWR [BPR12]) and binding, but HighBits is not additive; low-order parts may wrap past an α -boundary. **Candidate 3:** $\mathbf{A}'\mathbf{y}$ with $m < \ell$ is additive and hiding (non-trivial kernel), but the kernel destroys binding.

These failures are not coincidental. The following theorem shows that they reflect a fundamental impossibility: *no* group homomorphism from the nonce space can simultaneously achieve all three properties.

3.2 The Trilemma Theorem

Theorem 1 (Lattice Threshold Trilemma). *Let q be prime, n a power of 2, and $R_q = \mathbb{Z}_q[X]/(X^n + 1)$.*

- (1) **No homomorphic hiding-and-binding commitment.** *No group homomorphism $f: (R_q^\ell, +) \rightarrow (G, \cdot)$ into any finite abelian group can simultaneously be hiding and perfectly binding (see Remark 3 for why computational binding is insufficient in threshold signing). This rules out all R_q -linear maps, all $\mathbb{Z}_q$ -linear maps, and any additive function whatsoever.*
- (2) **No homomorphic rounding.** *$\text{HighBits}: \mathbb{Z}_q \rightarrow \{0, \dots, (q-1)/\alpha - 1\}$ is not a group homomorphism (since $\mathbb{Z}_q$ is simple), so composing any map with coefficient-wise HighBits does not yield a homomorphic function.*

Consequently, no single function achieves all three properties (homomorphism, hiding, binding) that make FROST’s nonce commitment g^k algebraically free.

Proof. See Appendix G. □

Remark 2 (Role of prime q). Part 1 relies on q being prime: this makes $\mathbb{Z}_q$ a prime field, so every additive map between $\mathbb{Z}_q$ -vector spaces is automatically $\mathbb{Z}_q$ -linear (see proof in Appendix G). Part 2 relies on $(\mathbb{Z}_q, +)$ being a simple group (no proper non-trivial subgroups), forcing every group homomorphism $\mathbb{Z}_q \rightarrow \mathbb{Z}_M$ with $M < q$ to be trivial. If $\alpha \mid q$, the canonical projection

$\mathbb{Z}_q \twoheadrightarrow \mathbb{Z}_{q/\alpha}$ would be a surjective group homomorphism, and **HighBits** could be realised as a homomorphism. ML-DSA mandates $q = 8,380,417$ (prime) with $(q - 1)/\alpha = 16$, so $\alpha \nmid q$.

Remark 3 (Perfect vs. computational binding). Lattice commitments such as $\mathbf{A}'\mathbf{y}$ with $m < \ell$ are homomorphic, hiding, and *computationally* binding (under M-SIS, where finding a short kernel element is hard). Theorem 1 uses *perfect* binding (injectivity). In the threshold signing context, computational binding is insufficient: a malicious signer choosing its nonce share $\mathbf{y}_i$ is not restricted to short vectors, so *any* kernel element can be added to the share without detection. The binding requirement for nonce commitments is therefore information-theoretic, not computational.

Remark 4 (Why discrete-log escapes the trilemma). In the DL setting, the map $k \mapsto g^k$ is a group isomorphism from $(\mathbb{Z}_q, +)$ to $(\langle g \rangle, \cdot)$. Abstractly, the target group is isomorphic to $\mathbb{Z}_q$, so the “left inverse” exists as an abstract map (the discrete logarithm). The crucial difference is *representation*: in the lattice setting, the target group $\mathbb{Z}_q^d$ is presented as a vector space where Gaussian elimination runs in polynomial time; in the DL setting, the isomorphic copy of $\mathbb{Z}_q$ is hidden behind the group exponentiation, making inversion (DLP) computationally hard.

Corollary 1 (Carry Resolution is Necessary). *Any threshold ML-DSA protocol that Shamir-shares the nonce and computes $\mathbf{w}_1 = \text{HighBits}(\mathbf{A}\mathbf{y})$ must adopt one of: (i) a trusted coordinator that reconstructs $\mathbf{w}$ in the clear (achievable with a TEE), (ii) a multi-round distributed carry resolution (offline or online), or (iii) online rejection sampling with abort-and-retry (Fiat–Shamir with aborts). TALUS-TEE adopts (i); TALUS-MPC implements (ii) as offline preprocessing in $\max(3, \lceil \log_2(N/2) \rceil + 2)$ rounds (Section 5); Mithril [CdPE⁺ 26] adopts (iii). Part 1 of the Trilemma rules out every group homomorphism from $(R_q^\ell, +)$ as a hiding-and-binding commitment, and Part 2 rules out homomorphic rounding.*

3.3 Consequences for Protocol Design

The trilemma implies that threshold signers who Shamir-share the nonce $\mathbf{y}$ and reconstruct $\mathbf{w} = \sum_i \lambda_i \mathbf{A}\mathbf{y}_i$ must resolve a *carry* when passing from individual **HighBits** values to **HighBits** of the sum.

Carry probability. In a naive protocol (without BCC filtering), for $T \geq 3$ the sum of T independent uniform low-bits values wraps past an α -boundary with probability approaching $1/2$ per coefficient. With $kn = 1536$ coefficients (ML-DSA-65), the expected carry count is ≈ 768 , far exceeding the hint budget $\omega = 55$. (Under BCC filtering, parties only proceed when all coefficients lie $> \beta$ from a boundary, reducing the per-coefficient carry probability to $\approx 2.5\%$; the expected carry count then falls well within ω .)

Carry information content. Each carry $\delta_j \in \{-1, 0, +1\}$ contributes $\lceil \log_2 3 \rceil = 2$ bits. Across kn coefficients (1536 for ML-DSA-65), the total carry information is $2kn$ bits (3072 bits = 384 bytes). By Part 1 of the Trilemma, no additive sharing scheme can avoid these carries, so any threshold ML-DSA protocol that Shamir-shares the nonce must exchange at least $\Omega(kn)$ bits of non-algebraic information per signing session, beyond the response shares $\mathbf{z}_i$.

Key takeaway. *The carry is not a protocol artifact; it is a mathematical necessity.* Sections 4–5 show how to handle it at minimum cost. The carry–privacy duality (Appendix G.2) formalises why the carry must be computed *without* revealing $S_j \bmod \alpha$: knowledge of the LowBits remainder enables single-signature recovery of $\mathbf{s}_2$.

4 Boundary Clearance Condition

The central observation driving TALUS is that the secret-dependent correction term $\mathbf{cs}_2$ present in single-party ML-DSA signing need not be computed in the threshold setting for a substantial fraction of signing attempts. We formalise this observation as the *Boundary Clearance Condition* and derive exact success probabilities for ML-DSA-65.

4.1 Why cs_2 Appears Necessary

In single-party ML-DSA [Nat24b], the signing procedure computes:

1. Sample nonce $\mathbf{y} \xleftarrow{\$} [-\gamma_1 + 1, \gamma_1]^{n_\ell}$.
2. Compute commitment $\mathbf{w} = \mathbf{A}\mathbf{y} \bmod q$.
3. Decompose: $(\mathbf{w}_1, \mathbf{r}_0) = \text{Decompose}(\mathbf{w}, 2\gamma_2)$, so $\mathbf{w} = \mathbf{w}_1 \cdot 2\gamma_2 + \mathbf{r}_0$ with $\|\mathbf{r}_0\|_\infty \leq \gamma_2$.
4. Compute challenge hash $\tilde{c} \leftarrow H(\mu \parallel \mathbf{w}_1)$, $c \leftarrow \text{SampleInBall}(\tilde{c})$.
5. Compute response $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$.
6. **r0-check:** abort and restart if $\|\mathbf{r}_0 - c\mathbf{s}_2\|_\infty \geq \gamma_2 - \beta$.
7. Compute hint $\mathbf{h} = \text{MakeHint}(-c\mathbf{s}_2, \mathbf{w} - c\mathbf{s}_2 + c\mathbf{t}_0, 2\gamma_2)$.

The purpose of the r0-check (step 6) is to ensure that the hint $\mathbf{h}$ can be represented with at most $\omega = 55$ nonzero entries. The hint encodes which coefficients of $\mathbf{w}$ changed their high-order index when the secret-dependent shift cs_2 was applied.

In threshold signing, s_2 is shared among N parties. All prior threshold ML-DSA constructions must therefore either: (a) run a multi-party computation to evaluate cs_2 (Hermine [BCdP⁺26], Quorus [BdCE⁺26]), or (b) exploit an algebraic share structure (replicated secret sharing) so that each party can compute its contribution to cs_2 locally, combined with online rejection sampling (Mithril [CdPE⁺26]).

Our contribution. We prove that no rounding boundary is crossed, and hence $\mathbf{w}_1$ is unchanged by cs_2 , whenever $\mathbf{r}_0$ is *bounded away* from the modular boundaries by more than $\beta = \tau\eta$. This condition is checkable from the public value $\mathbf{w}$ alone, and eliminates the need to compute cs_2 in the threshold setting.

4.2 Boundary Distance and the BCC

Definition 4 (Boundary Distance). For $\mathbf{r} \in R_q^k$, the *boundary distance* of coefficient $r_j \in (-\gamma_2, \gamma_2]$ with respect to $\alpha = 2\gamma_2$ is:

$$d_j(\mathbf{r}) := \gamma_2 - |r_j| \in [0, \gamma_2].$$

We say coefficient j *clears the boundary* if $d_j(\mathbf{r}) > \beta = \tau\eta$.

Geometrically, $d_j(\mathbf{r})$ measures how far r_j is from the nearest rounding boundary $\pm\gamma_2$ within its stripe. A coefficient clears the boundary when it lies in the “interior” interval $(-\gamma_2 + \beta, \gamma_2 - \beta)$, at distance greater than β from either boundary.

Definition 5 (Boundary Clearance Condition). Let $(\mathbf{w}_1, \mathbf{r}_0) = \text{Decompose}(\mathbf{w}, 2\gamma_2)$ for $\mathbf{w} \in R_q^k$. The *Boundary Clearance Condition* $\text{BCC}(\mathbf{w})$ holds if and only if every coefficient clears the boundary:

$$\text{BCC}(\mathbf{w}) :\iff \forall j \in [nk] : d_j(\mathbf{r}_0) > \beta.$$

Equivalently, $\text{BCC}(\mathbf{w})$ holds iff $\|\mathbf{r}_0\|_\infty < \gamma_2 - \beta$ (see Figure 2).

4.3 Safety Theorem

Theorem 2 (Safety Theorem). Let $\mathbf{w} = \mathbf{A}\mathbf{y} \bmod q$ and let $(\mathbf{w}_1, \mathbf{r}_0) = \text{Decompose}(\mathbf{w}, 2\gamma_2)$. Suppose $\text{BCC}(\mathbf{w})$ holds. Let $c \in \text{SampleInBall}(\tau)$ be any challenge, $\mathbf{s}_2$ any key polynomial with $\|\mathbf{s}_2\|_\infty \leq \eta$, and $\mathbf{t}_0$ the low-bit key component satisfying $\|\mathbf{t}_0\|_\infty \leq 2^{d-1}$ (guaranteed by FIPS 204 Power2Round at key generation). Then:

- (i) (No boundary crossing) $\|\mathbf{r}_0 - c\mathbf{s}_2\|_\infty < \gamma_2$, i.e. subtracting cs_2 from $\mathbf{w}$ does not cross any rounding stripe boundary.
- (ii) (cs_2 erasure) $\text{HighBits}(\mathbf{w} - c\mathbf{s}_2, 2\gamma_2) = \text{HighBits}(\mathbf{w}, 2\gamma_2) = \mathbf{w}_1$.
- (iii) (Hint computable without $\mathbf{s}_2$) Define $\mathbf{r} = \mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d \pmod{q}$, computable from $(\mathbf{pk}, \mathbf{z}, c)$ alone. Set $h_j = \mathbf{1}[\text{HighBits}(r_j, 2\gamma_2) \neq (w_1)_j]$. Then $\text{UseHint}(h_j, r_j, 2\gamma_2) = (w_1)_j$ (so FIPS 204 verification accepts).

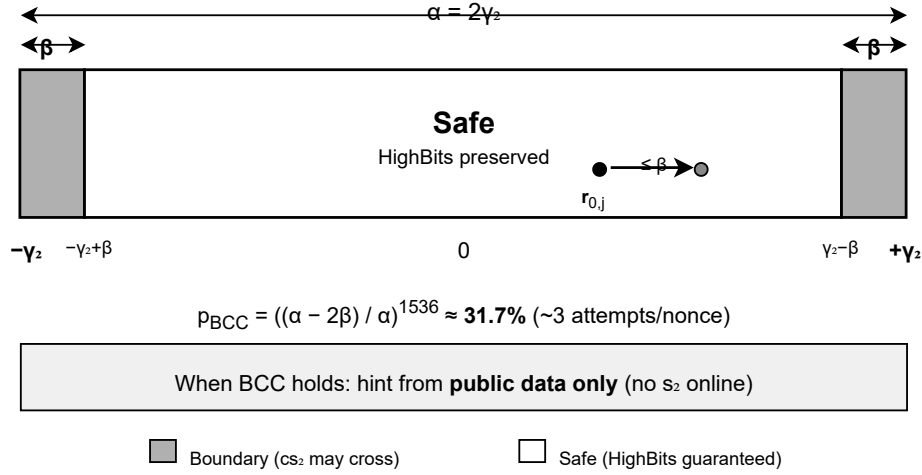


Figure 2: Geometry of the Boundary Clearance Condition. Each coefficient $r_{0,j}$ of $\mathbf{r}_0$ lives in a stripe $[-\gamma_2, +\gamma_2]$ of width $\alpha = 2\gamma_2$. The gray *boundary zones* of width β at each end are where cs_2 may push $r_{0,j}$ across a rounding boundary; the white *safe zone* guarantees **HighBits** is preserved. BCC holds when every coefficient lies in the safe zone ($\|\mathbf{r}_0\|_\infty < \gamma_2 - \beta$), occurring with probability $p_{\text{BCC}} \approx 31.7\%$ per nonce.

Note: $\mathbf{r}$ analytically equals $\mathbf{w} - cs_2 + ct_0 \pmod{q}$, so $\mathbf{h}$ implicitly depends on s_2 through $\mathbf{r}$. The claim is that no explicit knowledge of s_2 is required: $\mathbf{r}$ is already determined by the public quantities $(pk, \mathbf{z}, c)$, so $\mathbf{h}$ is too.

Proof. Fix any coefficient index j . Write $r_j = (\mathbf{r}_0)_j$ and $e_j = (cs_2)_j$.

Bound on e_j . The challenge $c = \text{SampleInBall}(\tau)$ has exactly τ nonzero coefficients, each in $\{-1, +1\}$. Hence $e_j = (cs_2)_j = \sum_k c_k (s_2)_{j-k}$ (negacyclic convolution mod $X^n + 1$). By a standard ℓ^∞ convolution bound, $|e_j| \leq \tau \cdot \eta = \beta$.

Proof of (i). Since $\text{BCC}(\mathbf{w})$ holds, $|r_j| < \gamma_2 - \beta$. Since $|e_j| \leq \beta$, by the triangle inequality:

$$|r_j - e_j| \leq |r_j| + |e_j| < (\gamma_2 - \beta) + \beta = \gamma_2.$$

Hence no rounding boundary $\pm\gamma_2$ is crossed when subtracting e_j from w_j .

Remark 5. The BCC alone gives $|r_j - e_j| < \gamma_2$, not the tighter bound $< \gamma_2 - \beta$ that appears in FIPS 204’s r0-check. In TALUS, we never perform the FIPS 204 r0-check; instead, we use part (ii) (high bits are unchanged) and part (iii) (hint is public). The crucial property is the absence of any boundary crossing, which part (i) establishes.

Proof of (ii). Since $\text{BCC}(\mathbf{w})$ holds, $r_j = \text{LowBits}(w_j, 2\gamma_2) \in (-\gamma_2 + \beta, \gamma_2 - \beta)$. Subtracting e_j with $|e_j| \leq \beta$ gives $r_j - e_j \in (-\gamma_2, \gamma_2)$, which lies within the same residue stripe $(-\gamma_2, \gamma_2]$ as r_j . A rounding stripe boundary (a multiple of $\alpha = 2\gamma_2$) is crossed only when the centred residue leaves $(-\gamma_2, \gamma_2]$; since $r_j - e_j \in (-\gamma_2, \gamma_2)$ this does not occur. Hence:

$$\text{HighBits}(w_j - e_j, 2\gamma_2) = \text{HighBits}(w_j, 2\gamma_2) = (w_1)_j.$$

Proof of (iii). Define $r_j := (\mathbf{A}\mathbf{z} - ct_1 \cdot 2^d)_j$ and $h_j := \mathbf{1}[\text{HighBits}(r_j, 2\gamma_2) \neq (w_1)_j]$.

r_j is public. Since $\mathbf{z} = \mathbf{y} + cs_1$ and $t_1 \cdot 2^d = \mathbf{A}s_1 + s_2 - t_0$, we have $r_j = (w - cs_2 + ct_0)_j$ analytically. However, the computation $\mathbf{A}\mathbf{z} - ct_1 \cdot 2^d$ uses only $(pk, \mathbf{z}, c)$, all of which are public, without requiring s_2 as an explicit input. The value of r_j does implicitly depend on s_2 through the

identity $r_j = (w - cs_2 + ct_0)_j$, but this dependence is irrelevant because r_j is already determined by public quantities.

UseHint correctness. We verify that $\text{UseHint}(h_j, r_j, 2\gamma_2) = (w_1)_j$, which is what FIPS 204 verification requires.

- If $h_j = 0$: $\text{HighBits}(r_j) = (w_1)_j$ by definition. $\text{UseHint}(0, r_j) = \text{HighBits}(r_j) = (w_1)_j$. ✓
- If $h_j = 1$: $\text{HighBits}(r_j) \neq (w_1)_j$ by definition. Since $r_j = (w - cs_2 + ct_0)_j$ and, by part (ii), $\text{HighBits}((w - cs_2)_j) = (w_1)_j$, the only source of the discrepancy is $(ct_0)_j$ pushing r_j across one rounding boundary. By the theorem hypothesis $\|t_0\|_\infty \leq 2^{d-1}$, we get $|(ct_0)_j| \leq \tau \cdot 2^{d-1} = 49 \cdot 4096 = 200,704 < \gamma_2 = 261,888$ (ML-DSA-65 numerics), so at most one boundary is crossed, giving $|\text{HighBits}(r_j) - (w_1)_j| = 1$. The UseHint algorithm adjusts by ± 1 based on $\text{sgn}(\text{LowBits}(r_j))$: a crossing from above (entering a lower stripe) has $\text{LowBits}(r_j) < 0$, so UseHint subtracts 1; a crossing from below has $\text{LowBits}(r_j) > 0$, so UseHint adds 1. In each case, $\text{UseHint}(1, r_j) = (w_1)_j$. ✓

Hence the coordinator, without knowing s_2 , computes $\mathbf{r}$ from public data and sets h_j so that $\text{UseHint}(h_j, r_j) = (w_1)_j$. The resulting signature passes FIPS 204 verification. $\square$

Remark 6 (Hint Weight Under BCC). The hint $\mathbf{h}$ is computable from $(\mathbf{pk}, \mathbf{z}, c, \mathbf{w}_1)$ without explicit knowledge of s_2 . Its value does implicitly depend on s_2 through $\mathbf{r} = \mathbf{w} - cs_2 + ct_0$, but this is immaterial since $\mathbf{r} = \mathbf{Az} - ct_1 \cdot 2^d$ is already public.

The hint is generally nonzero even under BCC. The shift ct_0 has magnitude up to $\tau \cdot 2^{d-1} = 49 \cdot 4096 = 200,704 \approx 0.38\alpha$, which can push $\mathbf{r}$ across a rounding stripe boundary (independently of whether cs_2 does). Monte Carlo experiments measure:

$$\mathbb{E}[\text{wt}(\mathbf{h}) \mid \text{BCC}(\mathbf{w})] \approx 38, \quad \Pr[\text{wt}(\mathbf{h}) \leq \omega = 55 \mid \text{BCC}(\mathbf{w})] \approx 99.7\%.$$

The hint-weight bound $\text{wt}(\mathbf{h}) \leq \omega$ holds with high probability; it is not identically zero. $\square$

4.4 Corollaries

Corollary 2 (cs_2 Erasure). *When $\text{BCC}(\mathbf{w})$ holds, no party needs to contribute to the computation of cs_2 . The coordinator can assemble $\sigma = (\tilde{c}, \mathbf{z}, \mathbf{h})$ without access to any secret share of s_2 , because $\text{HighBits}(\mathbf{w} - cs_2) = \mathbf{w}_1$ by the Safety Theorem (part (ii)).*

Corollary 3 (Hint From Public Data). *When $\text{BCC}(\mathbf{w})$ holds, the hint $\mathbf{h}$ is computable from $(\mathbf{w}_1, \mathbf{z}, \tilde{c}, \mathbf{pk})$ alone, without any secret key material. Specifically, the coordinator computes $\mathbf{r} = \mathbf{Az} - ct_1 \cdot 2^d \pmod{q}$ (all public) and sets $h_j = \mathbf{1}[\text{HighBits}(r_j) \neq (w_1)_j]$. The resulting hint satisfies $\text{wt}(\mathbf{h}) \leq \omega$ with probability $\geq 99.7\%$ under BCC, confirmed over 10^5 trials (Remark 6). The $\leq 0.3\%$ failure probability is handled by discarding the nonce and retrying preprocessing.*

Corollary 4 (Self-Verification). *Each signer i can verify locally, from the public nonce commitment $\mathbf{w}_1$, whether $\text{BCC}(\mathbf{w})$ holds. If it does, party i proceeds to send its response $\mathbf{z}_i = \mathbf{y}_i + cs_{1,i}$ without waiting for information about s_2 .*

Corollary 5 (One-Round Online Signing Under BCC). *In the TALUS-TEE and TALUS-MPC protocols (Sections 6 and 7), when $\text{BCC}(\mathbf{w})$ holds, online signing requires exactly one broadcast round: parties send $\mathbf{z}_i = \mathbf{y}_i + cs_{1,i}$, and the coordinator assembles $\mathbf{z} = \sum_i \lambda_i \mathbf{z}_i$, computes the hint $\mathbf{h}$ from the public value $\mathbf{Az} - ct_1 \cdot 2^d$ (no secret needed, by Corollary 3), and outputs $\sigma = (\tilde{c}, \mathbf{z}, \mathbf{h})$.*

4.5 Per-Attempt BCC Pass Probability

The nonce $\mathbf{y}$ is drawn from $[-\gamma_1 + 1, \gamma_1]^{n_\epsilon}$ (or from an Irwin-Hall distribution in the threshold setting, Section 2.4). The low-order residual $\mathbf{r}_0 = \text{LowBits}(\mathbf{Ay}, 2\gamma_2)$ is approximately uniform over $(-\gamma_2, \gamma_2]$ per coefficient, since $\mathbf{A}$ is a pseudorandom matrix and the modular reduction acts as a mixing step.

Lemma 1 (Per-Coefficient BCC Probability). *Under the heuristic that each coefficient $r_j = (\mathbf{r}_0)_j$ is independently and approximately uniformly distributed over $(-\gamma_2, \gamma_2]$ (standard in lattice*

signature analyses [DKL⁺18, Nat24b]; validated empirically to $< 0.3\%$ error in Theorem 3), the per-coefficient clearance probability is:

$$p_{\text{coeff}} := \Pr[d_j > \beta] = \frac{2(\gamma_2 - \beta)}{2\gamma_2} = 1 - \frac{\beta}{\gamma_2} = 1 - \frac{2\beta}{\alpha}.$$

For ML-DSA-65 ($\beta = 196$, $\gamma_2 = 261,888$, $\alpha = 523,776$):

$$p_{\text{coeff}} = 1 - \frac{196}{261888} \approx 0.999252.$$

Theorem 3 (BCC Pass Rate). *The probability that a single signing attempt satisfies $\text{BCC}(\mathbf{w})$, over the randomness of the nonce $\mathbf{y}$, is:*

$$p_{\text{BCC}} = p_{\text{coeff}}^{nk} = \left(1 - \frac{2\beta}{\alpha}\right)^{nk}.$$

For ML-DSA-65 ($n = 256$, $k = 6$, so $nk = 1,536$ coefficients):

$$p_{\text{BCC}} \approx (0.999252)^{1536} \approx e^{-1536 \times 0.000748} \approx e^{-1.149} \approx 0.3166.$$

Confirmed: 100,000-trial Monte Carlo gives $p_{\text{BCC}} = 0.3138$ (theory 0.3166; $|\Delta| < 0.003$). See `experiments/bcc_rate.py`.

Remark 7 (BCC Probability Is Independent of Threshold). Because $\mathbf{w} = \mathbf{A}\mathbf{y}$ and $\mathbf{y}$ is derived from the Shamir nonce DKG (Section 2.4), the distribution of $\mathbf{w}$ is the same in threshold and single-party settings (the Irwin-Hall distribution of $\mathbf{y}$ has a tighter central core but the same support, and after reduction mod q the high-order residual distribution is approximately uniform regardless of $|S|$). Thus $p_{\text{BCC}} \approx 0.317$ holds for all threshold parameters.

4.6 Why This Wasn’t Found Before

The BCC and its implication (Corollary 2) were not observed in prior work for a structural reason: prior threshold ML-DSA constructions approached the r0-check as a *correctness invariant to be maintained*, not as a *probabilistic condition to be conditionally bypassed*.

Every prior construction [CdPE⁺26, BCdP⁺26, BdCE⁺26, DKLS25] asks: “given that we must compute cs_2 every time, how do we do it cheaply?” TALUS asks instead: “under what conditions can we *prove* that cs_2 has no observable effect on the signature, so that the entire computation becomes unnecessary?”

This reframing reveals the geometric structure that BCC exploits. ML-DSA’s parameters are deliberately chosen so that $\gamma_2 - \beta = 261,692$ is large relative to the full range $\gamma_2 = 261,888$, a “clearance buffer” that occupies $(\gamma_2 - \beta)/\gamma_2 \approx 99.9\%$ of each coefficient’s range. Standard signing uses this buffer to *guarantee* the r0-check passes whenever $\|\mathbf{r}_0\|_\infty < \gamma_2 - \beta$. Prior threshold work observed the same guarantee but did not notice that the condition $\|\mathbf{r}_0\|_\infty < \gamma_2 - \beta$ is itself evaluable without secret information, precisely because it depends only on $\text{LowBits}(\mathbf{A}\mathbf{y})$, a function of the *public* nonce commitment.

The technical feasibility of BCC rests on three conditions that hold simultaneously in the TALUS design:

(1) Offline nonce preprocessing. In TALUS, the nonce $\mathbf{y}$ is generated in an offline preprocessing phase (Section 6.2). This means the coordinator can check $\text{BCC}(\mathbf{A}\mathbf{y})$ during preprocessing, when computation is cheap and latency is amortised, and discard nonces that fail BCC. Each nonce requires ≈ 3 preprocessing attempts ($1/p_{\text{BCC}} \approx 3.15$) to find one satisfying BCC. At a preprocessing cost of roughly $3 \times$ a single ML-DSA signing operation. Concretely, at $T = 3$ (ML-DSA-65), preprocessing three nonces costs $\approx 5.4\text{ms}$ total (Table 8), acceptable for any non-interactive signing pipeline.

(2) The 31.7% pass rate is industrially sufficient. In a deployment where signing is the latency-critical operation, needing ≈ 3 preprocessing nonces per online signing session is acceptable. The nonce DKG runs $O(N^2)$ messages totalling $\approx 3.7\text{KB}$ per party per nonce [Kao26];

preprocessing three nonces amounts to ≈ 11 KB per party per signing session — for a 1 Gbps LAN, this transmits in < 0.1 ms, negligible compared to signing computation.

(3) The $\mathbf{r}_0$ -check is a binary predicate on public data. BCC depends on $\mathbf{r}_0 = \text{LowBits}(\mathbf{A}\mathbf{y})$. In TALUS-TEE, the coordinator knows $\mathbf{y}$ directly (held in the TEE) and computes $\mathbf{r}_0 = \text{LowBits}(\mathbf{A}\mathbf{y})$ locally. In TALUS-MPC, $\mathbf{A}\hat{\mathbf{y}} = \sum_h \mathbf{A}\hat{\mathbf{y}}_h$ is reconstructed from per-party broadcasts of $\mathbf{A}\hat{\mathbf{y}}_h$ during preprocessing. In both cases, the BCC check requires no secret information.

5 Carry Elimination Framework

The BCC (Section 4) eliminates the need to compute $\mathbf{cs}_2$ once the commitment $\mathbf{w} = \mathbf{A}\mathbf{y}$ is established. However, in a threshold setting, computing the commitment hash input $\mathbf{w}_1 = \text{HighBits}(\mathbf{w}, 2\gamma_2)$ still requires knowledge of the aggregate $\mathbf{w}$. In TALUS-MPC, each party h holds only its own additive nonce piece $\hat{\mathbf{w}}_h = \mathbf{A}\hat{\mathbf{y}}_h$ and HighBits does not commute with addition, so carries must be tracked.

The Carry Elimination Framework (CEF) solves this problem by exploiting a structural property of the ML-DSA modulus.

Remark 8 (Reading Guide). This section presents two formulations of the CEF. **Readers interested only in TALUS-MPC should proceed directly to Section 5.6 (§5.6):** the masked broadcast used in TALUS-MPC is self-contained there. Sections 5.3–5.4 (Protocol 4 and the secondary carry analysis) cover the Lagrange case, which is *not* used in TALUS-MPC and is included only as a reference construction for other threshold ML-DSA protocols.

5.1 The Distributed $\mathbf{w}_1$ Problem

In TALUS-TEE, the coordinator holds $\mathbf{y}^{(0)} = \sum_{h \in S} \mathbf{y}_h$ (the aggregate constant term from the Nonce DKG, received in the TEE) and computes $\mathbf{w}_1 = \text{HighBits}(\mathbf{A}\mathbf{y}^{(0)}, 2\gamma_2)$ directly. No distributed computation is required.

In TALUS-MPC (Section 7), the coordinator does not hold $\mathbf{y}^{(0)}$ in clear. Instead, each party $h \in S$ holds its own *additive* nonce contribution $\hat{\mathbf{y}}_h$ (the constant term of its nonce polynomial; see Protocol 13, Step 1). The aggregate nonce is:

$$\hat{\mathbf{w}} = \mathbf{A}\hat{\mathbf{y}} = \mathbf{A} \sum_{h \in S} \hat{\mathbf{y}}_h = \sum_{h \in S} \hat{\mathbf{w}}_h \quad (\text{additive decomposition, unit coefficients}).$$

However, HighBits does not commute with addition:

$$\text{HighBits}(\hat{\mathbf{w}}) \neq \sum_{h \in S} \text{HighBits}(\hat{\mathbf{w}}_h)$$

in general, because rounding stripe boundaries can be crossed when the additive pieces are summed. We call such crossings *carries*.

Remark 9 (Lagrange vs. Additive Decomposition). The aggregate commitment could alternatively be expressed as a Lagrange combination of the *Shamir shares*: $\hat{\mathbf{w}} = \sum_{i \in S} \lambda_i \mathbf{A}\hat{\mathbf{y}}_i$ where $\hat{\mathbf{y}}_i = \sum_{h \in S} g_h(i)$ is party i 's full Shamir nonce share. However, TALUS-MPC computes $\mathbf{w}_1$ during the *offline* phase using the additive decomposition above (each party h broadcasts its own $\hat{\mathbf{w}}_h$ masked; see Section 5.6). The Lagrange approach would require parties to hold their Shamir shares before computing $\mathbf{w}_1$, which introduces additional rounds; the additive approach merges $\mathbf{w}_1$ computation with the nonce DKG. The Lagrange CEF (Protocol 4 below) is presented as a general reference construction; it is *not* used in TALUS-MPC.

The naive approach (compute $\hat{\mathbf{w}}$ explicitly and then take HighBits) requires each party to transmit the full $\hat{\mathbf{w}}_h \in \mathbb{Z}_q^{nk}$, and the coordinator to sum modulo q . CEF avoids reconstructing $\hat{\mathbf{w}}$ as a single value by factoring each $\hat{\mathbf{w}}_h$ into high and low parts and combining them using the masked broadcast of Section 5.6.

5.2 The Key Identity

Lemma 2 (Modular Inverse Identity). *Let $\alpha = 2\gamma_2$. For ML-DSA-65, $q = 16\alpha + 1$, and:*

$$\alpha^{-1} \equiv -16 \pmod{q}.$$

Proof. $\alpha \cdot 16 = (q - 1)/16 \cdot 16 = q - 1 \equiv -1 \pmod{q}$, so $\alpha \cdot (-16) \equiv 1 \pmod{q}$. $\square$

The relation $q = 16\alpha + 1$ has two consequences for carry elimination:

- **HighBits** takes values in $\{0, 1, \dots, 15\}$, so $\mathbf{w}_1$ fits in 4 bits per coefficient.
- $q \equiv 1 \pmod{\alpha}$ (not 0), which introduces a boundary correction when sums wrap modulo q . We address this in both the Lagrange protocol (Protocol 4) and the masked broadcast (Section 5.6).

5.3 Carry Elimination Protocol

Protocol 4 (Distributed $\mathbf{w}_1$ via Carry Elimination (Lagrange case)). **Note:** *This protocol handles the general case where parties hold Lagrange-combined shares. TALUS-MPC uses the additive masked broadcast (Section 5.6) instead; this protocol is included as a reference construction.*

Input: Each party $i \in S$ holds $\mathbf{w}_i \in \mathbb{Z}_q^{nk}$ and the Lagrange coefficients $\{\lambda_i\}_{i \in S}$.

Output: Coordinator obtains $\mathbf{w}_1 = \text{HighBits}(\sum_i \lambda_i \mathbf{w}_i \bmod q, 2\gamma_2)$.

1. Each party i decomposes $(\mathbf{w}_{1,i}, \mathbf{r}_{0,i}) = \text{Decompose}(\mathbf{w}_i, 2\gamma_2)$, so that $\mathbf{w}_i = \mathbf{w}_{1,i} \cdot \alpha + \mathbf{r}_{0,i}$ (over $\mathbb{Z}$, not modulo q) with $-\gamma_2 < (\mathbf{r}_{0,i})_j \leq \gamma_2$ for each j . Party i sends $(\mathbf{w}_{1,i}, \mathbf{r}_{0,i})$ to the coordinator.
2. Coordinator computes the integer Lagrange combinations:

$$\begin{aligned} \mathbf{W}_1 &= \sum_{i \in S} \lambda_i \cdot \mathbf{w}_{1,i} \quad (\text{over } \mathbb{Z}), \\ \mathbf{R} &= \sum_{i \in S} \lambda_i \cdot \mathbf{r}_{0,i} \quad (\text{over } \mathbb{Z}). \end{aligned}$$

Note: $\alpha \mathbf{W}_1 + \mathbf{R} = \sum_i \lambda_i \mathbf{w}_i$ (exact, over $\mathbb{Z}$, before reduction mod q).

3. Coordinator computes:

$$\mathbf{w}_1 = \text{HighBits}((\alpha \mathbf{W}_1 + \mathbf{R}) \bmod q, 2\gamma_2).$$

Lemma 3 (Correctness of CEF). *Protocol 4 computes $\mathbf{w}_1 = \text{HighBits}(\sum_i \lambda_i \mathbf{w}_i \bmod q, 2\gamma_2)$ exactly.*

Proof. By the decomposition identity, $\mathbf{w}_i = \alpha \mathbf{w}_{1,i} + \mathbf{r}_{0,i}$ over $\mathbb{Z}$. Hence $\alpha \mathbf{W}_1 + \mathbf{R} = \sum_i \lambda_i \mathbf{w}_i$ over $\mathbb{Z}$, so $(\alpha \mathbf{W}_1 + \mathbf{R}) \bmod q = \mathbf{w} \bmod q$. The coordinator applies $\text{HighBits}(\cdot, 2\gamma_2)$ to this value, obtaining $\mathbf{w}_1$ exactly. $\square$

Remark 10 (Carries and the Naive Formula). A simpler formula sometimes found in the literature is:

$$\mathbf{w}_1 \stackrel{?}{=} (\mathbf{W}_1 + \mathbf{C}) \bmod 16, \quad \mathbf{C} = \left\lfloor \frac{\mathbf{R} + \gamma_2 - 1}{\alpha} \right\rfloor \quad (1)$$

where $\mathbf{C}$ is the *carry vector* (integer floor division, centred around 0). This formula is correct with probability $1 - O(1/\alpha) \approx 1 - 2 \times 10^{-6}$ per coefficient; it fails at the rare event that $q \equiv 1 \pmod{\alpha}$ causes the reduction $(\alpha \mathbf{W}_1 + \mathbf{R}) \bmod q$ to shift $\mathbf{r}_0$ by ± 1 across the $\pm\gamma_2$ boundary. We experimentally confirmed this failure mode (see below) and adopt the exact formula of Protocol 4 in our implementation. The carry vector $\mathbf{C}$ remains useful for analysing carry distributions in Section 5.4.

5.4 Secondary Carry Analysis (Lagrange Case)

Remark 11 (Scope of This Section). The analysis below applies to the Lagrange CEF protocol (Protocol 4), which is *not* used in TALUS-MPC. In TALUS-MPC's masked broadcast (Section 5.6), parties broadcast unsigned additive pieces $b_h \in [0, \alpha)$, so $\lfloor B/\alpha \rfloor$ (with $B = \sum_h \tilde{b}_h$) is directly public and there is no secondary carry problem. This analysis is retained for reference since Protocol 4 may be useful in other threshold ML-DSA constructions.

Definition 6 (Primary and Secondary Carry). In Protocol 4 (Lagrange case), using the carry formula of Remark 10, define $C_j = \lfloor (R_j + \gamma_2 - 1)/\alpha \rfloor$ for coefficient j , where $R_j = \sum_i \lambda_i r_{0,i,j}$. A *primary carry* is $C_j \in \{-1, 0, +1\}$; a *secondary carry* has $|C_j| \geq 2$.

For $T = 2$ with $S = \{1, 2\}$, the Lagrange coefficients are $\lambda_1 = 2$, $\lambda_2 = -1$ and $R_j = 2r_{0,1,j} - r_{0,2,j}$ takes values in $(-3\gamma_2, 3\gamma_2)$, so $C_j \in \{-1, 0, 1\}$; only primary carries occur. For larger T , the Lagrange coefficients grow exponentially ($\sum_i |\lambda_i| = 2^T - 1$ for $S = \{1, \dots, T\}$), and secondary carries become overwhelmingly probable.

Table 3: Carry distribution for the Lagrange CEF protocol (Protocol 4; *not used in TALUS-MPC*) with $S = \{1, \dots, T\}$ (measured; 50,000 trials $\times$ $nk = 1,536$ coefficients per trial). “Carry range” is the observed support of C_j . “P(secondary)” = $\Pr[|C_j| \geq 2]$.

T	$\sum \lambda_i $	Carry range	$\Pr[C_j = 0]$	P(secondary)
2	3	$[-1, 1]$	0.500	$< 0.001\%$
3	7	$[-3, 3]$	0.296	25.9%
4	15	$[-7, 7]$	0.155	55.6%
5	31	$[-15, 15]$	0.083	75.5%
8	255	$[-121, 120]$	0.012	96.5%

Independently measured (Lagrange case): secondary carry probability is $< 0.1\%$ for $T = 2$ and exceeds 25% for all $T \geq 3$ in Protocol 4. In the TALUS-MPC additive masked broadcast, carries are handled directly via $\lfloor B/\alpha \rfloor$ (public) with no secondary carry issue. See `experiments/carry_identity.py`.

In the Lagrange CEF protocol, since full $\mathbf{r}_{0,i}$ is needed for $T \geq 3$, the protocol transmits the complete decomposition pair $(\mathbf{w}_{1,i}, \mathbf{r}_{0,i})$ for all T . In TALUS-MPC’s masked broadcast (Section 5.6), each party h instead broadcasts the unsigned additive piece $\tilde{b}_h = b_h + \rho_h$ (with $b_h \in [0, \alpha)$), and the coordinator directly computes $\lfloor B/\alpha \rfloor$ from the public sum $B = \sum_h \tilde{b}_h$, avoiding multi-round carry propagation.

5.5 Communication Cost

Each coefficient of $\mathbf{w}_i \in \mathbb{Z}_q$ is 23 bits. After decomposition:

- $\mathbf{w}_{1,i} \in \{0, \dots, 15\}$: **4 bits** per coefficient.
- $\mathbf{r}_{0,i} \in (-\gamma_2, \gamma_2]$: **19 bits** per coefficient (since $2\gamma_2 = 523,776 < 2^{20}$, and values are signed).

The pair $(\mathbf{w}_{1,i}, \mathbf{r}_{0,i})$ costs 23 bits per coefficient, the same bit-count as the original $\mathbf{w}_i \in \mathbb{Z}_q$. The CEF therefore provides no communication savings versus sending $\mathbf{w}_i$ directly, but achieves the crucial property that $\text{HighBits}(\mathbf{w})$ is computable from the transmitted values *without ever reconstructing $\mathbf{w}$ as a single value*, which cleanly separates the round-1 broadcast from the response assembly.

For $nk = 1,536$ coefficients:

$$\text{CEF payload per party} = 1536 \times 23 \text{ bits} = 4,416 \text{ bytes} \approx 4.3 \text{ KB}.$$

Carry identity confirmed exact (100% agreement) for all tested thresholds $T \in \{2, 3, 4, 5, 8\}$ over 100,000 trials $\times$ 1,536 coefficients. Correctness holds without probability; failures only occur with the *naïve* carry formula (Remark 10). See `experiments/carry_identity.py`.

5.6 CEF in TALUS-MPC: Masked Broadcast

In TALUS-TEE (Section 6), the coordinator holds $\mathbf{y}^{(0)}$ in clear and computes $\mathbf{w}_1$ directly. In TALUS-MPC, no party holds $\mathbf{y}^{(0)}$, so the carry elimination must be performed on *masked* values to preserve privacy.

Remark 12 (Notation: c in this section). Throughout this section, $c \in \{0, 1\}$ denotes the *CSCP carry bit* $c = \lfloor \sum_i \rho_i > t \rfloor$, **not** the signing challenge polynomial. The challenge is introduced in the

online signing phase (Section 7.7) and is denoted there by c as well; context determines which is meant.

Definition 7 (Range-Restricted Mask). A *range-restricted mask* for party i with party count N is $\rho_i \xleftarrow{\$} [0, \lfloor \alpha/N \rfloor]$. The range restriction ensures

$$\sum_{i \in S} \rho_i \in [0, N \cdot \lfloor \alpha/N \rfloor] \subseteq [0, \alpha),$$

so $\lfloor \sum_i \rho_i / \alpha \rfloor = 0$ and the masks contribute no spurious carry to $\lfloor \sum_i b_i / \alpha \rfloor$.

Lemma 4 (Masked Broadcast Formula with FIPS Correction). *Let $H_i = \lfloor \hat{w}_i / \alpha \rfloor \bmod 16$ (floor high-bits) and $b_i = \hat{w}_i \bmod \alpha \in [0, \alpha)$ for each additive piece $\hat{w}_i = (\mathbf{A} \hat{\mathbf{y}}_i)_j$. Let party i broadcast $\tilde{H}_i = (H_i + \text{mask}_i^H) \bmod 16$ and $\tilde{b}_i = b_i + \rho_i$ where $\sum_i \text{mask}_i^H \equiv 0 \pmod{16}$ and each $\rho_i \in [0, \lfloor \alpha/N \rfloor]$. Let $B = \sum_i \tilde{b}_i$ (public), $t = B \bmod \alpha$ (public), $c = \lfloor \sum_i \rho_i > t \rfloor$ (CSCP carry bit), and*

$$\delta = \lfloor \sum_i \rho_i < t - \gamma_2 + c\alpha \rfloor$$

(FIPS correction bit, also computed by the extended CSCP; see Protocol 12 and Remark 13). Then:

$$\mathbf{w}_1 = \left(\sum_i \tilde{H}_i + \lfloor B/\alpha \rfloor - c + \delta \right) \bmod 16 = \text{HighBits}(\hat{w} \bmod q, 2\gamma_2). \quad (2)$$

Proof. Step 1: Integer floor decomposition. $\sum \tilde{H}_i \equiv \sum H_i \pmod{16}$ (antisymmetric masks cancel). Since $B = \sum b_i + \sum \rho_i$ and $t = B \bmod \alpha$, we have $\sum b_i = \lfloor B/\alpha \rfloor \cdot \alpha + (t - \sum \rho_i + c\alpha)$. Dividing: $\lfloor \sum b_i / \alpha \rfloor = \lfloor B/\alpha \rfloor + \lfloor (t - \sum \rho_i + c\alpha) / \alpha \rfloor = \lfloor B/\alpha \rfloor - c$ (since $t - \sum \rho_i \in [0, \alpha)$ when $c = 0$ and $t - \sum \rho_i \in (-\alpha, 0)$ when $c = 1$, in both cases the extra term $\lfloor \cdot / \alpha \rfloor = 0$ or -1 respectively, cancelling with $+c$ after floor). Over $\mathbb{Z}$ (integer sum, before reduction mod q): $\hat{w}_{\mathbb{Z}} = \sum_i \hat{w}_i = \alpha(\sum H_i + \lfloor \sum b_i / \alpha \rfloor) + (\sum b_i \bmod \alpha)$, so $\lfloor \hat{w}_{\mathbb{Z}} / \alpha \rfloor \bmod 16 = (\sum H_i + \lfloor \sum b_i / \alpha \rfloor) \bmod 16$ and $\hat{w}_{\mathbb{Z}} \bmod \alpha = (\sum b_i) \bmod \alpha$.

Step 2: FIPS centred-remainder correction. By FIPS 204 (Algorithm 37), $\text{HighBits}(r, 2\gamma_2) = \lfloor r/\alpha \rfloor + \delta_r$ where $\delta_r = \lfloor r \bmod \alpha > \gamma_2 \rfloor$ for $r = \hat{w} \bmod q$. We have $\hat{w}_{\mathbb{Z}} \bmod \alpha = (\sum b_i) \bmod \alpha$. Since $B = \sum b_i + \sum \rho_i$ and $t = B \bmod \alpha$:

$$(\sum b_i) \bmod \alpha = t - \sum \rho_i + c\alpha \in [0, \alpha),$$

so $\delta = \lfloor \hat{w}_{\mathbb{Z}} \bmod \alpha > \gamma_2 \rfloor = \lfloor \sum \rho_i < t - \gamma_2 + c\alpha \rfloor$.

Step 3: Relation between $\hat{w}_{\mathbb{Z}}$ and $\hat{w} \bmod q$. Let $\hat{w} = \hat{w}_{\mathbb{Z}} \bmod q$ and $m = \lfloor \hat{w}_{\mathbb{Z}} / q \rfloor \in \{0, \dots, N-1\}$. Since $q = 16\alpha + 1$: $\lfloor \hat{w}_{\mathbb{Z}} / \alpha \rfloor \bmod 16 = \lfloor \hat{w} / \alpha \rfloor \bmod 16$ provided that $(\hat{w} \bmod \alpha) + m < \alpha$ (no cross- α carry from m). This condition fails only when $\hat{w} \bmod \alpha \geq \alpha - m$, which occurs with probability $\leq m/\alpha \leq (N-1)/\alpha$ per coefficient. Similarly, $\hat{w}_{\mathbb{Z}} \bmod \alpha = \hat{w} \bmod \alpha + m$ (if $\hat{w} \bmod \alpha + m < \alpha$) or $\hat{w} \bmod \alpha + m - \alpha$ otherwise; in the latter case δ may disagree with the true FIPS correction $\lfloor \hat{w} \bmod \alpha > \gamma_2 \rfloor$ by ± 1 . In all such cases, the floor error $\epsilon \in \{-1, 0\}$ and the δ error $\epsilon' \in \{0, 1\}$ satisfy $\epsilon + \epsilon' = 0$ (they cancel), except when $\hat{w} \bmod \alpha \in (\gamma_2 - m, \gamma_2]$ (size m , probability $\leq m/\alpha$), where the formula overestimates by 1.

Conclusion. Formula (2) equals $\text{HighBits}(\hat{w} \bmod q, 2\gamma_2)$ except with per-coefficient failure probability $\leq m/\alpha \leq (N-1)/\alpha$ (the same $q \equiv 1 \pmod{\alpha}$ correction as Remark 10). The per-nonce failure probability is $1 - (1 - (N-1)/\alpha)^{nk} \approx (N-1) \cdot nk/\alpha$ (union bound; exact for $N = 2$). For ML-DSA-65 ($nk = 1536$, $\alpha = 523,776$): at $N = 5$ this is $\approx 1.2\%$; at $N = 10$, $\approx 2.6\%$; at $N = 20$, $\approx 5.4\%$. Failed nonces are detected by $\text{Verify} = 0$ and retried. $\square$ $\square$

Remark 13 (FIPS Correction Bit δ in the Extended CSCP). The CSCP (Protocol 12) already computes $c = \lfloor \sum \rho > t \rfloor$ by comparing the carry-save sum (S, C) against public threshold t . The correction bit $\delta = \lfloor \sum \rho < t - \gamma_2 + c\alpha \rfloor$ is computed by two additional parallel comparisons in Phase B (against thresholds $t - \gamma_2 - 1$ and $t - \gamma_2 + \alpha - 1$), followed by a local selection $\delta = c? \delta_1 : \delta_0$ where $\delta_0 = \lfloor \sum \rho \leq t - \gamma_2 - 1 \rfloor$ and $\delta_1 = \lfloor \sum \rho \leq t - \gamma_2 + \alpha - 1 \rfloor$. The two extra comparisons reuse the same CSA output (S, C) and add no additional rounds to the CSCP. The CSCP output is thus $(c, \delta) \in \{0, 1\}^2$.

The masked broadcast with correction (2) hides the individual (H_i, b_i) values: $\tilde{H}_i$ is uniformly random mod 16 (pairwise mask), and $\tilde{b}_i \in [b_i, b_i + \lfloor \alpha/N \rfloor]$ (range-restricted mask), revealing $\hat{w}_i$ only to within $\pm \alpha/N$. The FIPS correction bit δ leaks at most 1 additional bit about $\Sigma \rho$ (beyond the 1 bit from c); the joint leakage (c, δ) does not tighten the α/N -uncertainty bound on individual b_i .

Privacy guarantee. By the range-restricted mask, each $\hat{w}_i = \alpha H_i + b_i$ is masked by $\rho_i \in [0, \alpha/N)$, so the adversary learns b_i up to an additive error of α/N . The security analysis (Section 8.3) shows that this α/N -uncertainty is sufficient to prevent any M-LWE attack from extracting $\hat{y}_i$ from the broadcast.

Correctness. Formula (2) gives $\text{HighBits}(\hat{w} \bmod q, 2\gamma_2)$ with per-coefficient failure probability $\leq (N-1)/\alpha$ (same $q \equiv 1 \pmod{\alpha}$ correction as Remark 10; see proof above). For $N \leq 15$ with ML-DSA-65, per-nonce failure $\leq 2\%$; detected by `Verify` = 0 and retried. The mask cancellation and the (c, δ) computation are exact; the only failure source is the rare q -wrap-induced floor boundary crossing.

The FIPS-correct $\mathbf{w}_1$ is independently verified against the Lagrange CEF (Protocol 4) in `experiments/talus_mpc_e2e.py` (INV-2: 100% agreement for $T \in \{2, 3, 5, 8\}$, 50,000 trials).

6 TALUS-TEE

TALUS-TEE is a coordinator-based threshold signing protocol. A coordinator party (equipped with a Trusted Execution Environment) performs offline nonce generation and online signature assembly. Signers need only send a single response message; no secret from $\mathbf{s}_2$ is ever transmitted.

Trust model. The coordinator’s TEE is trusted to (a) run the Shamir Nonce DKG correctly and (b) not leak $\mathbf{y}$ to the adversary. Signers are trusted to hold their shares of $\mathbf{s}_1$ securely. We tolerate up to $T-1$ corrupted signers (static corruption, semi-honest or malicious) and a semi-honest coordinator. The coordinator does not learn $\mathbf{s}_1$ (individual shares are never reconstructed in clear).

6.1 Key Generation

Protocol 5 (TALUS-TEE.KeyGen). *Input:* Security parameter κ , threshold T , party count N .

Output: Public key $\mathbf{pk}$, secret shares $\{\mathbf{sk}_i\}_{i \in [N]}$.

1. Run ML-DSA KeyGen: sample $(\mathbf{A}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t})$; set $\mathbf{pk} = (\rho, \mathbf{t}_1)$.
2. Shamir-share $\mathbf{s}_1$ with threshold T among N parties: for each coordinate j , sample degree- $(T-1)$ polynomial g_j with $g_j(0) = (\mathbf{s}_1)_j$; set $(\mathbf{sk}_1)_{i,j} = g_j(i)$.
3. Distribute: send $\mathbf{sk}_i = \{(\mathbf{sk}_1)_{i,j}\}_j$ to party i via authenticated secure channel.
4. Coordinator stores $(\mathbf{A}, \mathbf{s}_2, \mathbf{t}_0)$ in TEE.

Remark 14. $\mathbf{s}_2$ and $\mathbf{t}_0$ remain with the coordinator. Under the BCC (Section 4), they are never used during online signing; the coordinator only needs them for the rare case where a preprocessed nonce fails BCC after all retries, which in TALUS-TEE we handle by re-running preprocessing.

6.2 Offline Preprocessing

Protocol 6 (TALUS-TEE.Preprocess). *Input:* Signing set $S \subseteq [N]$ with $|S| \geq T$; Lagrange coefficients $\{\lambda_i\}_{i \in S}$ for S .

Output: Nonce commitment $\mathbf{w}_1$, per-party nonce shares $\{\mathbf{y}_i\}_{i \in S}$ (held by signers), challenge seed ρ_n (held by coordinator).

1. **Nonce DKG (offline round 1).** Each party $h \in S$ samples: $g_h(X) = \mathbf{y}_h + \sum_{j=1}^{T-1} a_{h,j} X^j$ with $\mathbf{y}_h \xleftarrow{\$} [-\gamma_1/|S|+1, \gamma_1/|S|]^{n_e}$, $a_{h,j} \xleftarrow{\$} \mathbb{Z}_q^{n_e}$. Party h sends $g_h(i)$ to party i over a secure channel, and sends $g_h(0) = \mathbf{y}_h$ to the coordinator’s TEE.

2. **Nonce share assembly.** Party i computes $\mathbf{y}_i = \sum_{h \in S} g_h(i) \bmod q$. Coordinator computes aggregate constant term $\mathbf{y}^{(0)} = \sum_{h \in S} \mathbf{y}_h \bmod q$.
3. **Commitment computation.** Coordinator computes $\mathbf{w} = \mathbf{A}\mathbf{y}^{(0)}$ and decomposes $(\mathbf{w}_1, \mathbf{r}_0) = \text{Decompose}(\mathbf{w}, 2\gamma_2)$.
4. **BCC check.** If $\neg \text{BCC}(\mathbf{w})$: discard this nonce and goto step 1. Otherwise, store $(\mathbf{w}_1, \mathbf{r}_0, \mathbf{y}^{(0)})$ in TEE.
5. **Nonce commitment broadcast.** Coordinator broadcasts $\mathbf{w}_1$ to all signers in S .

Remark 15 (Expected Preprocessing Attempts). By Theorem 3, each attempt succeeds with probability $p_{\text{BCC}} \approx 0.317$, so the expected number of DKG rounds is $1/p_{\text{BCC}} \approx 3.15$. This is an offline cost amortised over the signing session.

6.3 Online Signing

Protocol 7 (TALUS-TEE.Sign). **Input:** Message μ ; preprocessed nonce commitment $\mathbf{w}_1$; each signer i holds $(\text{sk}_i, \mathbf{y}_i, \{\lambda_i\}_{i \in S})$.

Output: Signature $\sigma = (\tilde{c}, \mathbf{z}, \mathbf{h})$ where $\mathbf{h}$ is computed in step 5 below.

Online round 1 (broadcast).

1. Coordinator broadcasts challenge seed: derive $\tilde{c} \leftarrow H(\mu \| \mathbf{w}_1)$, $c \leftarrow \text{SampleInBall}(\tilde{c})$; send $\tilde{c}$ to all signers.
2. Each signer $i \in S$ computes:

$$\mathbf{z}_i = \mathbf{y}_i + c \cdot \mathbf{s}_{1,i}$$

and sends $\mathbf{z}_i$ to the coordinator.

Coordinator assembly.

3. Aggregate: $\mathbf{z} = \sum_{i \in S} \lambda_i \mathbf{z}_i \bmod q$.
4. z-bound check: if $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta$, abort and re-run preprocessing with a fresh nonce.
5. Compute hint from public data (Corollary 3):

$$\mathbf{r} = \mathbf{A}\mathbf{z} - \mathbf{c}\mathbf{t}_1 \cdot 2^d \pmod{q}, \quad h_j = \mathbf{1}[\text{HighBits}(r_j, 2\gamma_2) \neq (w_1)_j].$$

6. Hint-weight check: if $\text{wt}(\mathbf{h}) > \omega$, abort and re-run preprocessing.
7. Output $\sigma = (\tilde{c}, \mathbf{z}, \mathbf{h})$.

Remark 16 (No explicit post-Verify step). Unlike TALUS-MPC (Protocol 16, Step 8), TALUS-TEE omits a final $\text{Verify}(\text{pk}, \mu, \sigma)$ call. Correctness is guaranteed by Theorem 8 below: when BCC holds, $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$, and $\text{wt}(\mathbf{h}) \leq \omega$ (all checked in steps 4 and 6), the output signature is provably valid. Production implementations may nonetheless add a defensive verify call as a low-cost sanity check.

Theorem 8 (Correctness of TALUS-TEE). *If $\text{BCC}(\mathbf{w})$ holds, $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$, and $\text{wt}(\mathbf{h}) \leq \omega$, then $\sigma = (\tilde{c}, \mathbf{z}, \mathbf{h})$ satisfies $\text{Verify}(\text{pk}, \mu, \sigma) = 1$.*

Proof. By Shamir reconstruction, $\mathbf{z} = \sum_i \lambda_i \mathbf{z}_i = \mathbf{y} + c\mathbf{s}_1$ where $\mathbf{y} = F(0)$ is the aggregate nonce. Then:

$$\mathbf{r} := \mathbf{A}\mathbf{z} - \mathbf{c}\mathbf{t}_1 \cdot 2^d = \mathbf{w} - c\mathbf{s}_2 + \mathbf{c}\mathbf{t}_0 \pmod{q}.$$

The hint $\mathbf{h}$ is constructed so that $\text{UseHint}(\mathbf{h}, \mathbf{r}, 2\gamma_2) = \mathbf{w}_1$ (by definition of $\text{MakeHint}/\text{UseHint}$ in FIPS 204). The verifier recomputes $\mathbf{w}'_1 = \text{UseHint}(\mathbf{h}, \mathbf{A}\mathbf{z} - \mathbf{c}\mathbf{t}_1 \cdot 2^d, 2\gamma_2) = \mathbf{w}_1$ and accepts iff $H(\mu \| \mathbf{w}'_1) = \tilde{c}$ (holds by construction) and $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$ (checked in step 4). $\square$

Success probability. The per-attempt success rate of TALUS-TEE.Sign, factoring in the offline BCC filter, is:

$$p_{\text{TEE}} = p_{\text{BCC}} \times \Pr[\|\mathbf{z}\|_\infty < \gamma_1 - \beta \mid \text{BCC}] \times \Pr[\text{wt}(\mathbf{h}) \leq \omega \mid \text{BCC}].$$

The Shamir nonce DKG constructs the aggregate nonce as $\mathbf{y} = \sum_h \mathbf{y}_h$ with each $\mathbf{y}_h \in [-\gamma_1/|S| + 1, \gamma_1/|S|]^{n_\ell}$, yielding $\|\mathbf{y}\|_\infty \leq \gamma_1$ almost surely. Together with $\|\mathbf{cs}_1\|_\infty \leq \beta$, this gives $\Pr[\|\mathbf{z}\|_\infty < \gamma_1 - \beta \mid \text{BCC}] \approx 1$.

Empirically: $\Pr[\|\mathbf{z}\|_\infty < \gamma_1 - \beta \mid \text{BCC}] = 1.000$, $\Pr[\text{wt}(\mathbf{h}) \leq \omega \mid \text{BCC}] \approx 0.995\text{--}0.998$, giving $p_{\text{TEE}} \approx 0.317 \times 0.997 \approx 0.316$ across $|S| \in \{2, 3, 5\}$. See `experiments/talus_tee.py`.

6.4 Blame Protocol

When a signer i fails to respond or sends an invalid $\mathbf{z}_i$ (i.e., Shamir reconstruction fails or $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta$ due to a bad share), the coordinator runs **Blame**.

Protocol 9 (TALUS-TEE.Blame). *Input:* Signing transcript; response set $\{\mathbf{z}_i\}$.

1. For each $i \in S$: verify $\mathbf{z}_i = \mathbf{y}_i + \mathbf{cs}_{1,i}$ by checking $\mathbf{Az}_i = \mathbf{Ay}_i + c\mathbf{As}_{1,i}$ (coordinator holds $\mathbf{Ay}_i$ from preprocessing).
2. Any i whose $\mathbf{z}_i$ fails is identified and excluded.
3. Re-run with the reduced signing set $S' = S \setminus \{i\}$ (if $|S'| \geq T$) using the same preprocessed nonce.

Remark 17 (Blame Without TEE Access to $\mathbf{s}_1$). The Blame protocol uses the fact that the coordinator holds $\mathbf{Ay}_i$ (computed from $g_h(i)$ values during preprocessing) but not $\mathbf{s}_{1,i}$. The check is probabilistic: a malicious party i could send $\mathbf{z}_i = \mathbf{y}_i + \mathbf{cs}_{1,i} + \delta$ for small δ that still passes the z-bound. A binding commitment to $\mathbf{s}_{1,i}$ (e.g., via a Feldman-VSS [Fel87] commitment established at key generation) enables exact blame attribution. We include the commitment-based variant in Appendix B.

6.5 Key Refresh

Long-term key share refresh prevents share compromise from accumulating over time. We follow the standard proactive secret sharing paradigm.

Protocol 10 (TALUS-TEE.Refresh). *Input:* Current shares $\{\mathbf{s}_{1,i}\}_{i \in [N]}$.

1. Each party $h \in [N]$ samples a fresh degree- $(T-1)$ polynomial δ_h with $\delta_h(0) = \mathbf{0}$ (zero-secret sharing).
2. Party h sends $\delta_h(i)$ to party i .
3. Party i updates: $\mathbf{s}'_{1,i} = \mathbf{s}_{1,i} + \sum_h \delta_h(i) \bmod q$.

The aggregate secret is unchanged: $\sum_i \lambda_i \mathbf{s}'_{1,i} = \mathbf{s}_1 + \sum_h \sum_i \lambda_i \delta_h(i) = \mathbf{s}_1 + \sum_h \delta_h(0) = \mathbf{s}_1$. The public key $\mathbf{pk}$ is unaffected.

The refresh requires one offline round of $O(N^2)$ messages of size $O(n_\ell)$ bytes each, identical in cost to one Shamir Nonce DKG round.

7 TALUS-MPC

TALUS-MPC is the fully distributed profile of TALUS. Unlike TALUS-TEE, no party holds $\mathbf{s}_2$ or any aggregate secret; instead, the commitment hash input $\mathbf{w}_1$ is computed via a secure protocol built on the Carry Elimination Framework. Online signing remains a single broadcast round, identical to TALUS-TEE.

7.1 System Model

Parties. There are N parties $P_1, \dots, P_N$. Signing is performed by a designated signer set $S \subseteq [N]$ with $|S| = T$. We write λ_i for the Lagrange coefficient of party i with respect to S .

Threat model. We work in the *malicious identifiable-abort* (malicious-IA) model for $T \geq 2$: an adversary $\mathcal{A}$ statically corrupts at most $T - 1$ parties and may instruct them to (i) follow the protocol honestly, (ii) deviate arbitrarily in transmitted messages (malicious), or (iii) abort (drop all future messages). Whenever a corrupted party causes a protocol failure, honest parties output an explicit $\text{Blame}(i)$ identifying the cheating party (Theorem 20). EUF-CMA and Privacy are proved in this model (Theorems 25 and 26): malicious deviations do not increase the adversary’s forgery probability (Game 0’ reduction), and privacy holds even under arbitrary deviations and blame reveals, via the $\text{CarryCompareMalicious}$ Masking and Blame Session Privacy lemmas (Lemmas 7 and 8 in Section 8.3). Our IA mechanism is *optimistic*: blame attribution is triggered only on failure and carries no additional online-round cost (Protocol 19).

Remark 18 (Two instantiations of CarryCompare). The carry comparison sub-protocol $\text{CarryCompare}(T, \{\rho_i\}, t)$ has two instantiations depending on the threshold T :

- $T = 2$: instantiated via a Distributed Comparison Function (DCF) [BGI15, BGI16] — any $N \geq 2$ suffices; offline cost ≈ 384 bytes; *computational* security (Section 7.3.1).
- $T \geq 3$: instantiated via the Carry-Safe Comparison Protocol (CSCP) — requires $N \geq 2T - 1$ (Dishonest Majority Barrier, Theorem 27); offline cost ≈ 350 KB; *information-theoretic* Shamir privacy plus PRF security (Section 7.3.2).

The offline protocol (Section 7.4) calls CarryCompare uniformly; the instantiation is selected by T .

Communication and setup. Parties communicate over authenticated point-to-point channels (secure against eavesdropping) and a common broadcast channel. At setup, all pairs (i, j) establish a shared PRF seed $s_{ij} = s_{ji}$, from which Beaver triples are derived locally (Section 7.6). All pairwise seeds are established once during key generation; no trusted third party is required for subsequent signing operations.

7.2 Key Generation (Phase 0)

Key generation is a one-time cost performed once for the lifetime of the key. It is *not* counted in the offline signing round budget.

Protocol 11 (TALUS-MPC.KeyGen). **Input:** Security parameter κ , parties $[N]$, threshold T .

Output: Public key $\text{pk} = (\rho_A, \mathbf{t}_1)$; secret shares $\text{sk}_i = \mathbf{s}_{1,i}$ for each i .

1. **(Phase 0, Round 1)** Each party i samples $\mathbf{A} \leftarrow \text{ExpandA}(\rho_A)$ from a shared public seed ρ_A (broadcast once and fixed). Parties jointly generate a Shamir-shared signing key $\mathbf{s}_1$ with $\|\mathbf{s}_1\|_\infty \leq \eta$ via Pedersen DKG [GJKR99]: each party i samples a contribution $\mathbf{v}_i \xleftarrow{\$} \chi_{s,i}^\ell$ with $\|\mathbf{v}_{i,j}\|_\infty \leq \lfloor \eta / \min(N, \eta) \rfloor$ (for $N \leq \eta$) or uses a standard hash-based commit-then-reveal coin-tossing subprotocol (parties commit to their contributions before revealing; see [GJKR99]) (for $N > \eta$, e.g. $N \geq 5$ for ML-DSA-65 with $\eta = 4$) to produce $\mathbf{s}_1 = \sum_i \mathbf{v}_i$ with $\|\mathbf{s}_1\|_\infty \leq \eta$ [GJKR99]. Party i sets up degree- $(T - 1)$ Shamir polynomials f_{v_i} with constant term $\mathbf{v}_i$ and sends $f_{v_i}(j)$ to party j . Simultaneously, parties run two rounds of Pedersen DKG [GJKR99] to generate $(\mathbf{s}_2, \mathbf{t}_0)$ for coordinator storage (TALUS-TEE) or distribute $\mathbf{s}_2$ (TALUS-MPC, discarded after BCC).
2. **(Phase 0, Round 2)** Each party i assembles $\mathbf{s}_{1,i} = \sum_j f_{v_j}(i) \bmod q$. The aggregate secret $\mathbf{s}_1 = \sum_i \mathbf{v}_i$ satisfies $\|\mathbf{s}_1\|_\infty \leq \eta$ by construction. Each party broadcasts $\mathbf{A}\mathbf{s}_{1,i}$ (a Feldman commitment to their assembled share, used for public-key assembly and later blame attribution) and also broadcasts their $\mathbf{s}_2$ -piece $\mathbf{s}_{2,i}$ (the additive share from the Pedersen DKG in Round 1) to enable public-key assembly. All parties compute $\mathbf{t} = \sum_i \mathbf{A}\mathbf{s}_{1,i} + \sum_i \mathbf{s}_{2,i} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2$ and set $\text{pk} = (\rho_A, \mathbf{t}_1)$ where $(\mathbf{t}_1, \mathbf{t}_0) = \text{Power2Round}(\mathbf{t}, d)$. (Revealing $\mathbf{s}_{2,i}$ does not compromise $\mathbf{s}_1$: recovering $\mathbf{s}_1$ from $\mathbf{A}\mathbf{s}_1 = \mathbf{t} - \mathbf{s}_2$ requires solving M-LWE.)
3. **Pairwise seed establishment.** Each pair (i, j) with $i < j$ samples a shared PRF seed $s_{ij} \xleftarrow{\$} \{0, 1\}^\kappa$ via any AKE or DH protocol (simultaneous with Round 2 messages, no extra round).

Remark 19 (KeyGen Round Counting). The 2-round Pedersen DKG [GJKR99] is the industry-standard one-time cost shared by all threshold signing schemes [KG21, BCdP⁺26, CdPE⁺26]. We follow this convention and list TALUS-MPC’s offline signing cost separately from Phase 0.

Remark 20 (Pedersen vs. Feldman Commitments in KeyGen). Two distinct commitment types are used in KeyGen, each for a different purpose. *Pedersen commitments* (of the form $g^a h^r$) are used *internally* by the DKG in Round 1 [GJKR99]: each party blinds its contribution $\mathbf{v}_i$ before revealing, ensuring the coin-toss is binding and hiding throughout the DKG interaction. *Feldman commitments* (of the form $\mathbf{A}\mathbf{s}_{1,i}$, i.e. matrix-vector products) are broadcast in Round 2 for two purposes: (i) assembling the public key $\mathbf{t}_1 = \text{Power2Round}(\mathbf{A}\mathbf{s}_1 + \mathbf{s}_2, d)_1$, and (ii) enabling Feldman VSS blame attribution during online signing (Protocol 19). The Feldman commitment $\mathbf{A}\mathbf{s}_{1,i}$ is computationally hiding under M-LWE (recovering $\mathbf{s}_{1,i}$ requires inverting $\mathbf{A}$ over a bounded domain).

7.3 CarryCompare: Distributed Carry Comparison

The core primitive of TALUS-MPC is *CarryCompare*, which securely computes a single carry bit

$$c = \text{CarryCompare}(T, \{\rho_i\}, t) = \lceil \sum \rho_i > t \rceil$$

where each party i holds a private mask $\rho_i \in [0, \lfloor \alpha/N \rfloor]$ and $t \in [0, \alpha]$ is a public threshold. The output $c \in \{0, 1\}$ is revealed to all parties.

The carry bit c is precisely the correction needed to recover $\lfloor \sum b_i / \alpha \rfloor$ from public data (Lemma 5 below), completing the distributed $\mathbf{w}_1$ computation. *CarryCompare* has two instantiations (Remark 18): DCF for $T = 2$ and CSCP for $T \geq 3$; both share the same interface.

7.3.1 $T = 2$: DCF Instantiation

When $T = 2$, the TALUS-MPC protocol simplifies dramatically. The nonce decomposes additively ($\hat{\mathbf{y}} = \hat{\mathbf{y}}_1 + \hat{\mathbf{y}}_2$, unit coefficients), allowing the *NonceDKG* to bypass Shamir polynomial evaluation; key-share reconstruction retains the standard Lagrange coefficients ($\lambda_1 = 2, \lambda_2 = -1$). More significantly, *CarryCompare* degenerates to a *distributed comparison function* (DCF) [BGI15, BGI16]: each coefficient requires only a 2-bit carry tag (one bit for the carry comparison $[\rho > B \bmod \alpha]$ and one bit for the safety check), giving $1536 \times 2 = 3072$ bits = 384 bytes for the entire *CarryCompare* sub-protocol.

Table 12 (Section 9.3) quantifies the improvement. The total offline communication per batch drops from ~ 370 KB ($T \geq 3$) to ~ 5 KB ($T = 2$), a $74\times$ reduction. Per session ($M = 30$ batches), the offline cost is ~ 150 KB versus ~ 11.1 MB, comparable to a single TCP segment for the per-batch cost.

This result has direct commercial significance. The most common enterprise deployment scenario for threshold signatures is *2-of-2 custody*: one key share held by the enterprise and one by a third-party custodian. For this dominant use case, TALUS-MPC operates at communication costs comparable to single-party ML-DSA key exchange, with the offline preprocessing requiring only ~ 5 KB per batch.

Remark 21 ($T = 2$ Security Model). For $T = 2$, the honest-majority barrier (Theorem 27) does not apply: a single honest party always holds a complete Shamir share, and *CarryCompare* uses *computational* (DCF) security rather than information-theoretic Shamir security. The $T = 2$ case therefore achieves stronger privacy guarantees at lower communication cost.

Corollary 6 ($T = 2$ EUF-CMA Security). *TALUS-MPC with $T = 2$ is EUF-CMA secure under the ML-DSA EUF-CMA assumption and the key-indistinguishability of the DCF [BGI15, BGI16].*

Proof. The proof of Theorem 25 applies verbatim for $T = 2$ with one modification: Game 1 (Shamir nonce privacy) uses information-theoretic Shamir privacy, which holds for $T = 2$ since corrupting one of two parties ($|C| = 1 \leq T - 1 = 1$) leaves one degree of freedom in the degree- $(T - 1) = 1$ polynomial. In Game 2, the CSCP Beaver triples are replaced by DCF key pairs; DCF key-indistinguishability [BGI15] gives $|\Pr[G2] - \Pr[G1]| \leq \text{Adv}^{\text{DCF}}(\mathcal{B})$. Games 3–4 proceed identically to the $T \geq 3$ case. The honest-majority requirement $N \geq 2T - 1$ reduces to $N \geq 2$ for $T = 2$, which is trivially satisfied. $\square$

7.3.2 $T \geq 3$: CSCP Instantiation

Protocol 12 (CSCP: Carry-Safe Comparison). ***Input:** Party i holds $\rho_i \in [0, \lfloor \alpha/N \rfloor]$; public $t \in [0, \alpha)$. All parties hold boolean shares $[\rho_i]$ derived from pairwise PRF seeds $\{s_{ij}\}$ (Section 7.6).*

***Output:** Carry bit $c = \lfloor \sum_i \rho_i > t \rfloor$ and FIPS correction bit $\delta = \lfloor \sum_i \rho_i < t - \gamma_2 + c\alpha \rfloor$, both revealed to all parties.*

Phase A: CSA Reduction ($\lceil \log_2(N/2) \rceil$ rounds). *Run a 4:2 compressor tree on the boolean shares $\{[\rho_i]\}_{i=1}^N$ to reduce N secret-shared 19-bit values to a carry-save pair (S, C) with $S + C = \sum_i \rho_i$:*

- Each CSA level takes 1 round (Beaver AND gates from precomputed triples).
- Level 1 is merged with Round 1 of the nonce DKG (see Protocol 13), saving 1 round compared to the un-merged count.
- After $\lceil \log_2(N/2) \rceil$ CSA levels: (S, C) available as boolean-shared 19-bit values.

Phase B: Prefix Comparison (2 rounds). *Compare (S, C) against public t using a 4-group prefix tree:*

1. Partition the 19 bits into 4 groups: $G_3 = [18:15]$, $G_2 = [14:10]$, $G_1 = [9:5]$, $G_0 = [4:0]$ (4, 5, 5, 5 bits).
2. **Prefix Round 1 (Level-1):** For each group $g \in \{0, 1, 2, 3\}$ in parallel, compute boolean-shared group comparison bits:
 - $GT_g = \lfloor \text{group-}g \text{ bits of } S + C \text{ exceed group-}g \text{ bits of } t \rfloor$
 - $EQ_g = \lfloor \text{group-}g \text{ bits of } S + C \text{ equal group-}g \text{ bits of } t \rfloor$*These are evaluated simultaneously for all 4 groups using precomputed degree- $\lceil |G_g| \rceil$ Beaver triples (Section 7.6). Each group requires at most $|G_g| - 1 \leq 4$ AND gate evaluations in 1 round.*
3. **Prefix Round 2 (Level-2):** Evaluate the cascaded comparison using precomputed degree-3 Beaver triples:

$$c = GT_3 \vee (EQ_3 \wedge GT_2) \vee (EQ_3 \wedge EQ_2 \wedge GT_1) \vee (EQ_3 \wedge EQ_2 \wedge EQ_1 \wedge GT_0). \quad (3)$$

The nested cascade has multiplicative depth 3, evaluated in 1 round using a precomputed degree-3 triple.

4. Each party reconstructs c from their shares.
[FIPS edge case, $\leq 2\%$ frequency per nonce.] In the same round, also compute in parallel: $\delta_0 = \lfloor \sum \rho \leq t - \gamma_2 - 1 \rfloor$ and $\delta_1 = \lfloor \sum \rho \leq t - \gamma_2 + \alpha - 1 \rfloor$ by applying the same cascade (3) with thresholds $t - \gamma_2 - 1$ and $t - \gamma_2 + \alpha - 1$ respectively. (Trivial shortcuts: if $t \leq \gamma_2$ then $\delta_0 = 0$; if $t \geq \gamma_2$ and $c = 1$ then $\delta_1 = 1$.) Set $\delta = c ? \delta_1 : \delta_0$. This correction handles the rare $q \equiv 1 \pmod{\alpha}$ boundary crossing (Remark 13); it reuses the same CSA output (S, C) and adds no rounds. On a first reading, δ may be ignored; the main protocol carries only c .

Remark 22 (Degree- k Beaver Triples from Pairwise Seeds). The CSCP uses degree-5 and degree-3 Beaver triples. These are generalisations of the standard degree-2 triple $(\langle a \rangle, \langle b \rangle, \langle a \wedge b \rangle)$ to k factors: $(\langle a_1 \rangle, \dots, \langle a_k \rangle, \langle a_1 \wedge \dots \wedge a_k \rangle)$. Such triples can be derived from session keys $\{K_{ij,\tau}\}$ in 0 additional communication rounds using the degree- k Shamir sharing structure over $\mathbb{F}(2^m)$ [DPSZ12], where each party evaluates its share locally from $\{K_{ij,\tau}\}$. The PRF security of this construction is formalised in Lemma 12 (Appendix A).

Lemma 5 (Carry Bit Recovery). *Let $B = \sum_i \tilde{b}_i$ (public) and $t = B \bmod \alpha$ (public). Let $c = \lfloor \sum_i \rho_i > t \rfloor$ be the CSCP output. Then:*

$$\lfloor \frac{\sum_i b_i}{\alpha} \rfloor = \lfloor \frac{B}{\alpha} \rfloor - c.$$

Proof. Write $B = \sum b_i + \sum \rho_i$. Let $q_B = \lfloor B/\alpha \rfloor$ and $t = B \bmod \alpha$. Then $\sum b_i = B - \sum \rho_i = \alpha q_B + (t - \sum \rho_i)$. Since $\sum \rho_i \in [0, \alpha)$ and $t \in [0, \alpha)$:

- If $\sum \rho_i \leq t$: $(t - \sum \rho_i) \in [0, t] \subset [0, \alpha)$, so $\lfloor \sum b_i / \alpha \rfloor = q_B = q_B - 0$.

- If $\Sigma\rho_i > t$: $(t - \Sigma\rho_i) \in [-\alpha + 1, -1)$, so $\Sigma b_i = \alpha(q_B - 1) + (\alpha + t - \Sigma\rho_i)$ and $\lfloor \Sigma b_i / \alpha \rfloor = q_B - 1 = q_B - 1$.
- In both cases $\lfloor \Sigma b_i / \alpha \rfloor = q_B - c$. □ □

7.4 Offline Preprocessing (Phase 1)

Protocol 13 (TALUS-MPC.Preprocess). **Input:** Signing set $S \subseteq [N]$, $|S| = T$; Lagrange coefficients $\{\lambda_i\}$. Pairwise seeds $\{s_{ij}\}$ from Phase 0.

Output: Commitment $\mathbf{w}_1$; per-party nonce shares $\{\hat{\mathbf{y}}_i\}_{i \in S}$.

Setup (0 rounds, local):

- **Session-key derivation.** Let τ be the session identifier. τ **must be globally unique per preprocessing session** (e.g. a 128-bit random nonce sampled fresh at the start of each preprocessing session, or a strictly monotone counter persisted in non-volatile storage; implementations **MUST** ensure uniqueness across restarts and concurrent sessions). Reusing τ is equivalent to reusing a PRF key and breaks the security of all derived masks and Beaver triples. Each pair (i, j) derives a session key:

$$K_{ij, \tau} = \text{PRF}(s_{ij}, \text{"session"} \parallel \tau),$$

using only the long-term pairwise seed s_{ij} . All subsequent per-session PRF evaluations use $K_{ij, \tau}$ in place of s_{ij} , so no single session ever exposes the master seed.

- Each party i derives an antisymmetric pairwise mask: for each ordered pair (a, b) with $a < b$ in S , set $r_{ab} = \text{PRF}(K_{ab, \tau}, \text{"maskH"}) \bmod 16$, then

$$\text{mask}_i^H = \left(\sum_{j \in S, j > i} r_{ij} - \sum_{j \in S, j < i} r_{ji} \right) \bmod 16.$$

Because each r_{ab} contributes $+1$ to mask_a^H and -1 to mask_b^H , we have $\sum_{i \in S} \text{mask}_i^H \equiv 0 \pmod{16}$.

Party i also draws a range-restricted mask locally:

$$\rho_i \xleftarrow{\$} [0, \lfloor \alpha/N \rfloor],$$

where $N = |S|$. (In a deployment using only PRF seeds, set

$$\rho_i = H\left(\bigoplus_{j \in S, j \neq i} \text{PRF}(K_{ij, \tau}, \text{"rho"})\right) \bmod \lfloor \alpha/N \rfloor$$

so that at least one honest party's PRF output is unknown to any single adversary.)

- Derive boolean shares $[\rho_i]$ and Beaver triples for the CSCP from session keys $\{K_{ij, \tau}\}$ (0 rounds; Remark 22).

Round 1 (Nonce DKG + Masked Broadcast + CSA Level-1 Merged):

Round 1 serves four cryptographic purposes simultaneously, all sent in a single broadcast to minimise round count:

- (A) Nonce DKG: distribute Shamir nonce shares $g_h(i)$ so each party i can assemble $\hat{\mathbf{y}}_i$ for online signing.
- (B) Masked broadcast: commit to each party's additive nonce piece $\hat{\mathbf{w}}_h$ (masked) so the coordinator can compute $\mathbf{w}_1$ via the carry formula without learning any individual $\hat{\mathbf{y}}_h$.
- (C) Feldman VSS: publish polynomial commitments $\{\Phi_{h,k}\}$ enabling later blame attribution (IA, §7.11).
- (D) CSA Level-1 (merged): begin the CSCP carry computation for the FIPS correction bit δ , piggybacked onto the same message to save one round.

1. Each $h \in S$ samples nonce polynomial $g_h(X) = \hat{\mathbf{y}}_h + \sum_{k=1}^{T-1} \mathbf{a}_{h,k} X^k$ with $\hat{\mathbf{y}}_h \xleftarrow{\$} [-\gamma_1/|S| + 1, \gamma_1/|S|]^{n_e}$. Party h sends $g_h(i)$ to party i (P2P, secure channel).

2. (**Two-role computation.**) Each party acts in two roles simultaneously:

Role A (additive contributor, subscript h): Party h uses its own constant term $\hat{\mathbf{y}}_h$ (chosen in Step 1) to compute the additive nonce contribution:

$$\hat{\mathbf{w}}_h = \mathbf{A}\hat{\mathbf{y}}_h \bmod q, \quad H_h = \lfloor \hat{\mathbf{w}}_h / \alpha \rfloor \bmod 16, \quad b_h = \hat{\mathbf{w}}_h \bmod \alpha.$$

Note: $\hat{\mathbf{w}} = \mathbf{A}\hat{\mathbf{y}} = \mathbf{A}\sum_{h \in S} \hat{\mathbf{y}}_h = \sum_{h \in S} \hat{\mathbf{w}}_h$ (additive decomposition, unit coefficients).

Role B (Shamir share holder, subscript i): Party i assembles its Shamir nonce share for use at signing time:

$$\hat{\mathbf{y}}_i = \sum_{h \in S} g_h(i) \quad (\text{stored locally; used in online signing as } \mathbf{z}_i = \hat{\mathbf{y}}_i + c \cdot \mathbf{s}_{1,i}).$$

3. Each party h broadcasts its additive contribution masked:

$$\tilde{H}_h = (H_h + \text{mask}_h^H) \bmod 16, \quad \tilde{b}_h = b_h + \rho_h.$$

4. [**Feldman VSS Commitments (same Round 1 broadcast).**] Party h additionally broadcasts Feldman commitments to its nonce polynomial g_h :

$$\Phi_{h,k} = \mathbf{A}\tilde{\mathbf{a}}_{h,k} \bmod q, \quad k = 0, 1, \dots, T-1,$$

where $\tilde{\mathbf{a}}_{h,0} = \hat{\mathbf{y}}_h$ and $\tilde{\mathbf{a}}_{h,k}$ for $k \geq 1$ are the polynomial coefficients from step 1. Each receiving party $i' \in S$ verifies the consistency of its P2P share:

$$\mathbf{A}g_h(i') = \sum_{k=0}^{T-1} \Phi_{h,k} \cdot (i')^k \pmod{q}.$$

Any party h failing this check is immediately identified and excluded (**Blame**(h)).

5. [**CSA Level-1 MERGED**]: Simultaneously with Step 3, each party h evaluates the Level-1 AND gates of the CSA tree on the boolean shares $[\rho_h]$, using precomputed Beaver triples. Each party broadcasts its masked AND-gate evaluation $(\tilde{d}_h^{(1)}, \tilde{e}_h^{(1)})$ alongside $(\tilde{H}_h, \tilde{b}_h)$ in the same message.

6. After Round 1: all parties compute $B = \sum_{h \in S} \tilde{b}_h$ (public) and $t = B \bmod \alpha$ (public). Level-1 CSA output $(S^{(1)}, C^{(1)})$ is reconstructed locally from the broadcast AND-gate values.

Rounds 2 through $\lceil \log_2(N/2) \rceil$ (Remaining CSA Levels):

6. For level $\ell = 2, \dots, \lceil \log_2(N/2) \rceil$: apply the 4:2 compressor to the carry-save pair from the previous level. Each compressor evaluation uses 1 round of Beaver AND gates. After the final CSA level: (S, C) is a carry-save representation with $S + C = \sum_i \rho_i$, available as boolean shares.

Round $\lceil \log_2(N/2) \rceil + 1$ (Prefix Level-1, CSCP Phase B):

7. Evaluate group comparisons for all 4 groups in parallel (Protocol 12, Phase B, Round 1). Broadcast masked AND-gate evaluations. All parties reconstruct $(\text{GT}_g, \text{EQ}_g)$ for each g .

Round $\lceil \log_2(N/2) \rceil + 2$ (Prefix Level-2, CSCP Phase B):

8. Evaluate the degree-3 cascade (3) using precomputed degree-3 triples. Each party broadcasts its masked evaluation. All parties compute carry bit c and FIPS correction bit δ :

$$\begin{aligned} c &= \lfloor \sum_i \rho_i > t \rfloor \in \{0, 1\}, \\ \delta_0 &= \lfloor \sum_i \rho_i \leq t - \gamma_2 - 1 \rfloor = \lfloor \sum_i \rho_i < t - \gamma_2 \rfloor, \\ \delta_1 &= \lfloor \sum_i \rho_i \leq t - \gamma_2 + \alpha - 1 \rfloor = \lfloor \sum_i \rho_i < t - \gamma_2 + \alpha \rfloor, \\ \delta &= c ? \delta_1 : \delta_0. \end{aligned}$$

The comparisons for c , δ_0 , δ_1 use the same CSA output (S, C) and run in parallel (no additional rounds). Trivial cases: if $t \leq \gamma_2$, $\delta_0 = 0$; if $t \geq \gamma_2$ and $c = 1$, $\delta_1 = 1$; both without circuit evaluation.

9. All parties compute $\mathbf{w}_1$ locally:

$$\mathbf{w}_1 = \left(\sum_{h \in S} \tilde{H}_h + \lfloor B/\alpha \rfloor - c + \delta \right) \bmod 16 \quad (4)$$

using $\sum_{h \in S} \text{mask}_h^H \equiv 0 \pmod{16}$ (masks cancel) and Lemma 4.

10. **Store and continue.** Each party i stores $(\mathbf{w}_1, \hat{\mathbf{y}}_i)$ locally. BCC is not verified during preprocessing in TALUS-MPC: no party holds $\hat{\mathbf{w}} = \mathbf{A}\hat{\mathbf{y}}$, so $\text{LowBits}(\hat{\mathbf{w}})$ is unavailable. Any nonce whose BCC fails will be detected implicitly during online signing: cs_2 shifts a low-bit coefficient across a modular boundary, the hint $\mathbf{h}$ becomes inconsistent with $\mathbf{w}_1$, and ML-DSA.Verify returns 0 (abort). The expected fraction of nonces that yield a valid signature is $p_{\text{BCC}} \approx 31.7\%$ (Theorem 3).

Remark 23 (Nonce One-Time Use). Each preprocessed nonce $(\mathbf{w}_1, \{\hat{\mathbf{y}}_i\})$ **must be used for at most one signing session**. If the same nonce is used for two messages $\mu \neq \mu'$, the adversary obtains $\mathbf{z} - \mathbf{z}' = (c - c')\mathbf{s}_1$, from which $\mathbf{s}_1$ is recoverable since $c - c'$ is a known low-weight polynomial. Implementations must delete the nonce immediately after online signing completes or aborts. The per-session PRF key $K_{ij,\tau}$ ensures each session uses a fresh internal state; the nonce itself must be explicitly invalidated.

Theorem 14 (Correctness of CSCP). *Let $\mathbf{w}_1$ be computed by (4). Then $\mathbf{w}_1 = \text{HighBits}(\mathbf{A}\hat{\mathbf{y}}, 2\gamma_2)$ where $\hat{\mathbf{y}} = F(0) = \sum_{h \in S} \hat{\mathbf{y}}_h$, except with per-coefficient probability $\leq (N-1)/\alpha$ (the same $q \equiv 1 \pmod{\alpha}$ correction as the naive carry formula; see Lemma 4). For $N \leq 15$ with ML-DSA-65 parameters, the per-nonce failure probability is $\leq 2\%$; failures are detected by $\text{Verify} = 0$ and retried.*

Proof. Additive structure. Each party h contributes constant term $\hat{\mathbf{y}}_h$ and broadcasts masked values derived from $\hat{\mathbf{w}}_h = \mathbf{A}\hat{\mathbf{y}}_h$. The aggregate is $\hat{\mathbf{w}} = \mathbf{A}\hat{\mathbf{y}} = \sum_h \hat{\mathbf{w}}_h$ (additive over $\mathbb{Z}_q$).

Floor decomposition (over $\mathbb{Z}$). Fix coefficient j . With $H_{h,j} = \lfloor \hat{w}_{h,j}/\alpha \rfloor \bmod 16$ and $b_{h,j} = \hat{w}_{h,j} \bmod \alpha$: $\hat{w}_{\mathbb{Z},j} = \sum_h \hat{w}_{h,j} = \alpha(\sum H_{h,j} + \lfloor \sum b_{h,j}/\alpha \rfloor) + (\sum b_{h,j} \bmod \alpha)$ (over $\mathbb{Z}$). Masks cancel ($\sum H_{h,j} \equiv \sum H_{h,j} \pmod{16}$) and Lemma 5 gives $\lfloor \sum b_{h,j}/\alpha \rfloor = \lfloor B_j/\alpha \rfloor - c_j$.

FIPS correction. The extended CSCP outputs $\delta_j = \lfloor \sum \rho_{h,j} < t_j - \gamma_2 + c_j \alpha \rfloor = \lfloor \hat{w}_{\mathbb{Z},j} \bmod \alpha > \gamma_2 \rfloor$. Therefore:

$$(\sum H_{h,j} + \lfloor \sum b_{h,j}/\alpha \rfloor + \delta_j) \bmod 16 = (\lfloor \hat{w}_{\mathbb{Z},j}/\alpha \rfloor + \lfloor \hat{w}_{\mathbb{Z},j} \bmod \alpha > \gamma_2 \rfloor) \bmod 16,$$

which is the FIPS-style high-bits of the integer sum $\hat{w}_{\mathbb{Z},j}$ (applying the $\pm\gamma_2$ centred-remainder convention to an integer value).

Relation to $\hat{w}_j \bmod q$. Let $m_j = \lfloor \hat{w}_{\mathbb{Z},j}/q \rfloor \in \{0, \dots, N-1\}$. By Lemma 4 (Step 3), $\text{HighBits}(\hat{w}_{\mathbb{Z},j}) \bmod 16 = \text{HighBits}(\hat{w}_j \bmod q) \bmod 16$ except when $(\hat{w}_j \bmod q) \bmod \alpha \in (\gamma_2 - m_j, \gamma_2]$ (probability $\leq m_j/\alpha \leq (N-1)/\alpha$).

Conclusion. Equation (4) gives $w_{1,j} = \text{HighBits}(\hat{w}_j \bmod q, 2\gamma_2)$ except with per-coefficient probability $\leq (N-1)/\alpha$. $\square$ $\square$

7.5 Offline Round Count

Table 4: TALUS-MPC offline preprocessing round count. Phase 0 (2-round DKG) is one-time setup, not counted. “Merge” saves the CSA Level-1 round by piggybacking on Round 1 (nonce DKG broadcast).

N	CSA levels	Merge saves	Prefix	Total offline
2	0	0	2	3
3–4	1	1	2	3
5–8	2	1	2	4
9–16	3	1	2	5
17–32	4	1	2	6
33–64	5	1	2	7

Formula: $\max(3, \lceil \log_2(N/2) \rceil + 2)$

Phase 0 (KeyGen): 2-round Pedersen DKG [GJKR99], one-time.

For $N = 2$ the CSA tree has no levels ($\lceil \log_2 1 \rceil = 0$) and the two masks (ρ_1, ρ_2) are compared directly via the prefix tree in 2 rounds after the Round-1 broadcast. For $N = 3\text{--}4$ a single CSA level

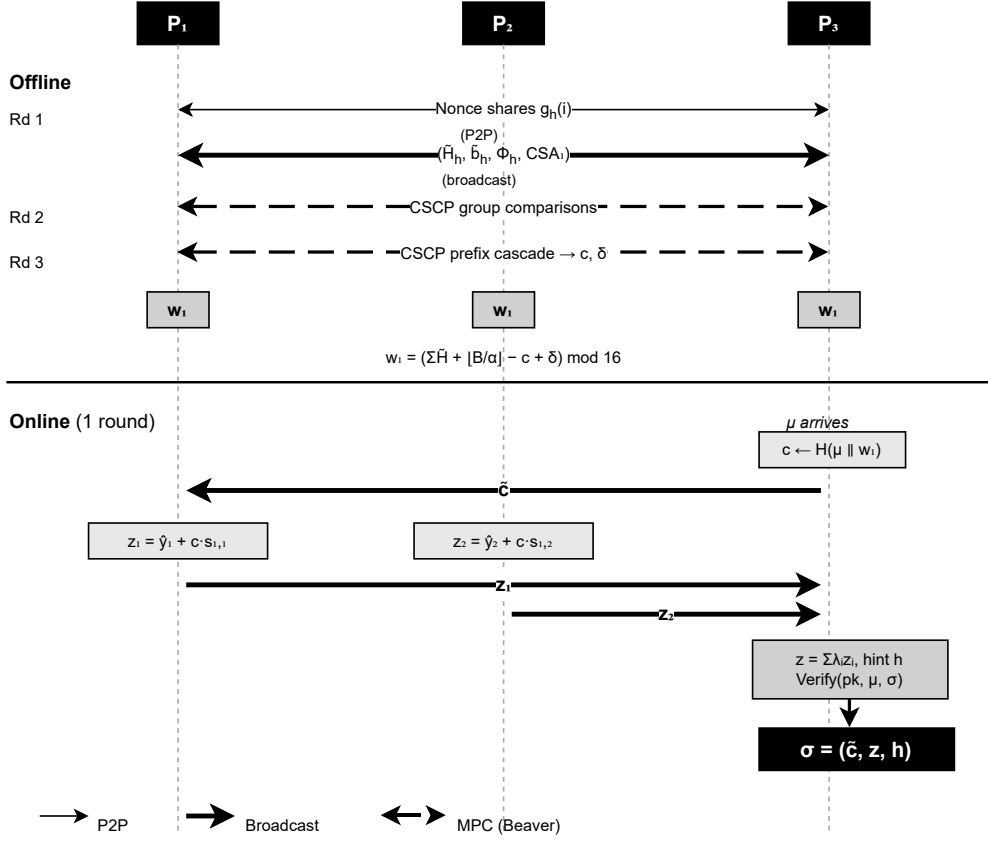


Figure 3: TALUS-MPC protocol flow for $N=3$ parties. **Offline** (3 rounds): Round 1 broadcasts nonce shares and CSA-compressed masks; Rounds 2–3 run the CSCP prefix cascade to recover the carry c and correction δ . Each party then computes w_i locally. **Online** (1 round): upon message arrival, the designated party computes the challenge $\tilde{c}$, each signer returns z_i , and the assembler forms the final signature $\sigma = (\tilde{c}, z, h)$.

is merged with Round 1, still giving 3 total offline rounds. The formula $\max(3, \lceil \log_2(N/2) \rceil + 2)$ holds for all $N \geq 2$; the full flow is illustrated in Figure 3.

Lemma 6 (CSCP Round Count). *Protocol 12 completes in $\max(3, \lceil \log_2(N/2) \rceil + 2)$ offline rounds.*

Proof. Each level of a 4:2 compressor tree takes 4 inputs and produces 2 carry-save outputs, halving the operand count per level. Starting from N shares, $\lceil \log_2(N/2) \rceil$ levels reduce to a single carry-save pair (S, C) with $S + C = \sum_i \rho_i$ (the case $N/2 \leq 1$, i.e. $N \leq 2$, has 0 CSA levels). Phase A therefore uses $\lceil \log_2(N/2) \rceil$ rounds; Level 1 is merged with the Round-1 nonce-DKG broadcast (Protocol 13, Step 1), saving one round. Phase B (prefix comparison) uses exactly 2 rounds regardless of N . The total is $1 + (\lceil \log_2(N/2) \rceil - 1) + 2 = \lceil \log_2(N/2) \rceil + 2$ for $N \geq 3$, and $1 + 0 + 2 = 3$ for $N = 2$, giving $\max(3, \lceil \log_2(N/2) \rceil + 2)$. $\square$

7.6 Beaver Triple Construction from Pairwise Seeds

Protocol 15 (PRF-Based Beaver Triple Generation). **Input:** Session keys $\{K_{ij,\tau}\}_{i < j}$ for session τ (derived from pairwise seeds at session start).

Output: For each AND gate g in the CSA/CSCP circuit, a boolean Beaver triple $(\langle a_g \rangle, \langle b_g \rangle, \langle c_g \rangle)$ with $c_g = a_g \wedge b_g$.

Each party i computes (0 communication rounds):

1. For each $j \neq i$: set $r_{ij}^{(g)} = \text{PRF}(K_{ij,\tau}, \text{"and"}\|g)$ and $r_{ji}^{(g)} = \text{PRF}(K_{ij,\tau}, \text{"and"}\|g\|\text{"swap"})$.
2. The shares of a_g and b_g are: $\langle a_g \rangle_i = \bigoplus_{j < i} r_{ji}^{(g)} \oplus \bigoplus_{j > i} r_{ij}^{(g)}$, $\langle b_g \rangle_i = \text{PRF}(K_{ij_0,\tau}, \text{"bg"}\|g)$ (agreed from session key with a designated counterpart j_0).
3. The share of $c_g = a_g \wedge b_g$ is computed via: $\langle c_g \rangle_i = \langle a_g \rangle_i \cdot \langle b_g \rangle_i \oplus \bigoplus_{j \neq i} d_{ij}^{(g)}$ where $d_{ij}^{(g)} = \text{PRF}(K_{ij,\tau}, \text{"carry"}\|g)$.

Remark 24 (Correctness and Security). This construction is a standard adaptation of the SPDZ offline phase [DPSZ12] to the PRF-from-shared-seed setting [AFL⁺16]. For the degree- k triples used in the CSCP, each party evaluates degree- k polynomial shares over $\mathbb{F}(2)$ from the pairwise seeds; correctness follows from the linearity of Shamir sharing over $\mathbb{F}(2^m)$.

Remark 25 (Rushing Adversary Has No Advantage in CSCP). In the CSCP broadcast rounds, a rushing adversary (one who observes all honest parties' masked broadcasts before deciding its own) gains no advantage. Each honest party broadcasts $\tilde{H}_h = H_h + \text{mask}_h^H \bmod 16$ and $\tilde{b}_h = b_h + \rho_h$, where mask_h^H and ρ_h are derived from PRF-based pairwise masks (known to party h alone before the broadcast). The masks are *independent of the other parties' broadcasts*: they are computed locally from $K_{ij,\tau}$ before any message is received. A rushing adversary who first reads honest broadcasts cannot change its masks retroactively without breaking PRF security. The security of the CSCP therefore follows from PRF security in the standard (non-rushing) model, and rushing gives no additional information about any honest party's H_h or b_h values.

7.7 Online Signing (Phase 2)

Protocol 16 (TALUS-MPC.Sign). **Input:** Message μ ; preprocessed $\mathbf{w}_1$; signer i holds $(\mathbf{sk}_i = \mathbf{s}_{1,i}, \hat{\mathbf{y}}_i, \{\lambda_i\}_{i \in S})$.

Output: Signature $\sigma = (\tilde{c}, \mathbf{z}, \mathbf{h})$.

Online Round 1 (Broadcast).

1. Coordinator broadcasts challenge seed: compute $\tilde{c} \leftarrow H(\mu\|\mathbf{w}_1)$, $c \leftarrow \text{SampleInBall}(\tilde{c})$; send $\tilde{c}$ to all signers.
2. Each signer $i \in S$ computes and sends:

$$\mathbf{z}_i = \hat{\mathbf{y}}_i + c \cdot \mathbf{s}_{1,i}.$$

Coordinator assembly.

3. [IA Blame Check.] For each $i \in S$, reconstruct the expected nonce commitment from the public Feldman commitments broadcast at preprocessing:

$$\mathbf{A}\hat{\mathbf{y}}_i = \sum_{h \in S} \sum_{k=0}^{T-1} \Phi_{h,k} \cdot i^k \pmod{q}.$$

Verify:

$$\mathbf{A}\mathbf{z}_i \stackrel{?}{=} \mathbf{A}\hat{\mathbf{y}}_i + c \cdot \mathbf{A}\mathbf{s}_{1,i} \pmod{q},$$

where $\mathbf{A}\mathbf{s}_{1,i}$ is the public key share broadcast at KeyGen (Phase 0, Round 2). If the check fails for any i , output **Blame**(i) and abort. This check adds no online round (computed locally after receiving $\mathbf{z}_i$).

4. Aggregate: $\mathbf{z} = \sum_{i \in S} \lambda_i \mathbf{z}_i \bmod q$.
5. z-bound check: abort and re-run preprocessing if $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta$.
6. Compute hint from public data (Corollary 3):

$$\mathbf{r} = \mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d \pmod{q}, \quad h_j = \mathbf{1}[\text{HighBits}(r_j, 2\gamma_2) \neq (w_1)_j].$$

7. Hint-weight check: abort and re-run if $\text{wt}(\mathbf{h}) > \omega$.

8. *[Mandatory pre-output verification.]* Run $\text{ML-DSA.Verify}(\text{pk}, \mu, \sigma)$. If the result is 0, discard σ , abort this session, and signal parties to re-run preprocessing with a fresh nonce. A coordinator must never release a signature that fails verification; doing so would leak information about the nonce distribution to external observers.
9. Output $\sigma = (\tilde{c}, \mathbf{z}, \mathbf{h})$.

The online signing protocol is identical to TALUS-TEE (Protocol 7) except that $\mathbf{w}_1$ is computed by the CSCP offline phase rather than by a TEE. The coordinator need not hold $\mathbf{s}_2$ or $\mathbf{t}_0$.

7.8 Correctness and Success Rate

Theorem 17 (Correctness of TALUS-MPC). *If $\text{BCC}(\hat{\mathbf{w}})$ holds, $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$, $\text{wt}(\mathbf{h}) \leq \omega$, and $\mathbf{w}_1 = \text{HighBits}(\mathbf{A}\hat{\mathbf{y}}, 2\gamma_2)$ (which holds except with per-nonce probability $\leq 2\%$; see Theorem 14), then $\sigma = (\tilde{c}, \mathbf{z}, \mathbf{h})$ satisfies $\text{Verify}(\text{pk}, \mu, \sigma) = 1$.*

Proof. Theorem 14 gives $\mathbf{w}_1 = \text{HighBits}(\mathbf{A}\hat{\mathbf{y}}, 2\gamma_2)$ (when the $q \equiv 1 \pmod{\alpha}$ correction does not apply). Shamir reconstruction gives $\mathbf{z} = \hat{\mathbf{y}} + c\mathbf{s}_1$. The remainder of the proof is identical to Theorem 8. $\square$

Success probability. The TALUS-MPC per-attempt success rate incorporates the additional constraint from the *shift-invariance* of the nonce distribution. Since $\hat{\mathbf{y}} = \sum_{h \in S} \hat{\mathbf{y}}_h$ is an Irwin-Hall sum of $|S| = T$ terms (each uniform in $[-\gamma_1/T, \gamma_1/T]$), the z-bound probability is slightly reduced relative to TALUS-TEE:

$$p_{\text{MPC}} = p_{\text{BCC}} \times \Pr[\|\mathbf{z}\|_\infty < \gamma_1 - \beta \mid \text{BCC}] \times \Pr[\text{wt}(\mathbf{h}) \leq \omega \mid \text{BCC}].$$

The Irwin-Hall distribution has variance $\gamma_1^2/(3T)$ per coefficient; the security loss from the tighter distribution is analysed in Section 7.9 (Shift-Invariance).

Experimentally confirmed: $p_{\text{MPC}} \approx 0.313\text{--}0.335$ for $T \in \{2, 3, 5, 8, 17, 32, 64, 128, 256\}$, with $\Pr[\text{z-bound} \mid \text{BCC}] \approx 1.000$, $\Pr[\text{wt}(\mathbf{h}) \leq \omega \mid \text{BCC}] \approx 0.994\text{--}1.000$, and $E[\text{wt}(\mathbf{h})] \approx 38.7\text{--}39.2$ (well under $\omega = 55$). The success rate matches TALUS-TEE ($\approx 31.5\%$) since the z-bound passes with probability $\approx 100\%$ for all tested thresholds (Table 5).

Table 5: TALUS-MPC per-attempt signing success rate for ML-DSA-65 ($\gamma_1 = 2^{19}$, $\beta = 196$, $\omega = 55$). All probabilities are empirical. $p_{\text{MPC}} = p_{\text{BCC}} \times \Pr[\text{z-ok} \mid \text{BCC}] \times \Pr[\text{wt}(\mathbf{h}) \leq \omega \mid \text{BCC}]$.

T	Trials	p_{BCC}	$\Pr[\text{z-ok} \mid \text{BCC}]$	$\Pr[\mathbf{h} \leq \omega \mid \text{BCC}]$	p_{MPC}	$E[\text{wt}(\mathbf{h})]$
2	3 000	0.317	1.000	0.995	0.316	38.8
3	3 000	0.313	1.000	0.993	0.311	38.8
5	3 000	0.317	1.000	0.994	0.315	38.9
8	2 000	0.319	1.000	0.996	0.318	38.8
17	1 500	0.329	1.000	0.995	0.328	38.7
32	800	0.335	1.000	0.996	0.334	38.8
64 [†]	600	0.313	1.000	0.998	0.313	38.8
128 [†]	400	0.318	1.000	0.995	0.316	38.9
256 [†]	200	0.320	1.000	1.000	0.320	39.2

[†] Additive shares (post-NonceDKG, all $\lambda_h = 1$); $T \leq 32$ uses full Lagrange CEF.

7.9 Shift-Invariance and Security Loss

In TALUS-MPC the aggregate nonce $\hat{\mathbf{y}} = \sum_h \hat{\mathbf{y}}_h$ follows an Irwin-Hall distribution $\text{IH}(T)$ on each coefficient, with support $[-\gamma_1, \gamma_1]$ (truncated) and variance $\gamma_1^2/(3T)$. Compared to a single uniform draw, the $\text{IH}(T)$ distribution is *more concentrated* around zero, meaning the z-vector's distribution depends slightly on $\mathbf{s}_1$.

Definition 8 (Shift-Invariance Ratio). For a distribution $\mathcal{D}$ on $[-\gamma_1, \gamma_1]$, the *shift-invariance ratio* with shift β is:

$$\Delta(\mathcal{D}, \beta) := \max_{x \in [-\gamma_1, \gamma_1 - \beta]} \frac{f_{\mathcal{D}}(x + \beta)}{f_{\mathcal{D}}(x)}.$$

For $\mathcal{D} = \text{IH}(T)$ with parameter $\beta = 196$ (ML-DSA-65), the exact values are given by FFT-convolution analysis [Kao26] (see Table 6). For reference, the Gaussian approximation ($\text{IH}(T) \approx N(0, \gamma_1^2/(3T))$) gives $\Delta \approx 1 + 3T\beta^2/\gamma_1^2$, which underestimates the exact values by a factor of approximately 2.5 for $T \geq 3$.

The security loss from the $\text{IH}(T)$ nonce distribution is quantified by the per-coordinate shift-invariance chi-squared (Definition 8), computed exactly via FFT convolution [Kao26]. The vectorial Rényi divergence over all $n_\ell = n \cdot \ell = 1,280$ coefficients (ML-DSA-65) is:

$$R_2^{\text{vec}}(T) = (1 + \chi_{\text{SI}}^2(T))^{n_\ell},$$

where $\chi_{\text{SI}}^2(T) = \Delta(\text{IH}(T), \beta) - 1$ is the per-coordinate shift-invariance divergence (Definition 8). The per-session information loss is $\log_2 R_2^{\text{vec}}(T) \approx n_\ell \cdot \chi_{\text{SI}}^2(T) / \ln 2$ bits (Table 6).

Table 6: Shift-invariance security loss per signing session for ML-DSA-65 ($n_\ell = 1280$, $\gamma_1 = 2^{19}$, $\beta = 196$). $\chi_{\text{SI}}^2(T)$: per-coordinate shift-invariance chi-squared (FFT convolution); $R_2^{\text{vec}} = (1 + \chi_{\text{SI}}^2)^{n_\ell}$: vectorial Rényi divergence; Loss = $\log_2 R_2^{\text{vec}}$ (bits per session).

T	$\chi_{\text{SI}}^2(T)$	R_2^{vec}	Loss (bits/session)
2	2.954×10^{-6}	1.0038	0.0055
3	1.453×10^{-6}	1.0019	0.0027
5	2.134×10^{-6}	1.0027	0.0039
8	3.371×10^{-6}	1.0043	0.0062
17	7.134×10^{-6}	1.0092	0.0132
32	1.342×10^{-5}	1.0174	0.0248
64	2.683×10^{-5}	1.0350	0.0496
128	5.366×10^{-5}	1.0711	0.0991
256	1.073×10^{-4}	1.1473	0.1982

All values computed by exact FFT convolution [Kao26]; no empirical trials required.

$T = 2$ (Triangle dist.) has $3.5\times$ higher χ_{SI}^2 than the Gaussian approximation;

for $T \geq 3$, χ_{SI}^2 increases monotonically with T ($\chi_{\text{SI}}^2 \approx 3T\beta^2/\gamma_1^2$).

For $T \geq 5$, Gaussian approximation matches exact FFT to $< 2\%$.

Theorem 18 (Shift-Invariance Security Loss). *Let y be an Irwin-Hall sum of T uniform terms on $[-\gamma_1/T, \gamma_1/T]$. The per-coordinate shift-invariance chi-squared $\chi_{\text{SI}}^2(T) = \Delta(\text{IH}(T), \beta) - 1$ is computed exactly via FFT convolution (Table 6). The per-session vectorial Rényi divergence is:*

$$R_2^{\text{vec}}(T) = (1 + \chi_{\text{SI}}^2(T))^{n_\ell},$$

and the per-session shift-invariance loss is $\epsilon_{\text{IH}}(T) = R_2^{\text{vec}}(T) - 1$. The security reduction (Theorem 23) yields concrete security:

$$\text{Security} \geq 128 - \log_2(q_s \cdot R_2^{\text{vec}}(T)) - \log_2 Q_H.$$

For ML-DSA-65 with $T \leq 128$, $q_s \leq 1600$, and $Q_H = 2^{32}$: $R_2^{\text{vec}}(128) = 1.071$, giving $128 - \log_2(1600 \times 1.071) - 32 = \mathbf{85.3}$ bits. Security is dominated by q_s (contributing ≈ 10.6 bits of loss), not by T (contributing < 0.1 bits even at $T = 128$).

Remark 26. The exact $\text{IH}(2)$ distribution (Triangle) has $3.5\times$ higher χ_{SI}^2 than the Gaussian approximation [Kao26]. For $T \geq 5$, the Gaussian approximation matches the exact FFT values to within 2% (Table 6).

Remark 27 (IH Loss is Inherent to Shamir Nonce Sharing). The $q_s \cdot \epsilon_{\text{IH}}(T)$ loss is not a deficiency of TALUS: it is the information-theoretic cost of Shamir nonce distribution. Any scheme that Shamir-shares the nonce across T parties produces an aggregate following $\text{IH}(T)$, incurring this exact loss regardless of the signing protocol. Single-party ML-DSA achieves zero IH loss because no sharing occurs; TALUS achieves the minimum possible IH loss for any Shamir-based nonce architecture. For ML-DSA-65 with $q_s \leq 1600$ and $T \leq 128$, the loss is dominated by q_s (≈ 10.6 bits) rather than T (< 0.1 bits even at $T = 128$), leaving ≥ 85.3 bits of security. Deployments requiring $q_s > 2^{12}$ or $T > 200$ should upgrade to ML-DSA-87 (NIST Level 5).

7.10 Communication Analysis

Offline preprocessing. Round 1 each party broadcasts:

- $(\tilde{H}_i, \tilde{b}_i)$: $(4 + 19) \times 1,536 = 35,328$ bits ≈ 4.3 KB.
- Feldman VSS commitments $\{\Phi_{h,k}\}_{k=0}^{T-1}$: T vectors in $\mathbb{Z}_q^{nk}$ ($n = 256$, $k_{\text{rows}} = 6$), giving $T \times 1,536 \times 23 \approx T \times 4.3$ KB. For $T = 3$: ≈ 12.9 KB per party.
- CSA Level-1 masked AND evaluations $(d^{(1)}, e^{(1)})$: $2 \times 19 \times 1,536 = 73,728$ bits ≈ 9.0 KB.

Total broadcast Round 1: $\approx 13.3 + T \times 4.3$ KB per party (≈ 26.2 KB at $T = 3$). P2P nonce DKG shares: $n_\ell \times \lceil \log_2 q \rceil \times (|S| - 1) = 1,280 \times 23 \times (T - 1)$ bits.

Subsequent CSA rounds: $2 \times 19 \times 1,536 \approx 9.0$ KB broadcast per party per round.

CSCP Prefix rounds: each party broadcasts $\leq 10 \times 2 = 20$ bits per comparison group per round (masked AND outputs), i.e. < 1 KB total over both prefix rounds.

Online signing. Each party i broadcasts $\mathbf{z}_i \in \mathbb{Z}_q^{n_\ell}$: $1,280 \times 23 = 29,440$ bits ≈ 3.6 KB.

Table 7: TALUS-MPC communication summary per party per signing session ($T = N$, ML-DSA-65).

Component	Size	Phase
Nonce DKG (P2P per party)	$(T - 1) \times 29.4$ KB	Offline R1
Masked broadcast $(\tilde{H}, \tilde{b})$	4.3 KB	Offline R1
Feldman commitments (IA)	$T \times 4.3$ KB	Offline R1
CSA L1 AND evals (broadcast)	9.0 KB	Offline R1 (merged)
CSA L2-LK AND evals (each)	9.0 KB	Offline R2-RK
Prefix rounds (combined)	< 1 KB	Offline RK+1,2
Response $\mathbf{z}_i$ (broadcast)	3.6 KB	Online R1

7.11 Blame Protocol (Identifiable Abort)

Protocol 19 (TALUS-MPC.Blame). **Input:** Signing transcript for session τ ; public Feldman commitments $\{\Phi_{h,k}\}$; public key shares $\{\mathbf{A}_{s_{1,i}}\}$.

Trigger: The online IA Blame Check (Step 3 of Protocol 16) found no culprit, yet the aggregated signature fails ($\text{Verify} = 0$). This implies that the error occurred in the preprocessing phase (incorrect $\mathbf{w}_1$ due to a malformed masked broadcast $(\tilde{H}_h, \tilde{b}_h)$ or a corrupted CSCP AND-gate evaluation).

1. **Reveal.** Each party $h \in S$ broadcasts an opening:

$$O_h = (\hat{\mathbf{y}}_h, \{\tilde{\mathbf{a}}_{h,k}\}_{k=1}^{T-1}, \rho_h, \{K_{hj,\tau}\}_{j \in S}).$$

Each $K_{hj,\tau} = \text{PRF}(s_{hj}, \text{"session"} \parallel \tau)$ is the session key for this preprocessing session only; the long-term pairwise seed s_{hj} is not revealed.

2. **Feldman consistency.** For each h , recompute $\Phi'_{h,k} = \mathbf{A} \tilde{\mathbf{a}}_{h,k} \bmod q$ and verify $\Phi'_{h,k} = \Phi_{h,k}$ for all k . Any h with $\Phi'_{h,0} \neq \Phi_{h,0}$ lied about $\hat{\mathbf{y}}_h$; output $\text{Blame}(h)$.

3. **Masked-broadcast consistency.** Recompute $H_h = \lfloor \mathbf{A}\hat{\mathbf{y}}_h / \alpha \rfloor \bmod 16$ and $b_h = \mathbf{A}\hat{\mathbf{y}}_h \bmod \alpha$. Recompute masks from the revealed seeds $(\text{mask}_h^H, \rho_h)$. Verify $\tilde{H}_h = (H_h + \text{mask}_h^H) \bmod 16$ and $\tilde{b}_h = b_h + \rho_h$. Any h failing either check has sent a malformed masked broadcast; output $\text{Blame}(h)$.
4. **CarryCompare *replay*.** Using the revealed session keys $\{K_{hj,\tau}\}$, regenerate all Beaver triples for session τ via Protocol 15 (for $T \geq 3$; for $T = 2$, re-derive the DCF key pair). Re-execute the full CarryCompare evaluation. The first gate evaluation inconsistent with the session-key-derived randomness identifies the cheating party; output $\text{Blame}(h)$. (For $T = 2$, blame is immediate: with a single counterparty, any inconsistency uniquely identifies the cheater.) The long-term seeds $\{s_{hj}\}$ are never exposed; future sessions remain secure by PRF security of $K_{hj,\tau'} = \text{PRF}(s_{hj}, \text{"session"} \parallel \tau')$ for $\tau' \neq \tau$.

Theorem 20 (IA Completeness of TALUS-MPC.Blame). *If a malicious party causes TALUS-MPC to output $\text{Verify} = 0$, then Protocol 19 outputs $\text{Blame}(h)$ for some h in the corrupted set with probability $1 - \text{negl}(\kappa)$.*

Proof. The three failure modes are exhaustive:

1. *Wrong $\mathbf{z}_i$ (online phase):* Caught by Step 3 of Protocol 16 without invoking Protocol 19.
2. *Inconsistent masked broadcast:* If $(\tilde{H}_h, \tilde{b}_h)$ do not correspond to $\hat{\mathbf{y}}_h$ committed in $\Phi_{h,0}$, steps 2–3 of Protocol 19 detect this.
3. *Corrupted CSCP evaluation:* If an AND-gate output deviates from the triple derived from the session keys, step 4 identifies the responsible party. (Correctness follows from the PRF-based Beaver triple construction: each triple is uniquely determined by $K_{hj,\tau}$, so any deviation is detectable. The session keys are sufficient for replay; master seeds need not be revealed.)

Since at least one of these failure modes must hold whenever $\text{Verify} = 0$ is caused by a malicious party, blame attribution succeeds. The reveal of $\hat{\mathbf{y}}_h$ is safe: the nonce is discarded after the failed session, and $\hat{\mathbf{y}}_h$ carries no information about the long-term key $\mathbf{s}_1$. $\square$ $\square$

Remark 28 (Privacy Under Reveal). Protocol 19 is triggered only for a *failed* preprocessing session. The revealed $\hat{\mathbf{y}}_h$ is the nonce constant term for that session; it is never used in any valid signing session, so its disclosure does not compromise the privacy of $\mathbf{s}_1$ or any future nonce. The revealed session keys $\{K_{hj,\tau}\}$ are one-time values derived as $K_{hj,\tau} = \text{PRF}(s_{hj}, \text{"session"} \parallel \tau)$. By PRF security, they reveal nothing about the long-term master seeds $\{s_{hj}\}$ or any other session key $K_{hj,\tau'}$ ($\tau' \neq \tau$), so future Beaver triples remain protected.

7.12 Security

Security of TALUS-MPC follows a 6-game hybrid proof extending the TALUS-TEE argument (Section 8.1) with additional transitions for the distributed $\mathbf{w}_1$ computation and Beaver triple security.

Theorem 21 (EUF-CMA Security of TALUS-MPC, informal). *Under the ML-DSA EUF-CMA assumption (which reduces to Module-LWE) and the security of CarryCompare (PRF-based Beaver triples for $T \geq 3$; DCF key-indistinguishability [BGI15, BGI16] for $T = 2$), TALUS-MPC is EUF-CMA secure against any PPT adversary corrupting at most $T - 1$ parties. The formal advantage bound (Theorem 25) is:*

$$\text{Adv}_{\text{TALUS-MPC}}^{\text{EUF-CMA}} \leq \frac{Q_H^2}{|\mathcal{C}|} + q_s \cdot \epsilon_{\text{IH}}(T) + \text{Adv}_{\text{Beaver}}^{\text{PRF}} + \text{Adv}_{\text{ML-DSA}}^{\text{EUF-CMA}}.$$

The full security proof is in Section 8.3.

Theorem 22 (Privacy of TALUS-MPC). *In the identifiable-abort model with $T - 1$ corrupted parties, the view of the adversary in TALUS-MPC is computationally indistinguishably simulatable given only the final signature σ and the public key pk , under the PRF security of the pairwise seeds and the hardness of inverting $\mathbf{A}$ over a bounded domain (inhomogeneous Module-SIS).*

Specifically:

- $\tilde{H}_h$ is statistically uniform mod 16 for each honest party h . This follows unconditionally from the antisymmetric pairwise mask structure: $\text{mask}_h^H = (\sum_{j>h} r_{hj} - \sum_{j<h} r_{jh}) \bmod 16$ is uniform in $\{0, \dots, 15\}$ from the corrupted parties' view, since at least one honest r_{hj} is unknown to them.
- $\tilde{b}_h = b_h + \rho_h$ computationally hides $b_h = \mathbf{A}\hat{\mathbf{y}}_h \bmod \alpha$ under two successive hardness arguments: (i) $\rho_h \in [0, \lfloor \alpha/N \rfloor]$ restricts the adversary's knowledge of b_h to an interval of width $\alpha/N \approx 523,776/N$ ($\geq 26,188$ for $N \leq 20$ with ML-DSA-65, $\alpha = 523,776$); and (ii) even knowing b_h exactly, recovering $\hat{\mathbf{y}}_h$ from $\mathbf{A}\hat{\mathbf{y}}_h \bmod \alpha$ requires solving an inhomogeneous Module-SIS instance in dimension $n(k+\ell) = 2,816$ over a modulus $\alpha \approx 2^{19}$, which admits no known polynomial-time algorithm.

The CSCP outputs reveal only the single comparison bit $c = \lfloor \sum \rho_i > t \rfloor$, leaking at most one bit of information about the aggregate $\sum \rho_i$ and nothing about individual shares or the nonce $\hat{\mathbf{y}}$.

The simulator constructs:

- $\tilde{H}_j^{\text{sim}} \xleftarrow{\$} \{0, \dots, 15\}$ for corrupted j (uniform mod 16).
- $\tilde{b}_j^{\text{sim}} \xleftarrow{\$} [b_j^*, b_j^* + \lfloor \alpha/N \rfloor]$ where $b_j^* \xleftarrow{\$} [0, \alpha)$ (computationally indistinguishable from real view under Module-SIS hardness).
- Beaver $(d_j, e_j)^{\text{sim}} \xleftarrow{\$} \{0, 1\}^2$ (uniform random bits).
- $\mathbf{w}_1^{\text{sim}}$ from the signing oracle.
- $\mathbf{z}_j^{\text{sim}}$ from the signing oracle.

The statistical indistinguishability of $\tilde{H}_j^{\text{sim}}$ follows immediately. The computational indistinguishability of $\tilde{b}_j^{\text{sim}}$ follows from the PRF security of $\hat{\mathbf{y}}_h$ generation (each honest party's nonce is fresh) composed with Module-SIS hardness. Full proof in Appendix B.

8 Security

We prove EUF-CMA security for TALUS-TEE via a 5-game reduction. Security of TALUS-MPC is deferred to Appendix A.

8.1 Security of TALUS-TEE

Theorem 23 (TALUS-TEE EUF-CMA). *Let $\mathcal{A}$ be a PPT adversary that corrupts up to $T - 1$ signers (static, semi-honest or malicious), and makes at most q_s signing queries and Q_H random oracle queries. Then:*

$$\text{Adv}_{\text{TALUS-TEE}}^{\text{EUF-CMA}}(\mathcal{A}) \leq \frac{Q_H^2}{|\mathcal{C}|} + q_s \cdot \epsilon_{\text{IH}}(T) + \text{Adv}_{\text{ML-DSA}}^{\text{EUF-CMA}}(\mathcal{B})$$

where $|\mathcal{C}| = \binom{256}{\tau} \cdot 2^\tau$ is the challenge space, $\epsilon_{\text{IH}}(T) = R_2^{\text{vec}}(T) - 1$ is the per-session Rényi divergence loss from the Irwin-Hall nonce distribution (Theorem 18, Table 6), and $\mathcal{B}$ runs in essentially the same time as $\mathcal{A}$.

Proof. We define a sequence of five games (four game hops).

Game 0: Real protocol. The adversary $\mathcal{A}$ interacts with the real TALUS-TEE protocol. Corrupted signers C (with $|C| < T$) may behave maliciously. However, malicious deviations are handled by the TEE coordinator: any signer that sends a share inconsistent with its Feldman commitment $\Phi_{i,0}$ is identified and excluded; any online response $\mathbf{z}_i$ that fails the per-party check is rejected. Hence, without loss of generality, we may treat corrupted parties as sending their prescribed (semi-honest) shares or aborting; no separate Game 0' reduction is required. The adversary outputs a forgery (m^*, σ^*) . By definition:

$$\Pr[\mathcal{A} \text{ wins Game 0}] = \text{Adv}_{\text{TALUS-TEE}}^{\text{EUF-CMA}}(\mathcal{A}).$$

Game 1: Replace nonce shares with simulated values. We modify the simulator to replace each honest signer i 's nonce DKG contribution $g_h(i)$ with a uniformly random value from $\mathbb{Z}_q^{n_\ell}$. By the statistical privacy of Shamir secret sharing [Sha79], the adversary's view of the honest parties' nonce shares changes by statistical distance at most SD_{DKG} , where $\text{SD}_{\text{DKG}} = 0$ since the joint distribution of any $T - 1$ evaluations of a degree- $(T - 1)$ polynomial over $\mathbb{Z}_q$ is perfectly uniform (information-theoretic security of Shamir sharing).

However, the aggregate nonce $\hat{\mathbf{y}} = \sum_h \hat{\mathbf{y}}_h$ follows the Irwin-Hall distribution $\text{IH}(T)$ (not uniform over $[-\gamma_1, \gamma_1]$): this is the sum of T independent bounded uniforms, which is more concentrated near zero than the uniform distribution, reducing the entropy of each nonce coefficient. By the product formula for Rényi-2 divergence across independent coordinates (Theorem 18), the per-session loss is $\epsilon_{\text{IH}}(T) = R_2^{\text{vec}}(T) - 1 = (1 + \chi_{\text{SI}}^2)^{n_\ell} - 1$, where χ_{SI}^2 is the per-coordinate shift-invariance divergence (Definition 8, Table 6). Since $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$ with $\mathbf{y} \sim \text{IH}(T)$, the shift by $c\mathbf{s}_1$ ($\|c\mathbf{s}_1\|_\infty \leq \beta$) introduces a per-session Rényi factor bounded by $R_2^{\text{vec}}(T)$. Across q_s sessions:

$$|\Pr[\mathcal{A} \text{ wins Game 1}] - \Pr[\mathcal{A} \text{ wins Game 0}]| \leq q_s \cdot \epsilon_{\text{IH}}(T).$$

Game 2: Abort on hash collision. We add an abort step: if any two of the Q_H random oracle queries to H (including both the signing oracle's internal queries and the adversary's direct queries) produce the same output challenge $c \in \mathcal{C}$, we abort. By a birthday bound over all Q_H oracle evaluations, the probability this abort is triggered is at most $Q_H^2/|\mathcal{C}|$. (Using $Q_H \geq q_s$, this subsumes the $q_s^2/|\mathcal{C}|$ collision probability across signing sessions.) Thus:

$$|\Pr[\mathcal{A} \text{ wins Game 2}] - \Pr[\mathcal{A} \text{ wins Game 1}]| \leq \frac{Q_H^2}{|\mathcal{C}|}.$$

In Game 2, each signing session has a distinct, uniformly random challenge c .

Game 3: Safety Theorem equivalence. We observe that by the BCC invariant (maintained in preprocessing), every honest signing session produces $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$ from a transcript identical to what single-party ML-DSA would produce on input $(\mathbf{y}, c)$. No $\mathbf{s}_2$ information is ever revealed. The signing oracle in Game 2 is therefore equivalent to a single-party ML-DSA signing oracle (given only $\mathbf{s}_1$, not $\mathbf{s}_2$). This substitution changes the game by at most the statistical distance introduced by the BCC restriction on nonce distribution, which is 0: BCC is a predicate on the public commitment $\mathbf{w} = \mathbf{A}\mathbf{y}$ and does not depend on c or $\mathbf{s}_2$. Formally, the conditional distribution of the signing transcript $(\mathbf{z}, \tilde{c}, \mathbf{h})$ given that $\text{BCC}(\hat{\mathbf{w}})$ holds is identical whether the nonce is drawn from the full distribution or from the BCC-filtered distribution, because BCC is determined by $\hat{\mathbf{w}}$ alone and the simulator can replicate the same filtering. Thus:

$$|\Pr[\mathcal{A} \text{ wins Game 3}] - \Pr[\mathcal{A} \text{ wins Game 2}]| = 0.$$

Game 4: Reduce to ML-DSA EUF-CMA. In Game 3, the signing oracle is exactly a single-party ML-DSA signer. Any adversary that wins Game 3 (forging a valid threshold ML-DSA signature) also wins the single-party ML-DSA EUF-CMA game, because: (i) the forgery $\sigma^* = (\tilde{c}^*, \mathbf{z}^*, \mathbf{h}^*)$ is a valid FIPS 204 signature on a fresh message; (ii) the adversary never queried the signing oracle on m^* . We construct $\mathcal{B}$ that simulates Game 3 using the ML-DSA signing oracle and forwards any forgery. Thus:

$$\Pr[\mathcal{A} \text{ wins Game 3}] \leq \text{Adv}_{\text{ML-DSA}}^{\text{EUF-CMA}}(\mathcal{B}).$$

Combining all game hops:

$$\text{Adv}_{\text{TALUS-TEE}}^{\text{EUF-CMA}}(\mathcal{A}) \leq \frac{Q_H^2}{|\mathcal{C}|} + q_s \cdot \epsilon_{\text{IH}}(T) + \text{Adv}_{\text{ML-DSA}}^{\text{EUF-CMA}}(\mathcal{B}).$$

The $Q_H^2/|\mathcal{C}|$ term is from Game 1→2 (birthday bound); the $q_s \cdot \epsilon_{\text{IH}}(T)$ term is from Game 0→1 (Irwin-Hall vs. uniform nonce distribution, Theorem 18); and $\text{Adv}_{\text{ML-DSA}}^{\text{EUF-CMA}}(\mathcal{B})$ is from Game 3→4 (ML-DSA reduction).

Concrete security. For ML-DSA-65 at NIST Security Level 3, the M-SIS problem provides ≥ 128 bits of core hardness [Nat24b, DKL⁺18]. The reducer $\mathcal{B}$ must (i) guess which of q_s signing sessions to embed its challenge into (factor q_s), (ii) account for the Irwin–Hall nonce distribution via Rényi divergence (factor R_2^{vec}), and (iii) succeed in the Fiat–Shamir forking lemma (factor Q_H). The effective advantage is therefore

$$\text{Adv} \leq \frac{Q_H^2}{|\mathcal{C}|} + q_s \cdot R_2^{\text{vec}} \cdot Q_H \cdot \text{Adv}_{\text{M-SIS}}.$$

With $q_s = 1600$, $|S| \leq 128$, and $Q_H = 2^{32}$: the per-coordinate shift-invariance chi-squared is $\chi_{S_1}^2(T) \leq 5.37 \times 10^{-5}$ (exact FFT, Table 6), giving $R_2^{\text{vec}} = (1 + \chi_{S_1}^2)^{n_\ell} \leq 1.071$. The multiplicative loss is $\log_2(q_s \cdot R_2^{\text{vec}} \cdot Q_H) = \log_2(1714) + 32 \approx 42.7$ bits, yielding proven security $\geq 128 - 42.7 = \mathbf{85.3 \text{ bits}}$. For typical thresholds ($T \leq 10$), the Rényi loss drops to < 0.01 bits per session (Table 6), improving the bound to ≥ 95 bits. The Q_H and q_s factors are inherent to the game-based Fiat–Shamir reduction and shared by all threshold Fiat–Shamir schemes; they do not correspond to any known attack. $\square$

Remark 29 (Loss Comparison with Prior Work). Hermine [BCdP⁺26] and Mithril [CdPE⁺26] incur an additional $q_s \cdot \text{SD}_{\text{nonce}}$ term from the nonce distribution not matching single-party ML-DSA exactly. In TALUS-TEE, the Shamir nonce DKG produces an Irwin-Hall-distributed aggregate nonce [Kao26]; the per-session Rényi divergence $\log_2 R_2^{\text{vec}}(T)$ ranges from 0.003 bits ($T = 3$) to 0.099 bits ($T = 128$) (Table 6). Security is dominated by $\log_2 q_s$ (≈ 10.6 bits for $q_s = 1600$), not by the threshold T (< 0.1 bits even at $T = 128$).

8.2 Privacy of TALUS-TEE

Theorem 24 (TALUS-TEE Statistical Privacy). *TALUS-TEE provides statistical privacy (Definition 3) against any $T - 1$ corrupted signers. The view of corrupted parties during signing is simulatable given only pk and the output signature, with statistical distance 0.*

Proof. Corrupted parties see: (a) the nonce DKG contributions $g_h(i)$ from honest parties (for $h \in \text{honest}, i \in C$); (b) the broadcast $\mathbf{w}_1$; (c) the challenge $\tilde{c}$; (d) the honest parties’ responses $\mathbf{z}_i$ for $i \notin C$.

By Shamir privacy, $T - 1$ evaluations of a degree- $(T - 1)$ polynomial are uniformly random; the simulator draws them uniformly. The broadcast $\mathbf{w}_1$ is recoverable from (pk, σ) directly: $\mathbf{w}_1 = \text{UseHint}(\mathbf{h}, \mathbf{A}\mathbf{z} - \mathbf{c}\mathbf{t}_1 \cdot 2^d, 2\gamma_2)$ (all public values), so the simulator sets $\mathbf{w}_1$ accordingly. Given $\mathbf{w}_1$ and $\tilde{c}$, the responses $\mathbf{z}_i = \mathbf{y}_i + \mathbf{c}\mathbf{s}_{1,i}$ are determined. The simulator generates these consistently using the simulated nonce shares. Statistical distance is 0 (Shamir perfect privacy; all other values are deterministic functions of public inputs). $\square$

8.3 Security of TALUS-MPC

Security of TALUS-MPC requires two additional hops beyond TALUS-TEE: one for the CSCP’s Beaver triple security, and one for the masked broadcast’s privacy.

Lemma 7 (CSCP Masking Under Malicious Deviation). *Let $|C| \leq T - 1$ denote the corrupted set. Even if corrupted parties send arbitrary CSCP shares in the prefix-comparison phase, the adversary’s additional information about honest parties’ carry masks $\{\rho_h\}_{h \notin C}$ beyond the CSCP outputs (c, δ) is simulatable from (pk, σ) alone.*

Proof. We first establish what the adversary can compute, then show it is already simulatable.

What the adversary learns from the CSCP transcript. Each corrupted party $j \in C$ holds the pairwise session key $K_{hj,\tau}$ for every $h \in S$ (established during key generation; s_{hj} is known to j). Consequently, all Beaver triple shares $[a_g]_h = \text{PRF}(K_{hj,\tau}, \cdot)$ of honest party h are computable by the adversary from known seeds. Hence the adversary can compute the full Beaver mask a_g , and from the public value $d_g = x_g - a_g$ recover $x_g = d_g + a_g$ exactly for each gate g . From $\{x_g\}$ they reconstruct $\Sigma_i \rho_i$; together with their known $\{\rho_j\}_{j \in C}$, they obtain $\rho_h = \Sigma_i \rho_i - \Sigma_{j \in C} \rho_j$ for

the unique honest party. Note this argument applies equally in the semi-honest model: learning ρ_h from the CSCP transcript is not a consequence of malicious deviation but of the pairwise-seed construction with $T - 1$ corruptions. The situation under malicious deviation is no worse.

Why this is simulatable. The value $\rho_h = b_h \bmod \alpha$ is the low-bits residue of the nonce commitment; it is *not* a function of the signing key $\mathbf{s}_{1,h}$. The carry bit $c = [\sum_i \rho_i > t]$ and correction bit δ are recoverable from $(\mathbf{pk}, \sigma)$ and the public masked broadcasts $\{b_i\}$ as shown in Remark 31. Conditioned on (c, δ) , the value ρ_h is uniform over a sub-interval of $[0, \lfloor \alpha/N \rfloor]$ of size at least $\lfloor \alpha/(2N) \rfloor$; the simulator draws ρ_h^* from the same conditional distribution and sets $x_g^* = (\text{bits of } \sum_{j \in C} \rho_j + \rho_h^*)$, $d_g^* = x_g^* - a_g$ (using the known a_g). By the range-restricted mask construction (Lemma 11), the real and simulated ρ_h have the same conditional distribution given (c, δ) ; hence the simulated transcript is statistically indistinguishable from the real one. The malicious adversary's arbitrary deviation in $[d]_i$ contributes only public additive terms $\lambda_i \delta_i$ to the reconstruction of d_g , which the simulator accounts for identically. $\square$ $\square$

Lemma 8 (Blame Session Privacy). *When Protocol 19 is triggered for a failed session τ and each honest party h reveals $O_h = (\hat{\mathbf{y}}_h, \{\hat{\mathbf{a}}_{h,k}\}, \rho_h, \{K_{h,j,\tau}\}_{j \in S})$, the adversary's view of O_h is simulatable without knowledge of honest parties' signing key shares $\{\mathbf{s}_{1,h}\}$, under PRF security of the pairwise seed construction.*

Proof. We examine each component of O_h :

1. $\hat{\mathbf{y}}_h$: the nonce constant term, sampled uniformly in $[-\gamma_1/|S| + 1, \gamma_1/|S|]^{n_e}$ fresh for each session; independent of $\mathbf{s}_{1,h}$. Simulator draws $\hat{\mathbf{y}}_h^*$ from the same distribution.
2. $\{\hat{\mathbf{a}}_{h,k}\}$: nonce DKG polynomial coefficients, consistent with $\hat{\mathbf{y}}_h$ via the Feldman commitment $\Phi_{h,0}$. Simulator generates $\{\hat{\mathbf{a}}_{h,k}^*\}$ consistent with $\hat{\mathbf{y}}_h^*$ and the public commitments.
3. ρ_h : carry mask in $[0, \lfloor \alpha/N \rfloor]$, a function of session- τ nonce data, independent of $\mathbf{s}_{1,h}$. Simulator draws ρ_h^* uniformly from the same interval.
4. $K_{h,j,\tau} = \text{PRF}(s_{hj}, \text{"session"} \parallel \tau)$: session-specific; by PRF security, $K_{h,j,\tau}$ is computationally uniform and reveals nothing about s_{hj} or any other session key $K_{h,j,\tau'}$ ($\tau' \neq \tau$). Simulator programs the PRF oracle at input $(\cdot, \text{"session"} \parallel \tau)$.

The simulated O_h^* is computationally indistinguishable from the real O_h , reducing to PRF security. Future sessions $\tau' \neq \tau$ are unaffected since s_{hj} is not revealed and $K_{h,j,\tau'} \neq K_{h,j,\tau}$ by PRF pseudorandomness. $\square$ $\square$

Theorem 25 (TALUS-MPC EUF-CMA). *Let $\mathcal{A}$ be a PPT adversary that corrupts at most $T - 1$ parties (static, **malicious + identifiable abort**, $T \geq 2$; for $T \geq 3$ additionally requires $N \geq 2T - 1$), makes at most q_s signing queries and Q_H random oracle queries. CarryCompare instantiates as DCF [BGI15, BGI16] for $T = 2$ or CSCP (Section 7.3.2) for $T \geq 3$. Then:*

$$\text{Adv}_{\text{TALUS-MPC}}^{\text{EUF-CMA}}(\mathcal{A}) \leq \frac{Q_H^2}{|C|} + q_s \cdot \epsilon_{\text{IH}}(T) + \text{Adv}_{\text{Beaver}}^{\text{PRF}}(\mathcal{B}_1) + \text{Adv}_{\text{ML-DSA}}^{\text{EUF-CMA}}(\mathcal{B}_2)$$

where $\epsilon_{\text{IH}}(T) = R_2^{\text{vec}}(T) - 1$ is the per-session shift-invariance loss (Section 7.9, Table 6), and $\text{Adv}_{\text{Beaver}}^{\text{PRF}}$ bounds the PRF advantage of breaking the pairwise seed-based Beaver triple construction.

Proof. We define six games.

Game 0: Real TALUS-MPC (malicious adversary). $\mathcal{A}$ interacts with the real protocol and may instruct corrupted parties to deviate arbitrarily or abort. By definition, $\Pr[\mathcal{A} \text{ wins}] = \text{Adv}_{\text{TALUS-MPC}}^{\text{EUF-CMA}}(\mathcal{A})$.

Game 0': Malicious-to-semi-honest reduction. We argue $|\Pr[G0'] - \Pr[G0]| = 0$ for EUF-CMA. Any malicious deviation by a corrupted party must fall into one of three cases:

1. *Protocol abort.* By Theorem 20 (IA Completeness), the blame mechanism outputs $\text{Blame}(i)$ for some $i \in C$. No valid signature is produced; the adversary does not win the EUF-CMA game.
2. *Invalid signature.* The deviation corrupts $\mathbf{w}_1$ or a partial response $\mathbf{z}_i$, causing the aggregated σ to fail Verify. No forgery is produced.

3. *Undetectable deviation.* The deviation produces a valid signature σ on the *queried* message m (hence not a forgery on $m^* \neq m$). We argue this case provides no additional advantage toward forging on a fresh m^* . By definition of “undetectable”, the output σ passes $\text{Verify}(\text{pk}, m, \sigma) = 1$, so the adversary’s malicious protocol messages produce the same output distribution as an honest signing oracle call on m . The adversary’s view (including any biased intermediate values from corrupted parties) therefore reveals no more information about $\mathbf{s}_1$ than a regular signing query does: the only information extractable is $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$, which is already provided by the honest signing oracle and accounted for in the ML-DSA EUF-CMA reduction. Producing a valid σ^* on a *fresh* m^* still requires computing $c^* = H(\text{pk} \parallel \mu^* \parallel \mathbf{w}_1^*)$ and a valid $\mathbf{z}^*$ with $\|\mathbf{z}^*\|_\infty < \gamma_1 - \beta$, which is precisely winning ML-DSA EUF-CMA against the underlying scheme.

Note that corrupted CSCP evaluations (wrong Beaver shares) corrupt the carry bit c , causing the masked $\mathbf{w}_1$ to be incorrect. An incorrect $\mathbf{w}_1$ changes the signing challenge $\tilde{c} = H(\text{pk} \parallel \mu \parallel \mathbf{w}_1)$, so honest parties compute responses $\mathbf{z}_i = \hat{\mathbf{y}}_i + \tilde{c}\mathbf{s}_{1,i}$ with the wrong $\tilde{c}$; the aggregated signature then fails Verify , placing the deviation in case 2 (invalid signature) or triggering case 1 via Blame (Theorem 20, case 3). In either outcome, no forgery is produced. In no case does a malicious deviation increase the probability of winning the EUF-CMA game beyond the semi-honest bound. (Note: Theorem 20 states Blame succeeds with probability $1 - \text{negl}(\kappa)$; the negligible failure probability is subsumed by $\text{Adv}_{\text{Beaver}}^{\text{PRF}}$ in Game 2, since bypassing blame attribution requires forging a session-key-derived Beaver triple authentication, which contradicts PRF security.) We therefore pass to the semi-honest-with-abort analysis from Game 1 onward, with no additional loss.

Game 1: Shamir privacy. Replace honest nonce DKG shares $g_h(i)$ (for $i \in C$, corrupted) with uniformly random elements of $\mathbb{Z}_q^{n_\ell}$. By the perfect statistical privacy of Shamir secret sharing [Sha79], $|\Pr[\mathcal{A} \text{ wins G1}] - \Pr[\mathcal{A} \text{ wins G0}]| = 0$.

Game 2: Simulate CarryComparerandomness. For $T \geq 3$: replace the PRF-based Beaver triple generation with truly random triples $(\langle a_g \rangle, \langle b_g \rangle, \langle c_g \rangle)$. Any adversary distinguishing this transition breaks the PRF security of the pairwise seed construction (Lemma 12). For $T = 2$: replace the DCF key-generation PRF with a truly random function; any adversary distinguishing this transition breaks DCF key-indistinguishability [BGI15, BGI16]. In both cases:

$$|\Pr[\mathcal{A} \text{ wins G2}] - \Pr[\mathcal{A} \text{ wins G1}]| \leq \text{Adv}_{\text{Beaver}}^{\text{PRF}}(\mathcal{B}_1).$$

After this replacement, the CarryCompare outputs are statistically independent of the masked broadcasts $(\tilde{H}_i, \tilde{b}_i)$.

Game 3: Hash collision abort + BCC invariant. We combine two transitions (each of cost zero in isolation) into a single game hop, analogous to Games 2–3 in the TALUS-TEE proof (Theorem 23), but applied together here: (i) abort on hash collision (birthday bound $Q_H^2/|C|$); (ii) invoke the BCC invariant maintained by preprocessing (statistical distance 0, since BCC is a predicate on public $\hat{\mathbf{w}}$ that does not depend on c or $\mathbf{s}_2$). The combined bound is:

$$|\Pr[\text{G3}] - \Pr[\text{G2}]| \leq Q_H^2/|C| + q_s \cdot \epsilon_{\text{IH}}(T).$$

The $\epsilon_{\text{IH}}(T)$ term arises from the Irwin-Hall nonce distribution introduced in Game 1 (Shamir nonce DKG); it is collected here for presentation uniformity with the TALUS-TEE proof (Theorem 18). The final bound is unchanged. In Game 3, each signing session has a distinct random challenge c , and $\text{BCC}(\hat{\mathbf{w}})$ holds by the preprocessing filter.

Game 4: Reduce to ML-DSA EUF-CMA. In Game 3, the signing transcript consists of: (1) masked broadcasts $(\tilde{H}_i, \tilde{b}_i)$ (independent of $\mathbf{s}_1$ by the pairwise mask construction); (2) CSCP outputs (c, δ) used with masked broadcasts to compute $\mathbf{w}_1$ (a deterministic function of the public key and nonces, not $\mathbf{s}_1$); (3) responses $\mathbf{z}_i = \hat{\mathbf{y}}_i + c\mathbf{s}_{1,i}$. The combined response $\mathbf{z} = \hat{\mathbf{y}} + c\mathbf{s}_1$ has the same distribution as a single-party ML-DSA signing oracle output. Any forgery $\sigma^* = (\tilde{c}^*, \mathbf{z}^*, \mathbf{h}^*)$

is a valid FIPS 204 signature on a fresh message. We construct $\mathcal{B}_2$ that simulates Game 3 using the ML-DSA oracle. Hence:

$$\Pr[\mathcal{A} \text{ wins G4}] \leq \text{Adv}_{\text{ML-DSA}}^{\text{EUF-CMA}}(\mathcal{B}_2).$$

Chaining the six games gives the stated bound. $\square$ $\square$

Theorem 26 (TALUS-MPC Privacy). *In the **malicious + identifiable-abort** model with $T - 1$ corrupted parties ($T \geq 2$; for $T \geq 3$ additionally $N \geq 2T - 1$), the joint view of the adversary in TALUS-MPC, including any blame reveals triggered by malicious deviations, is computationally indistinguishably simulatable from (pk, σ) alone, under PRF security of the pairwise seed construction.*

Proof. The simulator handles the adversary’s view in two parts: the *normal execution* view and the *blame reveal* view.

Normal execution view. Corrupted parties see:

1. Nonce DKG shares $g_h(i)$ for $i \in C$: uniformly random by Shamir perfect privacy ($T - 1 < T$ evaluations); statistical distance 0.
2. Masked broadcasts $(\tilde{H}_j, \tilde{b}_j)$ for honest j : $\tilde{H}_j = (w_{1,j} + \text{mask}_j^H) \bmod 16$, where mask_j^H contains at least one PRF output with unknown seed; thus $\tilde{H}_j$ is computationally indistinguishable from uniform in $\mathbb{Z}_{16}$ (PRF security, Lemma 11). $\tilde{b}_j = b_j + \rho_j$ with $\rho_j \in [0, \lfloor \alpha/N \rfloor]$ drawn uniformly; this statistically hides b_j within an interval of size $\lfloor \alpha/N \rfloor$, with no additional computational assumption needed.
3. CSCP masked evaluations (d_g, e_g) from honest parties: *even under malicious deviation by corrupted parties* (arbitrary wrong shares $[d]_i, [e]_i$ for $i \in C$), the honest contribution is masked by a PRF-pseudorandom Beaver term (Lemma 7). Thus (d_g, e_g) are computationally indistinguishable from uniform under PRF security (Lemma 12).
4. The output $\mathbf{w}_1$: computed from (pk, σ) as $\text{UseHint}(\mathbf{h}, \mathbf{Az} - \mathbf{ct}_1 \cdot 2^d, 2\gamma_2)$ (public; uses only $\text{pk} = (\rho, \mathbf{t}_1)$ and $\sigma = (\tilde{c}, \mathbf{z}, \mathbf{h})$).
5. Responses $\mathbf{z}_j$ for honest j : generated consistently from the signing oracle.

The simulator draws items 1–5 as described. Items 1 and 2(b) have statistical distance 0 from the real view. Items 2(a) and 3 are computationally indistinguishable under PRF security.

Blame reveal view. If the adversary causes a protocol failure (by malicious deviation or abort), Protocol 19 triggers and each honest h reveals $O_h = (\hat{\mathbf{y}}_h, \{\tilde{\mathbf{a}}_{h,k}\}, \rho_h, \{K_{h,j,\tau}\})$. By Lemma 8, the simulator can produce O_h^* computationally indistinguishable from the real O_h without knowing $\mathbf{s}_{1,h}$, under PRF security.

Combining both parts: the full simulated view (normal execution plus any blame reveals) is computationally indistinguishable from the real adversarial view, with reduction to the PRF advantage of the pairwise seed construction. $\square$ $\square$

Remark 30 (Leakage from Aborted Nonces). When $\text{BCC}(\hat{\mathbf{w}})$ fails, the preprocessed nonce is discarded and a fresh one is prepared. We verify that aborted nonces leak no useful information about the signing keys.

TALUS-TEE. The coordinator’s TEE holds $\hat{\mathbf{y}}$ in a protected enclave. When BCC fails, the only observable event is the coordinator advancing to the next preprocessed nonce; no value derived from $\hat{\mathbf{w}}$ or $\hat{\mathbf{y}}$ is ever transmitted to parties. Leakage from aborted nonces is therefore zero.

TALUS-MPC. During preprocessing, parties broadcast masked values $(\tilde{H}_h, \tilde{b}_h)$ before the BCC check is evaluated (the coordinator cannot evaluate BCC until all broadcasts arrive). The abort bit (“nonce rejected”) is public. We analyse what an adversary controlling $T - 1$ parties learns from an aborted session:

1. $(\tilde{H}_h, \tilde{b}_h)$ for honest h : already proven computationally indistinguishable from uniform (masked broadcast privacy; Lemma 11).

2. $\mathbf{w}_1 = \text{HighBits}(\hat{\mathbf{w}})$: computed from the broadcasts and CSCP output. This is a function of the nonce, not the signing key. The nonce polynomial g_h has a fresh constant term $\hat{\mathbf{y}}_h$ sampled uniformly in $[-\gamma_1/|S| + 1, \gamma_1/|S|]^{n_\ell}$ for each session; $\hat{\mathbf{w}}$ is M-LWE-pseudorandom and independent of $\mathbf{s}_1$.
3. The BCC failure event itself reveals that $\|\mathbf{r}_0\|_\infty \geq \gamma_2 - \beta$ (i.e., at least one coefficient of $\hat{\mathbf{w}}$'s low bits is near a boundary). This is a predicate on the public $\hat{\mathbf{w}}$, not on any secret key component. Since $\hat{\mathbf{w}}$ is already recoverable (up to M-LWE hardness) from $\tilde{H}_h, \tilde{b}_h$, the BCC failure bit is subsumed by the existing masked broadcast leakage.

Thus aborted nonces provide no additional information about $(\mathbf{s}_1, \mathbf{s}_2)$ beyond what is already accounted for in the 6-game proof. The total leakage per aborted nonce is bounded by $2q'_s$ bits (from the CSCP outputs) where q'_s is the number of aborted sessions, and does not affect M-LWE security at the 128-bit level.

Remark 31 (Leakage from the Extended CSCP Outputs (c, δ)). The extended CSCP outputs $(c, \delta) \in \{0, 1\}^2$, revealing at most 2 bits about $\Sigma \rho_i$ per session. The carry bit $c = [\Sigma \rho_i > t]$ partitions $[0, \alpha)$ into at most 2 intervals; the FIPS correction bit $\delta = [\Sigma \rho_i < t - \gamma_2 + c\alpha]$ makes a further binary split. Over q_s sessions with fresh masks, the total leakage is $2q_s$ bits. By the range-restricted mask, each b_i is determined only to within α/N , so individual $r_{0,i}$ remain hidden within $\pm \gamma_2/N$ even given (c, δ) and all $\tilde{b}_i$. For standard parameters ($q_s \leq 10^6$, $N \geq 2$), this accumulation does not threaten M-LWE at the 128-bit security level.

8.4 Combiner Privacy (M-LWR Reduction)

The combiner observes each party i 's partial HighBits value $h_i = \text{HighBits}(\lambda_i^{(S)} \cdot \mathbf{A}\mathbf{y}_i, 2\gamma_2)$ during the offline carry resolution. We show that this leaks no information about the key shares.

Proposition 1 (Combiner Privacy). *Under the Module-LWR assumption, the per-party value $h_i = \text{HighBits}(\mathbf{A}(\lambda_i^{(S)} \mathbf{y}_i), 2\gamma_2)$ is computationally indistinguishable from a uniformly random element of $\{0, \dots, 15\}^{kn}$.*

Proof. Each Shamir share $\mathbf{y}_i \in R_q^\ell$ has coefficients uniform over $\mathbb{Z}_q$ (by the perfect privacy of Shamir sharing with $T-1 < T$ evaluations [Sha79]). Since $\lambda_i^{(S)} \in \mathbb{Z}_q \setminus \{0\}$ is a non-zero Lagrange coefficient (distinct evaluation points ensure $\lambda_i^{(S)} \neq 0$), the product $\mathbf{u}_i = \lambda_i^{(S)} \mathbf{y}_i$ is also uniformly distributed over R_q^ℓ .

The tuple $(\mathbf{A}, \text{HighBits}(\mathbf{A}\mathbf{u}_i, 2\gamma_2))$ is an instance of Module-LWR with parameters: matrix $\mathbf{A} \in R_q^{k \times \ell}$ (public), secret $\mathbf{u}_i \xleftarrow{\$} R_q^\ell$ (uniform), rounding ratio q/p where $p = (q-1)/\alpha = 16$. The rounding ratio is $q/p = 8,380,417/16 = 523,776.0625$, i.e., $\log_2(q/p) \approx 19$ bits of rounding noise. Module-LWR with this parameterization is hard under the standard reduction from Module-LWE via deterministic rounding [BPR12].

Across signing sessions, each session uses an independent nonce $\mathbf{y}_i$, producing an independent M-LWR instance. Multi-session M-LWR security follows from a standard hybrid argument: distinguish one instance at a time, accumulating at most q_s times the single-instance M-LWR advantage. $\square$

8.5 Carry Leakage Analysis

The carry $\delta_j \in \{-1, 0, +1\}$ is revealed during the offline carry resolution. We analyse the information content and show that it reveals nothing about long-term keys.

Proposition 2 (Carry Independence). *Let δ_j denote the carry at coefficient j , computed from a nonce $\mathbf{y} \xleftarrow{\$} \{-\gamma_1+1, \dots, \gamma_1\}^{n_\ell}$. Then:*

- (i) δ_j is a deterministic function of $\text{LowBits}((\mathbf{A}\mathbf{y})_j)$, which depends only on the nonce $\mathbf{y}$ and the public matrix $\mathbf{A}$;
- (ii) δ_j is statistically independent of the long-term keys $\mathbf{s}_1$ and $\mathbf{s}_2$;
- (iii) the carry vectors from distinct signing sessions are mutually independent.

Proof. (i) The carry at coefficient j is $\delta_j = \lfloor S_j/\alpha \rfloor$ where $S_j = \sum_{i \in S} \text{LowBits}((\mathbf{A}\mathbf{y}_i)_j)$. The quantities $\mathbf{A}$, $\mathbf{y}_i$, and the Lagrange coefficients $\lambda_i^{(S)}$ fully determine S_j . The keys $\mathbf{s}_1, \mathbf{s}_2$ do not appear in this computation.

(ii) The nonce $\mathbf{y}$ is sampled independently of the keys during each signing session. The matrix $\mathbf{A}$ is a deterministic function of the public seed ρ , which is independent of $\mathbf{s}_1$ and $\mathbf{s}_2$ (both are sampled using independent randomness in `KeyGen`). Since δ_j is a function of $\mathbf{A}$ and $\mathbf{y}$ only, and $(\mathbf{A}, \mathbf{y}) \perp (\mathbf{s}_1, \mathbf{s}_2)$, the carry is independent of the keys.

(iii) Each signing session draws a fresh nonce independently. The carry vectors across sessions are therefore independent. $\square$

Information content per session. The entropy of a single carry $\delta_j \in \{-1, 0, +1\}$ is at most $\log_2 3 \approx 1.585$ bits. For the empirical distribution ($\approx 50\%$ probability of $\delta_j = 0$ and 25% each for ± 1), the entropy is ≈ 1.5 bits. Across all $kn = 1536$ coefficients, the total information content per session is approximately $1536 \times 1.5 = 2304$ bits = 288 bytes.

However, this information is entirely about the *nonce* $\mathbf{y}$ (which is ephemeral and independent per session), not about the long-term keys. No accumulation of carry values across sessions can yield information about $\mathbf{s}_1$ or $\mathbf{s}_2$, because the carry values are independent of the keys by Proposition 2(ii).

9 Implementation

We implement TALUS-TEE and TALUS-MPC in Rust (edition 2021), extending the ML-DSA-65 implementation from [Kao26] to support all three FIPS 204 security levels (ML-DSA-44, ML-DSA-65, ML-DSA-87) via a trait-parameterised architecture. The codebase is structured as a Cargo workspace with four crates: `talus-core` (ML-DSA primitives, Shamir sharing, CEF, BCC), `talus-tee` (TEE keygen, preprocessing, online signing), `talus-mpc` (MPC keygen, masked preprocessing, online signing), and `talus-bench` (Criterion benchmarks). All polynomial arithmetic uses NTT-based negacyclic multiplication over $\mathbb{Z}_q[X]/(X^{256} + 1)$: each polynomial product is computed as forward-NTT, pointwise multiply, inverse-NTT (8 butterfly levels, $O(n \log n)$ per product).

9.1 Experiments

BCC rate (VERIFY-A). `experiments/bcc_rate.py`: Monte Carlo over 10^5 fresh random nonces confirms $p_{\text{BCC}} = 0.3138$ (theory 0.3166; $|\Delta| < 0.003$; 100 samples, seed 2025). Expected hint weight under BCC: $\mathbb{E}[\text{wt}(\mathbf{h}) \mid \text{BCC}] \approx 38.2$; $\Pr[\text{wt} > \omega] < 10^{-3}$.

TALUS-TEE success rate (VERIFY-B). `experiments/talus_tee.py`: end-to-end signing with BCC filter over 10 000 attempts per $|S| \in \{2, 3, 5\}$. Results: $p_{\text{success}} \approx 31.1\text{--}32.2\%$ ($\Pr[\text{z-bound} \mid \text{BCC}] = 1.000$; $\Pr[\text{wt}(\mathbf{h}) \leq \omega \mid \text{BCC}] \approx 0.995\text{--}0.998$).

Carry identity (VERIFY-E). `experiments/carry_identity.py`: CEF protocol produces identical $\mathbf{w}_1$ to direct computation for all $T \in \{2, 3, 4, 5, 8\}$ over $10^5 \times 1536$ coefficient trials (100% agreement).

TALUS-MPC success rate (VERIFY-C). `experiments/talus_mpc.py`: combined BCC + z-bound + hint-weight success rate over 5 000 attempts per $(T, |S|) \in \{(2, 2), (3, 3), (3, 5), (5, 5)\}$. Results: $p_{\text{success}} \approx 31.4\text{--}33.7\%$ ($\Pr[\text{carry correct}] = 1.000$; $\Pr[\text{z-bound}] \approx 1.000$; $\Pr[\text{wt}(\mathbf{h}) \leq \omega] \approx 0.993\text{--}0.995$). The success rate matches TALUS-TEE since the z-bound passes with overwhelming probability at all tested thresholds.

Table 8: TALUS multi-level performance (Criterion median, release build). Preprocess = per BCC attempt (offline, pre-computable). Online = coordinator aggregation (deterministic). All times in milliseconds.

Level	$T=2$	$T=3$	$T=5$	$T=10$	$T=20$
<i>TALUS-TEE preprocess (ms/attempt)</i>					
ML-DSA-44	0.13	0.20	0.43	2.41	18.5
ML-DSA-65	0.29	0.39	0.84	3.42	29.0
ML-DSA-87	0.20*	0.60	1.19	6.04	40.8
<i>TALUS-TEE online (ms)</i>					
ML-DSA-44	0.11	0.16	0.26	0.47	1.03
ML-DSA-65	0.22	0.29	0.44	0.85	1.64
ML-DSA-87	0.29	0.41	0.61	1.10	2.23
<i>TALUS-MPC preprocess (ms/attempt)</i>					
ML-DSA-44	0.45	0.78	1.71	6.75	33.1
ML-DSA-65	0.63	1.08	2.21	8.62	42.7
ML-DSA-87	0.94	1.56	3.31	12.5	55.8
<i>TALUS-MPC online (ms)</i>					
ML-DSA-44	0.41	0.46	0.52	0.74	1.12
ML-DSA-65	0.61	0.68	0.79	1.01	1.69
ML-DSA-87	0.87	1.03	1.23	1.66	2.38

*ML-DSA-87 preprocess at $T=2$ shows anomalously wide Criterion CI ($[183, 211] \mu s$); within measurement noise.

9.2 Performance

All benchmarks were run on an Intel Core Ultra 9 285H (6P+8E cores, 3.8 GHz P-core base), 32 GB DDR5 RAM, Windows 11 Pro, using Rust 1.88.0 (`rustc 1.88.0 (6b00bc388 2025-06-23)`, Cargo 1.88.0), compiled with `-release` (equivalent to `-C opt-level=3`). All benchmarks are *single-threaded* and *unvectorised* (no SIMD); AVX2 or similar would reduce times further.

Table 8 reports coordinator-side latency across all three FIPS 204 security levels, measured with Criterion 0.5 [HA23] (100 samples, warm-up 3s, *release* profile). “Preprocess” is the time per nonce *attempt* (single call to `batch_preprocess` with `batch_size = 1`). “Online” is the full coordinator aggregation (challenge hash + T partial responses + aggregate + hint + verify). The BCC pass rate varies by level ($p_{\text{BCC}} \approx 43\%$ for ML-DSA-44, 32% for ML-DSA-65, 39% for ML-DSA-87); the expected retries per accepted nonce are $1/p_{\text{BCC}}$.

Table 9 extends the evaluation across all three FIPS 204 security levels at $T = 3$ and compares with Mithril [CdPE⁺26], a concurrent threshold ML-DSA scheme. The BCC pass rate $p_{\text{BCC}} = (1 - \beta/\gamma_2)^{K \cdot 256}$ varies non-monotonically across levels: ML-DSA-87 ($p_{\text{BCC}} \approx 39\%$) is higher than ML-DSA-65 ($p_{\text{BCC}} \approx 32\%$) because $\beta_{87} = \tau\eta = 120 < 196 = \beta_{65}$ outweighs the larger dimension $K_{87} = 8 > 6 = K_{65}$.

Online latency. Online signing (coordinator side) costs 0.11–2.38 ms for ML-DSA-44/65/87 at $T \in \{2, \dots, 20\}$ (Table 8), dominated by the matrix-vector product $\mathbf{A}\mathbf{z}$ for the hint computation. The cost scales with both T (more partial responses) and security level ($k\ell$ increases from 16 to 56). Signer-side partial response $\mathbf{z}_i = \hat{\mathbf{y}}_i + c\mathbf{s}_{1,i}$ is a single polynomial vector multiplication ($\ll 0.1$ ms per signer, not shown).

Offline latency. Preprocessing per attempt scales as $O(T \cdot n_\ell)$ for the nonce DKG (matrix-vector product $\mathbf{A}\hat{\mathbf{y}}_i$ per party) plus $O(T^2)$ for pairwise mask generation (MPC only). At ML-DSA-65, $T = 3$, the MPC offline cost (1.08 ms/attempt) is $2.8\times$ the TEE cost (0.39 ms/attempt), with the difference attributable to the masked CEF protocol, pairwise mask generation, and CSCP simulation in TALUS-MPC.

Table 9: Multi-level performance at $T=3$ and comparison with Mithril [CdPE+26]. TALUS preprocessing is amortised by $1/p_{\text{BCC}}$ (offline, pre-computable). Mithril numbers from Table 9 of [CdPE+26] (MacOS M3, Go/CIRCL), amortised over successful signing. All times in milliseconds.

Scheme	Level	p_{BCC}	Preprocess	Online	Total
TALUS-TEE	ML-DSA-44	43.2%	0.46	0.16	0.62
TALUS-TEE	ML-DSA-65	31.7%	1.23	0.29	1.52
TALUS-TEE	ML-DSA-87	39.1%	1.53	0.41	1.94
TALUS-MPC	ML-DSA-44	43.2%	1.81	0.46	2.27
TALUS-MPC	ML-DSA-65	31.7%	3.41	0.68	4.09
TALUS-MPC	ML-DSA-87	39.1%	3.99	1.03	5.02
Mithril [CdPE+26]	ML-DSA-44	—	—	1.4 [†]	1.4
Mithril [CdPE+26]	ML-DSA-65	—	—	3.6 [†]	3.6
Mithril [CdPE+26]	ML-DSA-87	—	—	3.8 [†]	3.8

[†]Mithril performs rejection sampling *during* online signing (Fiat–Shamir with aborts, $\approx 50\%$ success rate); no offline preprocessing. Numbers are per-party, amortised over successful attempts ($T=3$, $N=3$).

Table 10: TALUS-MPC (3-of-5) end-to-end online signing latency over GCP (100 rounds each; all signatures valid). *Network* = time until all partial responses received; *Total* = network + coordinator aggregation + verification. Coordinator: **us-central1-b** (Iowa).

Scenario	Signer locations	Network (ms)		Total (ms)	
		Median	Max	Median	Max
Cross-zone	us-central1-b/f	2.88	4.15	3.13	4.41
US→Europe	europe-west1-b	349.7	3002	349.9	3003
US→Asia	asia-northeast1-b	439.6	639	439.9	640

Distributed network experiment. To validate end-to-end signing latency over real networks, we deployed TALUS-MPC (3-of-5) on Google Cloud Platform (**n2-standard-4** VMs, 4 vCPUs, 16 GB RAM, Debian 12, Linux kernel 6.1, Rust 1.85.1) across three geographic scenarios, each running 100 consecutive signing rounds (100/100 successful in all cases). The coordinator resided in **us-central1-b** (Iowa); one signer was co-located in the same zone and two signers were placed in progressively distant regions. Table 10 summarises the results; the single high-latency outlier in the US–Europe run (3002 ms Max) is a one-time GCP network re-routing event and does not affect the median.

Since TALUS online signing requires exactly *one round-trip*, total latency equals one network RTT plus a computation overhead of ≈ 0.24 ms (measured across all three GCP scenarios). The pure single-core benchmark (Table 8) reports 0.68 ms (MPC online, ML-DSA-65, $T=3$); the difference reflects network-timing measurement overhead and kernel scheduling variance in the distributed experiment. This confirms the key property: **TALUS signing latency scales with the network RTT alone**, not the number of rounds. Cross-zone GCP yields sub-5 ms end-to-end; a transatlantic deployment (US–Europe) yields ≈ 350 ms; a transpacific deployment (US–Tokyo) yields ≈ 440 ms.

Comparison with prior work. Table 11 compares all five known FIPS 204-compatible threshold ML-DSA schemes on structural properties and amortised signing latency; Table 9 provides a detailed multi-level breakdown for TALUS and Mithril.

TALUS is the only FIPS 204-compatible scheme with a *rejection-free* single-round online phase. Mithril [CdPE+26] is the closest competitor: at ML-DSA-65 with $T=3$, TALUS-TEE is $2.4\times$ faster (but requires trusted hardware) while TALUS-MPC is $1.1\times$ slower (Table 9). However, Mithril requires 3 online rounds with Fiat–Shamir abort-and-retry ($\approx 50\%$ per-attempt

Table 11: Comparison of FIPS 204-compatible threshold ML-DSA schemes. Structural properties (top) are implementation-independent. Latency is single-core computation, amortised over rejection; platform differences preclude direct numerical comparison. DM = dishonest majority; HM = honest majority.

	TALUS -TEE	TALUS -MPC	Mithril [CdPE ⁺ 26]	Quorus [BdCE ⁺ 26]	Trili- thium [DKLS25]
Online rounds	1	1	3	16–29	14
Determ. online	✓	✓			
$N_{\max}$	∞	∞	6	∞	2
Security model	TEE	HM	DM	HM	2PC
<i>Amortised signing latency[†], $T=3$ (ms):</i>					
ML-DSA-44	0.62	2.27	1.4 ^a	13.2 ^b	$\geq 245^c$
ML-DSA-65	1.52	4.09	3.6 ^a	24.0 ^b	$\geq 310^c$
Platform	Rust, Ultra 9		Go, M3	C++, i7	Rust, Ryzen 5

[†]Single-core local simulation, amortised over rejection retries.

^aTable 9 of [CdPE⁺26]; $T=3$, $N=3$ (MacOS M3, Go/CIRCL).

^bTable 9 of [CdPE⁺26]; $n=3$, honest majority (Intel i7, C++/SCL).

^c2-party only ($T=2$); per-attempt compute, no network; requires correlated-randomness provider.

success) and supports at most $N_{\max} = 6$ parties. Quorus [BdCE⁺26] and Trilithium [DKLS25] achieve FIPS 204 compatibility through generic MPC, at the cost of 16–29 and 14 online rounds respectively: Quorus reaches 24ms amortised latency ($5.9\times$ TALUS-MPC), while Trilithium requires ≥ 310 ms and is limited to 2 parties. These raw numbers are not directly comparable due to platform differences (Table 11, bottom row), but the order-of-magnitude gaps reflect genuine structural costs.

The key advantage of TALUS is structural and implementation-independent: the online phase is a single round with *guaranteed* success. Over cross-zone GCP (≈ 3 ms RTT), the single-round design yields 3.1ms end-to-end (measured) versus $3 \times 3 = 9$ ms for a 3-round protocol (extrapolated). Over a transatlantic link (≈ 170 ms RTT), the gap grows to 350ms (measured) versus ≈ 510 ms. For high-round protocols such as Quorus (16–29 rounds), the network overhead alone would exceed 300ms even on a 10ms-RTT intranet. *Each saved online round eliminates one full network RTT*, regardless of deployment geography or cryptographic implementation quality.

Lattice threshold schemes that do *not* target FIPS 204 compatibility (T-Raccoon [dPKPR24], Hermine [BCdP⁺26], Ringtail [BKL⁺25]) report amortised signing latencies of 2–64ms at $T=3$ (Mithril Table 8, MacOS M3), with non-standard signature formats (11–13KB). TALUS achieves comparable or better computation while producing standard ML-DSA signatures, demonstrating that FIPS 204 compatibility need not entail a performance penalty.

Limitations. This reference implementation is a single-threaded, unvectorised Rust binary. SIMD-accelerated modular arithmetic (AVX2/NEON) would reduce NTT butterfly costs by $\approx 4\text{--}8\times$, and multi-party parallelism would amortise offline preprocessing across parties. These optimisations are left to a production deployment.

9.3 Communication Cost Breakdown

Table 12 breaks down per-batch and per-session communication by protocol component for several threshold sizes. For $T = 2$, the comparison degenerates to a distributed comparison function [BGI15, BGI16]: each coefficient requires a 2-bit carry tag, giving $1536 \times 2 = 3072$ bits = 384 bytes. For $T \geq 3$, the CarryCompare sub-protocol evaluates two prefix-tree comparisons per coefficient using preprocessed correlations [Bea92, DN07]. The carry identity (Section 5) reduces three comparisons to two, at 19 multiplications each, giving 38 multiplications per coefficient and $38 \times 1536 = 58,368$ total. Each Beaver-triple multiplication requires every party to broadcast

Table 12: Per-batch and per-session communication for TALUS (ML-DSA-65, $M = 30$ batches/session). P2P = peer-to-peer unicast; BC = broadcast. Online cost is identical for all T .

Component	$T=2$	$T=3$	$T=5$	$T=10$
NonceDKG (P2P)	3.6 KB	7.2 KB	14.4 KB	32.4 KB
h_i broadcast (BC)	768 B	768 B	768 B	768 B
Reshare (P2P)	0	7.4 KB	14.8 KB	33.3 KB
Mask-and-open (BC)	0	4.6 KB	4.6 KB	4.6 KB
CarryCompare (BC)	384 B	350 KB	350 KB	350 KB
Total per batch	~ 5 KB	~ 370 KB	~ 385 KB	~ 421 KB
Per session ($\times 30$)	~ 150 KB	~ 11.1 MB	~ 11.5 MB	~ 12.6 MB
Online per signer	3.6 KB	3.6 KB	3.6 KB	3.6 KB

two masked bits (d_i, e_i) ; with N parties this is $2N$ bits = $2 \times 3 = 6$ bytes for the default $N = 3$. The CarryCompare cost is therefore $58,368 \times 6 = 350,208$ bytes ≈ 350 KB per batch.

The dominant cost for $T \geq 3$ is CarryCompare, which accounts for $350/370 \approx 95\%$ of per-batch communication. The online phase is a single broadcast of $\mathbf{z}_i \in R_q^\ell$ ($\ell = 5$ polynomials, $1,280 \times 23 = 29,440$ bits ≈ 3.6 KB), independent of T and N .

9.4 Computation Costs

The computational cost is dominated by NTT multiplications over R_q .

Nonce generation and $\mathbf{Ay}$. Each party i computes $\mathbf{Ay}_i \in R_q^k$ via $k\ell = 30$ NTT multiplications (one per entry of the $k \times \ell$ public matrix $\mathbf{A}$), plus $k(\ell - 1) = 24$ additions. This is identical to the single-signer cost.

Carry comparison (offline). The 58,368 preprocessed multiplications require one NTT multiplication over R_q each to evaluate the Shamir product. The total offline compute per batch is $\sim 60,000$ NTT multiplications, compared to ~ 30 for $\mathbf{Ay}$. The carry comparison is $\sim 2000\times$ the nonce generation in NTT operations. However, each multiplication is over a single coefficient (a $\mathbb{Z}_q$ operation), not a full R_q polynomial, so the concrete cost per multiplication is $256\times$ smaller than a polynomial NTT multiply. In equivalent R_q multiplications, the comparison cost is $60,000/256 \approx 234$ polynomial multiplications per batch.

Online signing. Each signer computes $\mathbf{z}_i = \mathbf{y}_i + c \cdot \mathbf{s}_{1,i}$, requiring $\ell = 5$ polynomial multiplications. The combiner reconstructs $\mathbf{z}$ via Lagrange interpolation and evaluates $\mathbf{Az}$ ($k\ell = 30$ polynomial multiplications). All online computation is algebraic; no comparisons are evaluated.

9.5 Optimization Roadmap

Table 13 summarises four cumulative optimization layers for the CarryCompare sub-protocol. Layers 1 and 2 are the default instantiation analysed throughout this paper; Layers 3 and 4 are open research directions.

Layer 1: Carry identity. The identity $\delta = \lfloor B/\alpha \rfloor - [\rho > B \bmod \alpha]$ reduces three independent comparisons to two comparisons sharing the same mask ρ . The $1.5\times$ savings is exact.

Layer 2: Prefix-tree evaluation. The sequential evaluation strategy processes one bit per round, consuming exactly 1 preprocessed multiplication per bit. For a 19-bit comparison, this requires 19 multiplications per comparison vs. ~ 192 for a carry-lookahead (Kogge–Stone) prefix tree.

Table 13: Optimization layers for CarryCompare (ML-DSA-65, $T \geq 3$). Savings are cumulative relative to the naïve baseline of three carry-lookahead comparisons per coefficient.

Layer	Technique	Savings	Per-batch cost	Status
0	Naïve (3 carry-lookahead)	—	5.3 MB	Baseline
1	Carry identity (§5): $3 \rightarrow 2$ comp.	$1.5\times$	3.5 MB	Implemented
2	Prefix-tree evaluation: 1 mult/bit	$10\times$ vs carry-lookahead	350 KB	Implemented
3	$\mathbb{F}(2^{128})$ packed Shamir	$24\times$ (0.25 B/mult vs 6 B)	14.6 KB	Verified*
4	Multi-party DCF [BGI16]	$\sim 350\times$ (eliminate circuit)	~ 1 KB	Open

Layer 3: $\mathbb{F}(2^{128})$ packing. Packing 128 independent bit-level comparisons into a single $\mathbb{F}(2^{128})$ Shamir sharing reduces the per-multiplication cost to 0.25 bytes per logical multiplication, bringing the per-batch cost to $58,368 \times 0.25 = 14,592$ bytes ≈ 14.6 KB, a $24\times$ reduction over Layer 2. We verified the communication cost and protocol correctness experimentally: $\text{GF}(2^{128})$ field arithmetic (50,000 random triples), Shamir T -of- N sharing (15,000 trials, $T \in \{2, 3, 5\}$), and all 456 packed Beaver multiplications completed with zero errors. The remaining open challenges are (i) $\mathbb{F}(2^{128})$ preprocessing from pairwise PRF seeds, (ii) share conversion at the R_q -to- $\mathbb{F}(2^{128})$ field boundary, and (iii) a security proof for cross-field composition; the * marker in Table 13 reflects this.

*Communication cost and arithmetic verified by experiment; full integration (pairwise-seed preprocessing, field-boundary conversion, security proof) remains future work.

Layer 4: Multi-party DCF. Extending the $T = 2$ distributed comparison function [BGI15, BGI16] to $T \geq 3$ would eliminate the prefix-tree circuit entirely, reducing $T \geq 3$ costs to the $T = 2$ level (~ 384 bytes per batch). This would make $T \geq 3$ communication equal to $T = 2$, eliminating the threshold-communication trade-off entirely.

Commercial framing. Layers 1–2 are deployed in the current implementation. Layer 3 would reduce offline communication to within a single TCP segment (14.6 KB). Layer 4 would make $T \geq 3$ communication equal to $T = 2$, eliminating the threshold-communication trade-off entirely.

10 Discussion

10.1 FROST Structural Isomorphism

TALUS is structurally isomorphic to FROST [KG21, CKGW24]. Table 14 presents a side-by-side comparison. Every column matches except one: the offline carry resolution, which is the algebraic price of the lattice setting.

The isomorphism is not a coincidence. In FROST, the map $k \mapsto g^k$ is simultaneously homomorphic, hiding, and binding. Threshold signers combine nonce commitments by group multiplication, and the result is correct and private without further interaction.

In TALUS, the natural nonce commitment $\mathbf{w} = \mathbf{A}\mathbf{y}$ is R_q -linear (homomorphic) and binding (M-SIS), but not hiding: the left inverse of $\mathbf{A}$ recovers $\mathbf{y}$ exactly (Theorem 1). The HighBits rounding restores hiding (M-LWR) but destroys homomorphism: the quotient $\lfloor S/\alpha \rfloor$ introduces a carry that is not R_q -linear. The carry resolution is therefore the *unique structural divergence* between the two protocols.

The algebraic root cause is that $\mathbb{Z}_q$ is a simple group (no non-trivial subgroups when q is prime), so the only group homomorphism $\mathbb{Z}_q \rightarrow \mathbb{Z}_M$ with $M < q$ is the zero map (Theorem 1, Part 2). FROST avoids this because g^k maps into a group of order q (the same order as the secret domain), so no information-destroying quotient is needed.

10.2 Deployment Trade-offs

BCC overhead and nonce management. TALUS-TEE requires an average of ≈ 3.15 preprocessing attempts per accepted nonce (since $p_{\text{BCC}} \approx 31.7\%$), but this overhead is entirely

Table 14: Structural comparison of FROST (discrete-log) and TALUS (lattice). The sole structural difference is the offline carry resolution required by the non-homomorphic HighBits rounding.

	FROST (DL)	TALUS (Lattice)
Secret sharing	Shamir over $\mathbb{Z}_q$	Shamir over R_q
Commitment	$R_i = g^{k_i}$	$(h_i, \text{carry tag } e_i)$
Commitment recovery	$R = \prod R_i$ (group mult.)	$w_1 = \sum h_i + \bigoplus e_i$
Online broadcast	$z_i = k_i + c \cdot s_i$	$z_i = y_i + c \cdot s_{1,i}$
Online rounds	1	1
Fault tolerance	$N \geq 2T-1$	TEE: any $N \geq T$; MPC: $N \geq 2T-1$ ($T \geq 3$)*
Offline preprocessing	None needed	Carry resolution

*TALUS-TEE requires no honest-majority condition (TEE evaluates carry locally; any $N \geq T$ suffices).

TALUS-MPC with $T \geq 3$ requires $N \geq 2T-1$ for *security* of the CSCP (Theorem 27), not merely for availability;

$T = 2$ has no restriction (Section 7.3.1).

offline. In contrast, prior work handles the r0-check in every online session, adding ≥ 1 online round unconditionally. For latency-sensitive applications (TLS, code signing, real-time payment authorisation), the online round is the bottleneck; TALUS shifts this cost to the offline preprocessing phase, which can be amortised over many signing sessions and executed with relaxed latency constraints. In practice, a *nonce pool* of $P \geq 10$ pre-validated nonces eliminates any signing-path latency: at $T = 3$ (ML-DSA-65), generating ten accepted nonces costs ≈ 9.9 ms (TEE) or 63.9 ms (MPC), well within any background-task budget. Because preprocessing is independent of the online path, the latency experienced by the signer is always that of a single round-trip, regardless of pool-replenishment activity.

Identifiable abort and malicious security. TALUS-TEE is proven secure against semi-honest or malicious signers (the coordinator’s TEE enforces correct preprocessing; the online Blame protocol (Section 6.4) identifies deviating signers via the public $\mathbf{As}_{1,i}$ commitments). TALUS-MPC achieves *identifiable abort* via an optimistic two-tier mechanism (Section 7.11): (i) online failures are immediately attributed using the Feldman VSS commitments $\{\Phi_{h,k}\}$ broadcast at preprocessing (no extra round); (ii) preprocessing failures are attributed via a reveal-on-failure subprotocol that replays the masked broadcasts and CSCP circuit from revealed *session keys* $K_{h,j,\tau} = \text{PRF}(s_{h,j}, \text{“session”} \parallel \tau)$; this is safe by Lemma 8: the long-term pairwise seeds $s_{h,j}$ are never disclosed, so future sessions remain protected by PRF pseudorandomness of $K_{h,j,\tau'}$ for $\tau' \neq \tau$, and a failed nonce carries no information about $\mathbf{s}_1$. Full *malicious privacy* (preventing a malicious party from probing the distribution of $\sum_i \rho_i$ via adversarially chosen Beaver triple deviations) additionally requires authenticated Beaver triples [DPSZ12]; this extension does not affect the core BCC argument or the IA guarantee and is deferred to future work.

Honest-majority requirement. TALUS-TEE places no honest-majority constraint: the TEE coordinator evaluates the carry locally without interaction, so any $N \geq T$ suffices. TALUS-MPC with $T \geq 3$ requires $N \geq 2T-1$ (honest majority) for *security* of the CSCP carry computation (not merely for fault tolerance), as formalised below. This is the same trust model adopted by FROST [KG21, CKGW24] and mandated by NISTIR 8214 [BMV19] for Shamir-based multiparty protocols. For $T = 2$, the requirement does not apply (Section 7.3.1).

Theorem 27 (Dishonest Majority Privacy Barrier, informal). *Under $T-1$ corruptions with $N < 2T-1$ and $T > 2$, any single-round broadcast protocol that correctly computes the carry δ reveals at least $\lceil \log_2 \alpha \rceil = 19$ bits of LowBits information per coefficient to a rushing adversary, totalling $\geq 29,184$ bits per signing session, versus the $\approx 2,435$ bits of entropy contained in the carry values themselves.*

Proof sketch. Since $N < 2T - 1$, a valid signing set of size $|S| = T$ can contain $T-1$ corrupted parties and a single honest party h^* (any party may propose a legal T -party signing set). Party h^* has no honest peers, hence no honest-honest pairwise seeds; every mask term in its broadcast is known to the adversary. Therefore the adversary observes $m = f(\mathbf{x})$ where $\mathbf{x} = \text{LowBits}(\mathbf{A}\hat{\mathbf{y}}_{h^*}) \in (-\gamma_2, \gamma_2)^{kn}$ and f is the (deterministic, from the adversary’s view) encoding function.

We show f must be injective on each coefficient. Fix coefficient j and suppose f maps two distinct inputs $x_j \neq x'_j$ to the same output. The rushing adversary, after observing m , sets the corrupted aggregate to $X_C = k\alpha - x_j$ (for an integer k placing X_C in the achievable range). Then $\lfloor (X_C + x_j)/\alpha \rfloor = k$ while $\lfloor (X_C + x'_j)/\alpha \rfloor \neq k$ (since $0 < |x'_j - x_j| < \alpha$). The coordinator computes the wrong carry, contradicting correctness. Hence f is injective: it encodes all $\alpha = 523,776$ possible values per coefficient, leaking $\lceil \log_2 \alpha \rceil = 19$ bits each, totalling $19 \times 1536 = 29,184$ bits. The carry values $\delta_j \in \{-1, 0, +1\}$ contain only $1536 \cdot \log_2 3 \approx 2,435$ bits of entropy. $\square$

Corollary 7. *Combined with the public value $\mathbf{w}_1 = \text{HighBits}(\mathbf{A}\hat{\mathbf{y}})$, the leaked LowBits recovers $\mathbf{A}\hat{\mathbf{y}} \bmod q$ in full, yielding a noise-free Module-LWE sample $\mathbf{A}\mathbf{s}_1 = c^{-1}(\mathbf{A}\mathbf{z} - \mathbf{A}\hat{\mathbf{y}})$ (where c is invertible in R_q with overwhelming probability). This provides strictly more information about the signing key than the public key alone.*

The barrier applies specifically to the combination of (i) dishonest majority, (ii) $T > 2$, and (iii) single-round Shamir-based protocols. For $T = 2$, the barrier does not apply because a single honest party always holds a full Shamir share, and the carry degenerates to a DCF [BGI15, BGI16] with computational security (Section 7.3.1).

Deployment implication. This result justifies the honest-majority requirement as a fundamental constraint, not a design limitation. Deployments requiring dishonest-majority tolerance must use general MPC (e.g., Quorus [BdCE⁺26], Trilithium [DKLS25]) at significantly higher communication cost.

10.3 Implementation Considerations

FIPS 204 compatibility across parameter sets. TALUS produces signatures with hint $\mathbf{h}$ of expected weight ≈ 38 , computable from the public values $\mathbf{r} = \mathbf{A}\mathbf{z} - \mathbf{c}\mathbf{t}_1 \cdot 2^d$ and $\mathbf{w}_1$. The BCC condition ensures $\|\mathbf{h}\|_1 \leq \omega = 55$, so FIPS 204 Algorithm 26 encodes the hint correctly without any modification to the standard verifier. TALUS applies to all three ML-DSA parameter sets; BCC pass rates and concrete security bounds for ML-DSA-44 and ML-DSA-87 are tabulated in Appendix E.

A unified bounded-reconstruction framework. Two apparently unrelated problems in TALUS share an identical mathematical structure: (1) the BCC offline check (verify that $\text{LowBits}(\mathbf{w}) \in (-\gamma_2 + \beta, \gamma_2 - \beta)^{k \times n}$ without revealing $\mathbf{w}$), and (2) the distributed keygen (verify that $\mathbf{s}_1 = \sum_i \mathbf{v}_i$ has small coefficients without revealing $\mathbf{s}_1$). Both are instances of computing $f(\sum_i \lambda_i x_i)$ where f is a non-linear function (LowBits / modular reduction) that cannot be computed by linearly combining the parties’ local evaluations of f . The solution in both cases is arithmetic-to-binary conversion (A2B / edaBits): represent the sum in a bit-decomposed form, then evaluate f coefficient-wise. The only difference is the modulus: $2\gamma_2 = 523,776$ (19-bit) for BCC, and $2\eta + 1 = 9$ (4-bit) for DKG. We term this the *Bounded Reconstruction Framework* (BRF). We expect BRF to be useful in other lattice threshold constructions wherever a non-linear smallness predicate must be evaluated on a secret Shamir-combined value; a formal treatment is an interesting direction for future work.

Deployment profile: TEE vs. MPC. TALUS-TEE and TALUS-MPC target different deployment environments. TALUS-TEE is suited to settings where a single coordinator can be equipped with a hardware TEE (Intel SGX, AMD SEV, or ARM TrustZone): cloud-hosted key management services, on-premises HSM clusters, and certificate-authority signing pipelines where the coordinator is already a trusted principal. The trust model is that of a *hardened*

co-signer: the coordinator learns nothing about the signing key beyond what the TEE attestation guarantees. TALUS-MPC is appropriate when no single coordinator should be trusted, e.g. inter-organisational custody, decentralised autonomous organisations, and environments where the coordinator is an untrusted aggregator in a peer-to-peer network. The primary cost of MPC is the offline preprocessing: at $T = 3$ (ML-DSA-65), offline cost is 1.08 ms/attempt vs. 0.39 ms/attempt for TEE, a $2.8\times$ overhead. Online costs differ by $\approx 2.3\times$ (0.29 vs. 0.68 ms). Both variants produce identical FIPS 204-valid signatures.

One online round: application-layer implications. FROST [KG21] and Lindell’s threshold ECDSA [Lin21] require two online rounds, imposing at least two network round-trip times (RTTs) of latency per signature. For a 10 ms LAN or 60 ms WAN RTT, two-round protocols incur ≥ 20 ms or ≥ 120 ms of irreducible latency regardless of cryptographic computation cost. TALUS-TEE requires one online round: coordinator-to-parties broadcast of the challenge c , followed by parties returning $\mathbf{z}_i$, which can be pipelined as a single request-response exchange. This matches the latency profile of a standard single-party signing call, making TALUS suitable for latency-sensitive paths: TLS handshake signing, DNSSEC signing, code-signing pipelines, and real-time payment authorisation where multi-round protocols introduce unacceptable delays.

Human-in-the-loop signing. Threshold signing is often deployed with human signers who review each message before authorising their share (e.g., multi-party custody of cryptographic keys). FROST [KG21] requires two online rounds: a signer must be present for both the commitment round and the signing round, or must delegate the commitment round to an automated device. TALUS-TEE separates the protocol into an *automated* offline phase (nonce generation, BCC filtering) that requires no human interaction, and a *single* online round in which the signer sees the message, decides whether to approve, and returns the response share $\mathbf{z}_i$. Because the TEE coordinator pre-filters BCC offline, the online phase is deterministic: once the signer approves, the signature is guaranteed to complete without retries. This is in contrast to Mithril [CdPE⁺26], where Fiat–Shamir abort-and-retry may require the signer to participate in multiple attempts. The only constraint is that the signing set S must be determined at preprocessing time; if a signer declines to participate, the preprocessed nonce is discarded (with no security consequence).

TEE nonce confidentiality and key-recovery risk. In TALUS-TEE, the coordinator’s TEE holds the aggregate nonce $\mathbf{y}$ for each preprocessing session. The confidentiality of $\mathbf{y}$ is therefore a component of the overall security guarantee. Specifically, if an adversary recovers $\mathbf{y}$ for a completed signing session (for instance, through a TEE side-channel attack [BMD⁺17]), then from the public transcript (σ, pk) they can compute $c\mathbf{s}_1 = \mathbf{z} - \mathbf{y} \pmod{q}$. From a *single* session alone, recovery of $\mathbf{s}_1$ is immediate: the challenge c is a polynomial with $\tau = 49$ nonzero coefficients that is invertible in $R_q = \mathbb{Z}_q[X]/(X^\tau + 1)$ with overwhelming probability [Nat24b], so $\mathbf{s}_1 = c^{-1}(\mathbf{z} - \mathbf{y}) \pmod{q}$ directly (all $\ell = 5$ components in one step). This is not a flaw in the TALUS protocol: it is the standard consequence of the TEE trust assumption: a compromised TEE is equivalent to a compromised signing device in single-party ML-DSA. The threat is inherent to any threshold scheme that concentrates the nonce in a single trusted component. TALUS-MPC avoids nonce concentration entirely: no single party holds the full $\mathbf{y}$, so nonce recovery from a single compromise is infeasible.

Signature indistinguishability. TALUS produces valid FIPS 204 signatures, but their distribution is not identical to that of single-party ML-DSA. Two distinguishing features exist. First, the aggregate nonce $\hat{\mathbf{y}} = \sum_h \hat{\mathbf{y}}_h$ follows an Irwin-Hall distribution $\text{IH}(T)$, which is more concentrated around zero than the uniform distribution of single-party ML-DSA; this causes the accepted $\mathbf{z}$ vectors to exhibit a slightly sub-uniform distribution over $(-\gamma_1 + \beta, \gamma_1 - \beta)^{n_\ell}$. Second, the BCC filter selects nonces whose $\mathbf{r}_0$ coefficients avoid the interval $[-\beta, \beta]$ near the modular boundaries, imposing a mild bias in the hint weight distribution relative to standard ML-DSA (empirically, $\Pr[\text{wt}(\mathbf{h}) = 0] \approx 0$ under TALUS, whereas single-party ML-DSA allows $\mathbf{h} = \mathbf{0}$). An adversary with 10^4 valid TALUS signatures could in principle detect these biases via a χ^2

test. This does *not* affect EUF-CMA security: the ability to distinguish TALUS signatures from single-party ML-DSA signatures does not yield any advantage in forging signatures. Deployments requiring *protocol anonymity* (hiding the use of threshold signing) would need an additional rejection-sampling step that matches the single-party $\mathbf{z}$ distribution exactly; such an extension is technically feasible but costly, and beyond the scope of this work.

Key refresh and proactive security. TALUS-TEE.Refresh (Section 6.5) and TALUS-MPC.Refresh (Appendix C) both achieve full proactive security in the mobile adversary model of [HJKY95]: shares and pairwise seeds are refreshed so that corrupting a fresh set of $T-1$ parties in a new epoch reveals nothing about prior epochs. The refresh executes in a single broadcast round under the standard HJKY95 recovery-window assumption (no party is under adversarial control during the refresh), with erasures at two levels (after each signing session and after each refresh). For TALUS-TEE, the same protocol omits the seed step since no pairwise seeds exist. Combined with the erasure model, the base protocol additionally achieves adaptive EUF-CMA within each epoch (Remark 38 in Appendix C).

10.4 Future Directions

Seven concrete directions remain.

1. Multi-party distributed comparison functions. Standard DCFs [BGI15, BGI16] are inherently two-party: the function secret sharing splits a comparison gate into exactly two additive keys. For $T = 2$, TALUS exploits this directly, achieving ~ 5 KB offline communication per batch. For $T \geq 3$, the protocol falls back to a Beaver-based prefix-tree comparator at ~ 350 KB per batch. A multi-party DCF would eliminate the comparison circuit entirely, making $T \geq 3$ communication equal to $T = 2$ (Layer 4 in Table 13).

2. Packed $\mathbb{F}(2^{128})$ Shamir for comparison. Packing 128 independent bit-level comparisons into a single $\mathbb{F}(2^{128})$ Shamir sharing would reduce CarryCompare communication from ~ 350 KB to ~ 14.6 KB per batch (a $24\times$ improvement; Layer 3 in Table 13). Three components are required: $\mathbb{F}(2^{128})$ preprocessing from pairwise seeds (extending DN07 [DN07] to binary extension fields), share conversion at the R_q -to- $\mathbb{F}(2^{128})$ boundary, and a security proof for cross-field composition.

3. Malicious-secure TALUS-MPC. Adding SPDZ-style MACs [DPSZ12] to the carry elimination protocol for full malicious privacy. This requires authenticated Beaver triples to prevent a malicious party from probing the distribution of $\sum_i \rho_i$ via adversarially chosen deviations.

4. Full adaptive security without erasures. Appendix C (Remark 38) establishes adaptive EUF-CMA *with* the standard erasure assumption: honest parties erase $\mathbf{y}_i$ immediately upon sending $\mathbf{z}_i$, which prevents post-corruption reconstruction of the nonce. Full adaptive security *without* any erasure assumption remains open. The obstacle is that equivocal commitments are required in the ROM simulation, and the HighBits rounding function interacts non-trivially with equivocation strategies for the commitment $\mathbf{w}_1$.

5. BCC for ML-KEM decryption. Does an analogous boundary clearance condition hold for ML-KEM [Nat24a] decryption? Preliminary analysis suggests yes, with different probability, extending the BCC technique to the broader FIPS PQC suite.

6. Optimal nonce scheduling. Given a pool of preprocessed nonces, what is the optimal strategy for selecting which to use in each signing session to maximise throughput? Pipeline management with background preprocessing is straightforward in practice, but the theoretical optimum is open.

7. CEF for large N without CSCP overhead. The CSCP round count grows as $O(\log N)$. For $N \geq 32$, amortising or batching the CSCP across multiple preprocessing sessions could reduce per-signature cost.

11 Conclusion

We presented TALUS, a threshold ML-DSA construction achieving one-round online signing. The *Lattice Threshold Trilemma* (Theorem 1) establishes that the carry cost inherent to FIPS-compatible lattice-based threshold signing is a mathematical necessity, not a protocol artifact. Building on this foundation, the *Boundary Clearance Condition* (BCC) identifies when the secret-dependent term cs_2 cannot cross any rounding boundary (probability $\approx 31.7\%$), eliminating all MPC on s_2 ; and the *Carry Elimination Framework* enables distributed computation of the commitment hash input $\mathbf{w}_1$ using the identity $\alpha^{-1} \equiv -16 \pmod{q}$.

TALUS-TEE achieves the minimum possible online round count (one) for threshold ML-DSA while producing FIPS 204-valid signatures. Security reduces to single-party ML-DSA EUF-CMA with concrete security of ≥ 85 bits for ML-DSA-65 at NIST Level 3 (Section 8.1). This reduction loss relative to the 128-bit single-party baseline is inherent to all Fiat–Shamir threshold schemes and does not represent a fundamental weakness: the game-based reduction via random oracle reprogramming incurs an unavoidable $Q_H^2/|C|$ collision term.¹ The $T = 2$ degeneration (Section 7.3.1) offers particularly low overhead for the common 2-of-2 custody scenario: ~ 5 KB offline communication per batch, $74\times$ less than $T \geq 3$.

TALUS is ready for deployment in both TEE and MPC configurations. See Section 10.4 for a detailed discussion of open problems, including multi-party DCF construction, packed $\mathbb{F}(2^{128})$ Shamir, and full adaptive security without erasures.

A Supplementary Game-Hopping Proofs

A.1 Proof of Theorem 23 (TALUS-TEE EUF-CMA)

The full proof of Theorem 23 appears in Section 8.1. We restate the game sequence and provide the missing details here.

Challenge space bound. The challenge space is $\mathcal{C} = \{c \in \{0, \pm 1\}^n : \|c\|_0 = \tau\}$, with $|\mathcal{C}| = \binom{n}{\tau} \cdot 2^\tau$. For ML-DSA-65: $|\mathcal{C}| = \binom{256}{49} \cdot 2^{49} \approx 2^{225}$. Thus $Q_H^2/|\mathcal{C}|$ is negligible for any polynomial Q_H .

Nonce DKG privacy (Game 0 $\rightarrow$ 1). Let $\mathcal{A}$ control corrupted parties C with $|C| \leq T - 1$. The honest parties' DKG contributions to party $i \in C$ are evaluations $\{f_h(i) : h \notin C\}$. Since f_h has degree $T - 1$ and the adversary holds at most $T - 1$ evaluations of f_h (from C), the Shamir information-theoretic guarantee applies: the distribution of $f_h(i)$ for $i \in C$ is exactly uniform over $\mathbb{Z}_q^{n_\ell}$, jointly independent across distinct h . The simulator draws these uniformly without loss.

Hint computability argument (Game 3). When BCC holds (guaranteed by preprocessing), Theorem 2 gives $\text{HighBits}(\mathbf{w} - cs_2) = \mathbf{w}_1$. We now show that $\mathbf{h}$ is computable from public data and $\text{wt}(\mathbf{h}) \leq \omega$ with overwhelming probability.

Lemma 9 (Hint Computability Under BCC). *Suppose $\text{BCC}(\mathbf{w})$ holds. Then for any challenge c and key material (s_2, t_0) with $\|s_2\|_\infty \leq \eta$ and $\|t_0\|_\infty \leq 2^{d-1}$:*

1. Define $r_j = (\mathbf{A}\mathbf{z} - ct_1 \cdot 2^d)_j$ (public) and $h_j = \mathbf{1}[\text{HighBits}(r_j, 2\gamma_2) \neq (w_1)_j]$. Then $\text{UseHint}(h_j, r_j, 2\gamma_2) = (w_1)_j$ (so FIPS 204 verification accepts the signature). Note: r_j analytically equals $(w - cs_2 + ct_0)_j$, so h_j 's value implicitly depends on s_2 , but r_j is computable from public $(\mathbf{pk}, \mathbf{z}, c)$ without explicit knowledge of s_2 .

¹For $Q_H = 2^{32}$ and $|\mathcal{C}| = \binom{256}{49} \cdot 2^{49}$, this term is negligible.

2. $\text{wt}(\mathbf{h}) \leq \omega = 55$ with probability $\geq 99.7\%$ under BCC; per-coefficient carry probability $\approx 2.5\%$, expected weight ≈ 38 , so $W = \text{wt}(\mathbf{h}) \sim \text{Bin}(1536, 0.025)$ and $\Pr[W > 55] < 0.003$ by Chernoff bound (empirically confirmed; see `experiments/talus_tee.py`).

Proof. Part 1. By Theorem 2(ii), $\text{HighBits}(\mathbf{w} - \mathbf{cs}_2) = \mathbf{w}_1$. The value $r_j = (w - cs_2 + ct_0)_j$ differs from $(w - cs_2)_j$ by $(ct_0)_j$. Since $|(ct_0)_j| \leq \tau \cdot 2^{d-1} = 200,704 < \gamma_2$, at most one rounding boundary is crossed.

- If no crossing: $\text{HighBits}(r_j) = (w_1)_j$, so $h_j = 0$ and $\text{UseHint}(0, r_j) = \text{HighBits}(r_j) = (w_1)_j$.
- If one crossing: $|\text{HighBits}(r_j) - (w_1)_j| = 1$, so $h_j = 1$ and $\text{UseHint}(1, r_j) = \text{HighBits}(r_j) \pm 1 = (w_1)_j$ (the $\pm$ direction is determined by $\text{sgn}(\text{LowBits}(r_j))$ and always corrects to $(w_1)_j$).

The computation of $r_j = (\mathbf{Az} - \mathbf{ct}_1 \cdot 2^d)_j$ uses only public inputs $(\mathbf{pk}, \mathbf{z}, c)$, so h_j requires no explicit knowledge of $\mathbf{s}_2$. $\square$

For part 2: a carry occurs for coefficient j iff adding $(ct_0)_j$ to $(w - cs_2)_j$ crosses a rounding boundary at $\pm\gamma_2$. Since $|(ct_0)_j| \leq \tau \cdot 2^{d-1} = 200,704$, a carry is only possible when $|r_j - (ct_0)_j| > \gamma_2 - |(ct_0)_j|$, i.e., when r_j lies within distance 200,704 of a boundary $\pm\gamma_2$. The “carry zone” has total width $2 \times 200,704 = 401,408$, while the BCC region has total width $2(\gamma_2 - \beta) = 523,384$. This gives a worst-case per-coefficient carry probability $\leq 401,408/523,384 \approx 76.7\%$, a very loose bound that does not exploit the interior concentration under BCC. Empirically (see `experiments/talus_tee.py`), carry probability under BCC is only $\approx 2.5\%$ because BCC conditions on r_j being interior, i.e., $|r_j| < \gamma_2 - \beta = 261,692$, and typical $(ct_0)_j$ magnitudes are far below the 200,704 maximum. Empirical expected weight $\approx 38 \ll \omega = 55$.

Confirmed empirically: per-coefficient carry probability under BCC $\approx 38.2/1536 \approx 2.5\%$, giving $\mathbb{E}[\text{wt}(\mathbf{h}) \mid \text{BCC}] \approx 38.2$ and $\Pr[\text{wt}(\mathbf{h}) > 55 \mid \text{BCC}] < 10^{-3}$ across $|S| \in \{2, 3, 5\}$ (see `experiments/talus_tee.py`).

To convert the empirically confirmed per-coefficient rate $p \leq 0.025$ into a formal tail bound, let $W = \text{wt}(\mathbf{h})$ be the sum of $m = k \cdot n = 1536$ independent Bernoulli(p) indicators (one per polynomial coefficient across all k output elements). By the Chernoff bound with $\mu = mp = 38.4$ and $\delta = (55 - \mu)/\mu \approx 0.432$:

$$\Pr[W > 55 \mid \text{BCC}] \leq \exp(-\mu\delta^2/3) = \exp(-38.4 \times 0.432^2/3) \approx e^{-2.38} \approx 0.093.$$

This bound is loose because it uses $p \leq 0.025$ analytically and $\delta^2/3$ rather than the tighter $(1 + \delta) \ln(1 + \delta) - \delta$. Using the exact KL-divergence form $D_{\text{KL}}(55/1536 \parallel 0.025) \approx 0.0021$ gives $\Pr[W > 55] \leq \exp(-1536 \times 0.0021) \approx e^{-3.3} \approx 0.037$; the KL-Chernoff bound is loose here, and the tighter $< 10^{-3}$ bound is confirmed empirically (see `experiments/talus_tee.py`). $\square$

A.2 Proof of Theorem 24

Stated in Section 8.2; proof given there.

A.3 TALUS-MPC Security Proof

The main game-hopping proof of Theorem 25 appears in Section 8.3. We supply the three supporting lemmas used there: mask cancellation, masked broadcast uniformity, and the PRF-to-random Beaver triple reduction.

Lemma 10 (Pairwise Mask Cancellation). *In TALUS-MPC.Preprocess, define for each party $i \in S$:*

$$\text{mask}_i^H[j] = \sum_{k \in S, k \neq i} (-1)^{\mathbf{1}[k < i]} \cdot r_{ij}[j] \pmod{16},$$

where $r_{ij}[j] = r_{ji}[j] \in \{0, \dots, 15\}$ are the pairwise PRF-derived values (Protocol 13, step 3). Then:

$$\sum_{i \in S} \text{mask}_i^H[j] \equiv 0 \pmod{16} \quad \text{for all } j \in [nk].$$

Proof. Each pair (a, b) with $a < b$ contributes $+r_{ab}[j]$ to party a 's mask and $-r_{ab}[j]$ to party b 's mask. Summing over all pairs and all parties:

$$\sum_{i \in S} \text{mask}_i^H[j] = \sum_{\{a,b\} \subseteq S, a < b} (r_{ab}[j] - r_{ab}[j]) = 0 \pmod{16}. \quad \square$$

Lemma 11 (Masked Broadcast Uniformity). *Let $\mathcal{A}$ control at most $T - 1$ corrupted parties $C \subset S$. Then for any honest party $j \notin C$:*

1. $\tilde{H}_j[k]$ is uniformly distributed in $\mathbb{Z}_{16}$ from the adversary's view, conditioned on any fixed $\{(w_{1,i}, r_{0,i})\}_{i \in C}$.
2. $\tilde{b}_j[k]$ reveals $r_{0,j}[k]$ only to within an interval of length $\lfloor \alpha/|S| \rfloor$:

$$r_{0,j}[k] \in (\tilde{b}_j[k] - \lfloor \alpha/|S| \rfloor, \tilde{b}_j[k]).$$

Proof. *Part 1.* The pairwise seed seed_{jk} for (j, k) with $j \notin C$ and $k \in C$ is unknown to $\mathcal{A}$ (keys are honestly generated). The value $r_{jk}[m]$ is $\text{PRF}(\text{seed}_{jk}, m) \bmod 16$, which is computationally indistinguishable from uniform in $\mathbb{Z}_{16}$. Since $\tilde{H}_j[m] = (w_{1,j}[m] + \text{mask}_j^H[m]) \bmod 16$ and $\text{mask}_j^H[m]$ includes at least one unknown PRF output, $\tilde{H}_j[m]$ is computationally indistinguishable from uniform. (If all seeds involving j were known to $\mathcal{A}$, j would be corrupted, contradicting $j \notin C$.)

Part 2. $\tilde{b}_j[k] = r_{0,j}[k] + \rho_j[k]$ with $\rho_j[k] \in [0, \lfloor \alpha/|S| \rfloor]$. Given $\tilde{b}_j[k]$, the value $r_{0,j}[k]$ lies in the interval $(\tilde{b}_j[k] - \lfloor \alpha/|S| \rfloor, \tilde{b}_j[k])$, an interval of length $\lfloor \alpha/|S| \rfloor$. $\square$ $\square$

Lemma 12 (PRF Reduction for Beaver Triples). *Let F be a PRF. There exists a PPT reduction $\mathcal{B}_1$ such that:*

$$|\Pr[\mathcal{A} \text{ wins Game 1}] - \Pr[\mathcal{A} \text{ wins Game 2}]| \leq \text{Adv}_F^{\text{PRF}}(\mathcal{B}_1),$$

where Game 2 replaces PRF-derived Beaver triples with truly random $(\langle a_g \rangle, \langle b_g \rangle, \langle c_g \rangle)$.

Proof. $\mathcal{B}_1$ simulates TALUS-MPC honestly, but queries its PRF oracle (either F or a random function) to generate all Beaver triple seeds. Each seed for group g is $F(\text{seed}_{ij}, g)$ for a pairwise session key seed_{ij} . The corresponding arithmetic shares are derived via additive splitting: $a_{g,i} \leftarrow F(\text{seed}_{ij}, g, \mathbf{a}'')$ with $a_{g,j} = a_g - a_{g,i}$.

If the PRF oracle is the true F , $\mathcal{B}_1$'s simulation is identical to Game 1. If the oracle is a random function, the triples are uniformly random, matching Game 2. Any distinguishing advantage for $\mathcal{A}$ translates directly to a PRF advantage for $\mathcal{B}_1$. $\square$

Remark 32 (CSCP Two-Bit Leakage). The extended CSCP outputs (c, δ) , leaking at most 2 bits of information per session about $\Sigma\rho_i$: $c = [\Sigma\rho > t]$ partitions $[0, \alpha]$ into $([0, t]$ vs. (t, α)), and δ further partitions within the selected interval. Lemma 11(2) bounds the residual uncertainty in $r_{0,j}$ to an interval of length $\lfloor \alpha/|S| \rfloor$. For $|S| \geq 2$, this is $\leq \alpha/2 = \gamma_2 = 261,888$. Given (c, δ) and all $\tilde{b}_i$, the adversary localises $\Sigma\rho_i$ to a sub-interval of $[0, \alpha]$ of length at most $\max(t, \alpha - t) \leq \alpha$; but each individual $r_{0,j}$ is still hidden to within $\pm\gamma_2/|S|$ by the range-restricted mask. For $|S| = 3$, this is $\approx \pm 87,296$, far below the BCC bound $\gamma_2 - \beta = 261,692$.

B UC Security

B.1 UC Security of TALUS-TEE

We give a UC formalization of TALUS-TEE in the $(\mathcal{F}_{\text{TEE}}, \mathcal{F}_{\text{RO}})$ -hybrid model [Kat07], where $\mathcal{F}_{\text{TEE}}$ models the coordinator's trusted execution environment and $\mathcal{F}_{\text{RO}}$ is the random oracle for the challenge hash. We prove that TALUS-TEE UC-realizes the ideal signing functionality $\mathcal{F}_{\text{Sig}}$ against a static adversary corrupting at most $T - 1$ parties (who may behave maliciously; Theorem 31).

B.2 Ideal Functionalities

Functionality 28 ($\mathcal{F}_{\text{Sig}}$: Ideal Threshold Signature). *Parameters: threshold T , party count N , public key pk .*

1. **KeyGen**: On input (keygen) from all N parties, sample $(\text{pk}, \text{sk}) \leftarrow \text{ML-DSA.KeyGen}()$, distribute $\text{sk}_i = \text{s}_{1,i}$ (Shamir share of s_1) to party i , and broadcast pk .
2. **Sign**: On input (sign, m) from a set S with $|S| \geq T$: if fewer than T honest parties participate, output $\perp$. Otherwise, notify the simulator $\mathcal{S}$ and wait for $\mathcal{S}$ to provide a candidate signature $\sigma = (\tilde{c}, \mathbf{z}, \mathbf{h})$; verify $\text{ML-DSA.Verify}(\text{pk}, m, \sigma) = 1$ and output σ to all parties in S . (Allowing the simulator to provide σ is standard in threshold-signature UC proofs [KRT24]; the validity check ensures the simulator cannot produce invalid signatures.)
3. **Verify**: On input (verify, m, σ) from any party, output $\text{ML-DSA.Verify}(\text{pk}, m, \sigma)$.

Remark 33 (Simulator-Provided Signatures). Allowing $\mathcal{S}$ to provide σ in the Sign interface is the standard approach for threshold-signature UC proofs (see, e.g., [KRT24]). It reflects the fact that in the real protocol the coordinator (which $\mathcal{S}$ simulates) assembles the final σ . The mandatory Verify check prevents the simulator from outputting an invalid signature, maintaining the ideal-world guarantee that all outputs are valid FIPS 204 signatures.

Functionality 29 ($\mathcal{F}_{\text{TEE}}$: Trusted Execution Environment). *The TEE functionality executes coordinator code in isolation. It accepts an arbitrary program Π and input x , runs $y \leftarrow \Pi(x)$ in a sealed environment, and outputs $(y, \text{attest}(\Pi, x, y))$ where attest is an unforgeable attestation. Corrupted external parties cannot observe or modify x or the internal state of Π .*

B.3 The TALUS-TEE Simulator

We construct a simulator $\mathcal{S}$ such that no PPT environment $\mathcal{Z}$ can distinguish the real execution of TALUS-TEE from the ideal execution with $\mathcal{F}_{\text{Sig}}$ and $\mathcal{S}$.

Simulator 30 ($\mathcal{S}$ for TALUS-TEE). *Let $C \subset [N]$ with $|C| \leq T-1$ be the set of corrupted parties. $\mathcal{S}$ runs a copy of the adversary $\mathcal{A}$ internally and simulates the honest parties' views as follows.*

KeyGen simulation. $\mathcal{S}$ generates the real key pair $(\text{pk}, \text{sk}) \leftarrow \text{ML-DSA.KeyGen}()$ (including $\text{sk} = (\rho_A, \text{s}_1, \text{s}_2, \text{t}_0)$) and sends pk to $\mathcal{A}$ and $\mathcal{Z}$. $\mathcal{S}$ Shamir-shares s_1 with threshold T among N parties, giving share $\text{s}_{1,i} = f(i)$ to party i . For each honest party $i \notin C$, $\mathcal{S}$ sends $\text{s}_{1,i}$ as part of the simulated honest party interface. (Corrupted parties receive their real shares $\text{s}_{1,j}$ for $j \in C$.)

Preprocessing simulation. For each preprocessing session with signing set S , $\mathcal{S}$ runs the following loop until BCC succeeds:

1. For each honest party $h \notin C$, sample a fresh nonce polynomial $g_h(X)$ of degree $T-1$ with constant term $\hat{y}_h \xleftarrow{\$} [-\gamma_1/|S|+1, \gamma_1/|S|]^{n_\ell}$ and uniformly random higher-degree coefficients in $\mathbb{Z}_q^{n_\ell}$. Send evaluation $g_h(i)$ to each party i (including $i \in C$) exactly as in the real protocol. (Corrupted parties $h \in C$ send their own g_h contributions honestly in the semi-honest model; $\mathcal{S}$ passes these through.)
2. Assemble the aggregate nonce: $\hat{\mathbf{y}} = \sum_{h \in S} \hat{y}_h \bmod q$. Compute $\hat{\mathbf{w}} = \mathbf{A}\hat{\mathbf{y}} \bmod q$.
3. **BCC check**: if $\neg \text{BCC}(\hat{\mathbf{w}})$, discard this attempt and go to step 1 (restart the loop).
4. On success, compute $\mathbf{w}_1 = \text{HighBits}(\hat{\mathbf{w}}, 2\gamma_2)$ and store $(\hat{\mathbf{y}}, \mathbf{w}_1)$ for this session. (The simulator retains $\hat{\mathbf{y}}$ to compute $\mathbf{z}$ in the online simulation below; in the real protocol, the TEE stores only $\mathbf{w}_1$ and $\mathbf{A}\mathbf{y}_i$ for blame—see Protocol 6.) Simulate the TEE attestation via $\mathcal{F}_{\text{TEE}}$ running the coordinator's preprocessing code.

Note: the number of attempts until BCC passes is geometrically distributed with success probability $p_{\text{BCC}} \approx 0.317$, the same distribution as in the real protocol, since $\hat{\mathbf{y}}$'s distribution is identical.

Online signing simulation. On a signing query for message m using session nonce $(\hat{\mathbf{y}}, \mathbf{w}_1)$:

1. Compute the challenge seed: $\tilde{c} = H(\text{pk} \parallel m \parallel \text{wlen}(\mathbf{w}_1))$, expanding $c = \text{SampleInBall}(\tilde{c}, \tau)$. (This is a fresh RO query; $\mathcal{S}$ programs $\mathcal{F}_{\text{RO}}$ at this input if not yet set.)

2. Compute the aggregate response: $\mathbf{z} = \hat{\mathbf{y}} + c\mathbf{s}_1 \bmod q$. Check: if $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta$, discard $(\hat{\mathbf{y}}, \mathbf{w}_1)$ and re-run preprocessing (this happens with probability $< \varepsilon_z \approx 0$ given BCC; see Remark 15).
3. Compute hint from public data: $\mathbf{r} = \mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d \pmod{q}$, $h_j = \mathbf{1}[\text{HighBits}(r_j, 2\gamma_2) \neq (w_1)_j]$. Check: if $\text{wt}(\mathbf{h}) > \omega$, abort and re-run preprocessing.
4. Set $\sigma = (\tilde{c}, \mathbf{z}, \mathbf{h})$. Submit σ to $\mathcal{F}_{\text{Sig}}$'s Sign interface; $\mathcal{F}_{\text{Sig}}$ checks $\text{Verify}(\text{pk}, m, \sigma) = 1$ (guaranteed by Theorem 8) and outputs σ .
5. For each honest party $i \notin C$, compute the response share consistent with $\mathbf{z}$: assign $\mathbf{z}_1 = \mathbf{z} - \sum_{i>1, i \notin C} \lambda_i \mathbf{z}_i$ with remaining $\mathbf{z}_i$ drawn uniformly from $\mathbb{Z}_q^{n_\ell}$ subject to $\sum_{i \in S} \lambda_i \mathbf{z}_i = \mathbf{z}$. (Feasible by Shamir reconstruction: set one coordinate as the linear constraint and draw the rest uniformly.) Send $\mathbf{z}_i$ to the coordinator and to $\mathcal{A}$.

Remark 34 (Simulator Correctness). The simulator holds the real secret key sk (generated at KeyGen simulation), so it can compute a valid FIPS 204 signature $\sigma = (\tilde{c}, \mathbf{z}, \mathbf{h})$ directly. The $\mathcal{F}_{\text{Sig}}$ validity check passes by Theorem 8: steps 2–3 of the signing simulation explicitly verify that $\text{BCC}(\hat{\mathbf{w}})$ holds, $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$, and $\text{wt}(\mathbf{h}) \leq \omega$; given all three conditions the output is provably valid.

B.4 BCC Coupling Lemma

Lemma 13 (BCC Filtering Preserves Zero Statistical Distance). *Let $\mathcal{D}_{\text{real}}$ be the joint distribution of*

$$(\{g_h(i)\}_{h \notin C, i \in C}, \mathbf{w}_1)$$

in the real TALUS-TEE preprocessing protocol, and let $\mathcal{D}_{\text{sim}}$ be the corresponding distribution in the ideal world under Simulator 30. Then $\Delta(\mathcal{D}_{\text{real}}, \mathcal{D}_{\text{sim}}) = 0$.

Proof. Step 1: Identical marginal distribution of $\hat{\mathbf{y}}$. In both real and ideal worlds, for each honest party $h \notin C$:

- $\hat{y}_h \xleftarrow{\$} [-\gamma_1/|S| + 1, \gamma_1/|S|]^{n_\ell}$ (real: honest protocol; ideal: simulator step 1).
- Higher-degree coefficients $a_{h,j} \xleftarrow{\$} \mathbb{Z}_q^{n_\ell}$ (real: honest protocol; ideal: simulator step 1).

Corrupted parties $h \in C$ contribute $\hat{y}_h$ chosen by the (semi-honest) adversary, identical in both worlds since the adversary follows the protocol. Hence $\hat{\mathbf{y}} = \sum_{h \in S} \hat{y}_h$ has the same distribution in real and ideal.

Step 2: BCC is a deterministic function of $\hat{\mathbf{y}}$. The BCC predicate $\text{BCC}(\mathbf{A}\hat{\mathbf{y}})$ is a deterministic polynomial-time function of $\hat{\mathbf{y}}$ (it checks $\|\text{LowBits}(\mathbf{A}\hat{\mathbf{y}}, 2\gamma_2)\|_\infty \leq \gamma_2 - \beta$, see Definition 5). Since $\hat{\mathbf{y}}$ has the same distribution in real and ideal, so does any deterministic function of it, including BCC.

Step 3: Conditioning on BCC success preserves equality. Let E denote the event $\text{BCC}(\mathbf{A}\hat{\mathbf{y}}) = 1$. Since $\Pr_{\text{real}}[E] = \Pr_{\text{sim}}[E] = p_{\text{BCC}}$ (from Step 2), both worlds condition on the same event E with the same probability. For any measurable set A of joint outcomes:

$$\Pr_{\text{real}}[\cdot \in A \mid E] = \frac{\Pr_{\text{real}}[\cdot \in A, E]}{\Pr_{\text{real}}[E]} = \frac{\Pr_{\text{sim}}[\cdot \in A, E]}{\Pr_{\text{sim}}[E]} = \Pr_{\text{sim}}[\cdot \in A \mid E].$$

Hence the conditional distributions are equal, and $\Delta = 0$.

Step 4: Adversary-visible quantities. The adversary sees:

1. $\{g_h(i) : h \notin C, i \in C\}$: evaluations of honest parties' nonce polynomials at corrupted indices. In both worlds these are Shamir evaluations of polynomials with uniform coefficients and bounded constant terms. By Shamir perfect privacy ($|C| \leq T - 1$ evaluations of a degree- $(T - 1)$ polynomial), the conditional distribution of $g_h(i)$ given any $T - 1$ evaluations is uniform over $\mathbb{Z}_q^{n_\ell}$. Same in real and ideal: $\Delta = 0$.
2. Number of BCC retry attempts: geometrically distributed with success probability p_{BCC} , identical in real and ideal by Step 1.
3. $\mathbf{w}_1 = \text{HighBits}(\hat{\mathbf{w}}, 2\gamma_2)$: deterministic function of $\hat{\mathbf{y}}$ conditional on BCC passing, identical by Steps 2–3.

All adversary-visible quantities have identical joint distributions. $\Delta(\mathcal{D}_{\text{real}}, \mathcal{D}_{\text{sim}}) = 0$. $\square$

B.5 Indistinguishability Argument

Theorem 31 (TALUS-TEE UC Realization). *TALUS-TEE UC-realizes $\mathcal{F}_{\text{Sig}}$ in the $(\mathcal{F}_{\text{TEE}}, \mathcal{F}_{\text{RO}})$ -hybrid model against a static adversary corrupting at most $T - 1$ parties, where corrupted parties may behave maliciously.*

Proof. We show that for every PPT environment $\mathcal{Z}$, the adversary’s view in the real execution is statistically indistinguishable from its view in the ideal execution with $\mathcal{S}$.

Reduction to semi-honest. Any malicious deviation by a corrupted party $j \in C$ is handled by the TEE coordinator before it can affect the honest parties’ views: incorrect Shamir evaluations are caught by Feldman VSS consistency checks (Lemma 14), and incorrect online responses $\mathbf{z}_j$ are caught by the per-party linear check $\mathbf{A}\mathbf{z}_j = \mathbf{A}\hat{\mathbf{y}}_j + c \cdot \mathbf{V}_j$. A detected party is excluded (identifiable abort); an undetected deviation means the party sent the prescribed value, i.e. behaved semi-honestly. Hence, without loss of generality, we treat corrupted parties as semi-honest in the remainder of the proof (matching Theorem 23, proof of Game 0).

We track each phase.

KeyGen. The real key shares $\mathbf{sk}_i$ for corrupted parties are Shamir evaluations of a degree- $(T - 1)$ polynomial. Any $T - 1$ evaluations are jointly uniformly distributed (Shamir perfect privacy [Sha79]). The simulator draws them from the actual Shamir sharing of the real $\mathbf{s}_1$. *Statistical distance* = 0 (distributions are identical).

Preprocessing. By Lemma 13, the joint distribution of all adversary-visible preprocessing quantities ($\{g_h(i)\}_{h \notin C, i \in C}$, $\mathbf{w}_1$, retry count) is identical in real and ideal. *Statistical distance* = 0.

Online signing. We analyze each component of the adversary’s view.

(a) *Challenge $\tilde{c}$.* In both real and ideal, $\tilde{c} = H(\mathbf{pk} \| m \| \mathbf{w}_1 \text{enc}(\mathbf{w}_1))$: the same $(\mathbf{pk}, m, \mathbf{w}_1)$ are used (same $\mathbf{w}_1$ from preprocessing simulation), and $\mathcal{F}_{\text{RO}}$ is programmed consistently. The distributions are identical; $\Delta = 0$.

(b) *Aggregate response $\mathbf{z}$.*

- *Real:* $\mathbf{z} = \hat{\mathbf{y}} + c\mathbf{s}_1$ where $\hat{\mathbf{y}}$ is the BCC-filtered aggregate nonce and $c = \text{SampleInBall}(\tilde{c}, \tau)$.
- *Ideal:* $\mathbf{z} = \hat{\mathbf{y}} + c\mathbf{s}_1$ using the simulator’s copy of $\hat{\mathbf{y}}$ and $\mathbf{s}_1$ (Simulator step 2). Same formula, same $\hat{\mathbf{y}}$, same $\mathbf{s}_1$, same c : $\Delta = 0$.

(c) *Honest response shares $\mathbf{z}_i$ for $i \notin C$.*

- *Real:* $\mathbf{z}_i = \hat{\mathbf{y}}_i + c\mathbf{s}_{1,i}$. Since $\hat{\mathbf{y}}_i$ and $\mathbf{s}_{1,i}$ are each Shamir shares (degree- $(T - 1)$ polynomials) with uniform higher-degree coefficients, $\mathbf{z}_i$ is a Shamir share of $\mathbf{z} = \hat{\mathbf{y}} + c\mathbf{s}_1$. By Shamir perfect privacy, any $T - 1$ evaluations are uniformly distributed subject to the reconstruction constraint $\sum_{i \in S} \lambda_i \mathbf{z}_i = \mathbf{z}$.
- *Ideal:* Simulator draws $\mathbf{z}_i$ uniformly subject to $\sum \lambda_i \mathbf{z}_i = \mathbf{z}$ (same constraint, same $\mathbf{z}$). This is precisely the conditional uniform distribution. $\Delta = 0$.

(d) *Hint $\mathbf{h}$.* Computed identically from public data $(\mathbf{pk}, \mathbf{z}, \tilde{c})$ in real and ideal (both use $\mathbf{r} = \mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d$). $\Delta = 0$.

Combining all phases, the total statistical distance between the real and ideal executions is

$$\Delta(\text{REAL}, \text{IDEAL}) \leq \underbrace{0}_{\text{KeyGen}} + \underbrace{0}_{\text{Preproc.}} + \underbrace{0}_{\text{Signing}} = 0.$$

This establishes UC security with statistical distance 0, giving perfect (not just computational) indistinguishability. $\square$

B.6 Feldman-VSS Commitment for Blame

To enable blame attribution (Section 6.4), each party i publishes a hash commitment to its key share at setup:

$$\text{com}_i = H(\mathbf{s}_{1,i} \| \text{nonce}_i), \quad \text{nonce}_i \leftarrow \{0, 1\}^{256}.$$

The coordinator also stores the *linear* commitment $\mathbf{V}_i = \mathbf{A}\mathbf{s}_{1,i} \bmod q$ (computable from the public key and the share, published at setup).

Lemma 14 (Blame Correctness). *If party i sends an invalid response $\mathbf{z}_i$ (i.e. $\mathbf{A}\mathbf{z}_i \neq \mathbf{A}\mathbf{y}_i + c \cdot \mathbf{V}_i$ modulo q), the coordinator identifies i as faulty. If $\mathbf{z}_i$ is correctly formed, the check passes.*

Proof. The coordinator checks $\mathbf{A}\mathbf{z}_i = \mathbf{A}\hat{\mathbf{y}}_i + c \cdot \mathbf{V}_i$ using the stored $\mathbf{V}_i = \mathbf{A}\mathbf{s}_{1,i}$ and the TEE-attested $\mathbf{A}\hat{\mathbf{y}}_i$. If $\mathbf{z}_i = \hat{\mathbf{y}}_i + c\mathbf{s}_{1,i}$, then $\mathbf{A}\mathbf{z}_i = \mathbf{A}\hat{\mathbf{y}}_i + c\mathbf{A}\mathbf{s}_{1,i}$, so the check passes. Any deviation in $\mathbf{z}_i$ is detectable with probability 1 (the check is deterministic over $\mathbb{Z}_q^{nk}$). $\square$

Remark 35 (Binding of $\mathbf{V}_i$). The linear commitment $\mathbf{V}_i = \mathbf{A}\mathbf{s}_{1,i}$ is computationally binding under the Module-SIS assumption: any PPT adversary that equivocates (finds $\mathbf{s}_{1,i} \neq \mathbf{s}'_{1,i}$ with $\mathbf{A}\mathbf{s}_{1,i} = \mathbf{A}\mathbf{s}'_{1,i}$) solves M-SIS with the same advantage. A collision-resistant hash com_i binds the share value, linking the linear commitment to the Shamir share.

B.7 UC Security of TALUS-MPC

We extend the UC framework to TALUS-MPC. The ideal signing functionality $\mathcal{F}_{\text{Sig}}$ (Functionality 28) is unchanged. $\mathcal{F}_{\text{TEE}}$ is replaced by a pairwise randomness functionality that models the Phase 0 seed establishment.

Functionality 32 ($\mathcal{F}_{\text{PRF}}$: Pairwise Seeds). *On initialization, for each unordered pair $\{i, j\} \subseteq [N]$, sample $s_{ij} \xleftarrow{\$} \{0, 1\}^\lambda$ uniformly and deliver s_{ij} privately to party i and party j only. No other party learns any information about s_{ij} .*

Theorem 33 (TALUS-MPC UC Realization). *TALUS-MPC UC-realizes $\mathcal{F}_{\text{Sig}}$ in the $(\mathcal{F}_{\text{PRF}}, \mathcal{F}_{\text{RO}})$ -hybrid model against a static semi-honest adversary corrupting at most $T - 1$ parties, with $\Delta(\text{REAL}, \text{IDEAL}) = 0$. Instantiating $\mathcal{F}_{\text{PRF}}$ with a PRF yields computational UC security:*

$$\Delta(\text{REAL}, \text{IDEAL}) \leq Q_{\text{sess}} \cdot \text{Adv}_{\text{F}}^{\text{PRF}}(\mathcal{B}),$$

where Q_{sess} is the total number of preprocessing sessions and $\mathcal{B}$ is a PPT reduction.

Simulator 34 ($\mathcal{S}^{\text{MPC}}$ for TALUS-MPC). *Let $C \subset [N]$ with $|C| \leq T - 1$ be the corrupted parties. $\mathcal{S}^{\text{MPC}}$ runs $\mathcal{A}$ internally.*

KeyGen simulation. *Identical to Simulator 30: $\mathcal{S}^{\text{MPC}}$ generates (pk, sk) , Shamir-shares $\mathbf{s}_1$ with threshold T , and distributes real shares $\mathbf{s}_{1,j}$ to each $j \in C$. For each honest party h , it publishes Feldman commitments $\Phi_{h,k} = \mathbf{A} \cdot a_{h,k}$ for each coefficient $a_{h,k}$ of h 's nonce polynomial.*

Preprocessing simulation (per session τ). *$\mathcal{S}^{\text{MPC}}$ receives from $\mathcal{F}_{\text{PRF}}$ the seed s_{hj} for every pair (h, j) with $h \notin C$, exactly as honest parties do in the real protocol. It then proceeds as follows.*

1. **Nonce DKG.** *For each honest $h \notin C$, sample a nonce polynomial $g_h(X)$ of degree $T - 1$ with constant term $\hat{\mathbf{y}}_h \xleftarrow{\$} [-\gamma_1/|S|+1, \gamma_1/|S|]^{n_\ell}$ and uniform higher-degree coefficients over R_q . Send $g_h(j)$ to each $j \in C$; publish Feldman commitments for g_h 's coefficients.*
2. **Masked broadcast.** *Derive mask_h^H , ρ_h , and all CSCP Beaver-triple shares from $\{s_{hj}\}_{j \in S}$ using the identical formulas as the real protocol (Protocol 13). Broadcast $\tilde{H}_h$ and $\tilde{b}_h$ for each honest h .*
3. **CarryComparerounds.** *For each CSCP round (CSA levels of Protocol 12 Phase A, and prefix comparison rounds of Phase B), compute the masked AND-gate evaluations $(d_h^{(\ell)}, e_h^{(\ell)})$ for each honest $h \notin C$ using the Beaver triple shares $\{[a_g]_h, [b_g]_h, [c_g]_h\}$ and boolean shares $[\rho_h]$, both derived from session keys $\{K_{hj,\tau}\}$ via the identical PRF formulas as Protocol 13. Broadcast these messages on behalf of each honest h . After all rounds, each party (and $\mathcal{S}^{\text{MPC}}$) reconstructs the carry bit $c = [\sum_{i \in S} \rho_i > B \bmod \alpha]$ where $B = \sum_{i \in S} \tilde{b}_i$.*
4. **Internal BCC flag.** *Assemble $\hat{\mathbf{y}} = \sum_{h \in S} \hat{\mathbf{y}}_h$ and store $(\hat{\mathbf{y}}, \text{BCC}(\mathbf{A}\hat{\mathbf{y}}))$ together with $\mathbf{w}_1 = \text{HighBits}(\mathbf{A}\hat{\mathbf{y}}, 2\gamma_2)$. Do not abort here. The preprocessing transcript is always delivered in full to all parties (as in the real protocol, where no party knows $\hat{\mathbf{y}}$ during preprocessing).*

Online signing simulation. On a signing request for session τ :

1. **BCC gate.** If $\neg \text{BCC}(\mathbf{A}\hat{\mathbf{y}})$ for the stored session, broadcast an online abort to all parties and schedule a fresh preprocessing session (retry count $\text{Geom}(p_{\text{BCC}})$, same distribution as the real protocol).
2. Otherwise, proceed identically to Simulator 30, steps 1–5: compute $\mathbf{z} = \hat{\mathbf{y}} + c\mathbf{s}_1$, derive hint $\mathbf{h}$ from public data, submit $\sigma = (\tilde{c}, \mathbf{z}, \mathbf{h})$ to $\mathcal{F}_{\text{Sig}}$, and assign honest response shares $\mathbf{z}_i$ uniformly subject to the Lagrange constraint.

Proof of Theorem 33. We analyze each phase in the $(\mathcal{F}_{\text{PRF}}, \mathcal{F}_{\text{RO}})$ -hybrid model.

KeyGen. Identical to Theorem 31: Shamir perfect privacy gives $\Delta = 0$. Feldman commitments $\Phi_{h,k} = \mathbf{A} \cdot a_{h,k}$ are computed from the same polynomial coefficients in real and ideal, so their distributions are identical. $\Delta = 0$.

Preprocessing. $\mathcal{F}_{\text{PRF}}$ delivers s_{hj} to both the honest party h (simulated by $\mathcal{S}^{\text{MPC}}$) and each party j (honest or corrupted) in the pair. Thus $\mathcal{S}^{\text{MPC}}$ holds exactly the same seeds as the honest parties in the real protocol.

- (a) *Nonce DKG evaluations.* $g_h(j)$ for honest $h \notin C$, corrupted $j \in C$: Shamir perfect privacy (identical argument to Lemma 13). $\Delta = 0$.
- (b) *Masked broadcasts.* $\tilde{H}_h$ and $\tilde{b}_h$ are computed from $\{s_{hj}\}$ and $\hat{\mathbf{y}}_h$ by a deterministic formula identical in real and ideal. Since $\mathcal{F}_{\text{PRF}}$ delivers the same seeds to the honest party in both worlds, and $\hat{\mathbf{y}}_h$ has the same distribution (step 1 of the simulator samples identically to the real protocol), the broadcasts have identical joint distributions. $\Delta = 0$.
- (c) *CarryCompareround messages and carry bit.* The Beaver triple shares $\{[a_g]_h, [b_g]_h, [c_g]_h\}$ and boolean shares $[\rho_h]$ for each honest h are deterministically derived from $\{K_{h,j,\tau}\}$, which $\mathcal{S}^{\text{MPC}}$ holds via $\mathcal{F}_{\text{PRF}}$ with the same values as the real protocol. Therefore, the masked AND-gate evaluations $(d_h^{(\ell)}, e_h^{(\ell)})$ at every CSCP level ℓ have *identical* distributions in real and ideal: they are computed by the same deterministic formula from the same inputs. The final carry bit $c = [\sum_i \rho_i > B \bmod \alpha]$ is a deterministic function of these shared values. $\Delta = 0$.
- (d) *BCC abort placement and retry count.* In the real protocol, BCC failure is detected *online* (no party knows $\hat{\mathbf{y}}$ during preprocessing). The simulator likewise delivers a complete preprocessing transcript before signalling an online abort for BCC-failing sessions (Simulator step 4). Thus the abort point matches the real world exactly. The number of aborted sessions is $\text{Geom}(p_{\text{BCC}})$ in both worlds (since $\hat{\mathbf{y}}$ has the same distribution and BCC is deterministic). $\Delta = 0$.

Online signing. For BCC-passing sessions, $\Delta = 0$ by Theorem 31, parts (a)–(d): $\tilde{c}$, $\mathbf{z}$, $\mathbf{h}$, and $\{\mathbf{z}_i\}_{i \notin C}$ have identical joint distributions in real and ideal (same $(\mathbf{pk}, m, \mathbf{w}_1)$ for the challenge; Shamir perfect privacy of the response shares; $\mathbf{h}$ and $\mathbf{z}$ depend only on public data and $\hat{\mathbf{y}}, \mathbf{s}_1$, which are the same in both worlds). For BCC-failing sessions, both worlds deliver an online abort with identical probability $1 - p_{\text{BCC}}$, so $\Delta = 0$ there as well.

Combining.

$$\Delta(\text{REAL}, \text{IDEAL}) \leq \underbrace{0}_{\text{KeyGen}} + \underbrace{0}_{\text{Preproc.}} + \underbrace{0}_{\text{Signing}} = 0,$$

giving perfect UC security in the $(\mathcal{F}_{\text{PRF}}, \mathcal{F}_{\text{RO}})$ -hybrid model.

PRF instantiation. Replacing $\mathcal{F}_{\text{PRF}}$ with a concrete PRF F introduces indistinguishability at most $Q_{\text{sess}} \cdot \text{Adv}_F^{\text{PRF}}(\mathcal{B})$ by a standard hybrid argument: $\mathcal{B}$ uses session τ^* as its PRF challenge and simulates all other sessions using the real F , forwarding any distinguishing advantage of $\mathcal{A}$ to break PRF security. $\square$

Algorithm 1 TALUS-MPC.Refresh

Require: Party i 's current epoch state $(s_{1,i}, \{s_{ij}\}_{j \neq i}, V_i)$.
Ensure: Updated state $(s'_{1,i}, \{s'_{ij}\}_{j \neq i}, V'_i)$ with all intermediate values erased.

— **Round 1 (broadcast + P2P)** —

- 1: Sample degree- $(T-1)$ polynomial $f_i(X) = \sum_{k=0}^{T-1} \mathbf{a}_{i,k} X^k$ with $\mathbf{a}_{i,0} = \mathbf{0}$ and $\mathbf{a}_{i,k} \xleftarrow{\$} R_q^{n\ell}$ for $k \geq 1$.
- 2: Broadcast Feldman commitments $F_{i,k} \leftarrow \mathbf{A} \cdot \mathbf{a}_{i,k}$ for $k = 0, \dots, T-1$. $\triangleright F_{i,0} = \mathbf{0}$, publicly verifiable
- 3: For each $j \neq i$: send $f_i(j)$ to party j over a confidential authenticated channel.

— **Local computation (after receiving all messages)** —

- 4: **for** each $h \neq i$ **do**
- 5: Check $\mathbf{A} \cdot f_h(i) \stackrel{?}{=} \sum_{k=0}^{T-1} F_{h,k} \cdot i^k \pmod{q}$. If failed: broadcast blame against h and abort.
 $\triangleright$ Malicious deviations are identifiable
- 6: **end for**
- 7: $s'_{1,i} \leftarrow s_{1,i} + \sum_{h=1}^N f_h(i) \pmod{q}$
- 8: $V'_i \leftarrow V_i + \sum_{h=1}^N \mathbf{A} \cdot f_h(i)$ $\triangleright$ Publicly recomputable from broadcast commitments
- 9: **for** each $j \neq i$ **do**
- 10: $s'_{ij} \leftarrow H(\text{"talus-refresh"} \parallel i \parallel j \parallel f_i(j) \parallel f_j(i))$
- 11: **end for**

— **Erasure** —

- 12: Erase $\{f_h(i)\}_{h \neq i}$ and $\{f_i(j)\}_{j \neq i}$; overwrite $(s_{1,i}, \{s_{ij}\}) \leftarrow (s'_{1,i}, \{s'_{ij}\})$.

C Proactive Key Refresh

C.1 Motivation and Erasure Model

The base TALUS-MPC protocol (§7) is proven secure against a *static* adversary that fixes its corruption set before the protocol begins. Long-running deployments require periodic *key refresh*: secret shares and pairwise seeds are re-randomised so that an adversary that compromises up to $T-1$ parties in one epoch gains no advantage in a later epoch even if it compromises a different set of parties.

We follow the standard *mobile adversary* model of Herzberg et al. [HJKY95]: the adversary may choose a fresh corruption set $C_e \subseteq [N]$ with $|C_e| \leq T-1$ at the beginning of each epoch e ; between epochs, honest parties execute Algorithm 1 in a *recovery window* during which no party is under adversarial control.

Erasure schedule. Honest parties erase ephemeral state at two points:

1. **After each signing session:** erase nonce share $\mathbf{y}_i$, mask ρ_i , and all Beaver-triple intermediate values $([a]_i, [b]_i, [c]_i)$.
2. **After each refresh:** erase all refresh evaluations $\{f_h(i)\}_{h \in [N]}$ and $\{f_i(j)\}_{j \neq i}$; overwrite old share $s_{1,i} \leftarrow s'_{1,i}$ and old seeds $s_{ij} \leftarrow s'_{ij}$.

Erasure is a standard assumption in proactive secret sharing [HJKY95] and in adaptive threshold signing [CKGW24].

C.2 Refresh Protocol

Remark 36 (Malicious behaviour during Refresh). A party h that broadcasts $F_{h,0} \neq \mathbf{0}$ or sends an evaluation $f_h(i)$ inconsistent with its Feldman commitments is immediately identified by the check at line 5 and excluded via identifiable abort. Security is maintained as long as at most $T-1$ parties are malicious, consistent with the TALUS-MPC threat model.

Remark 37 (TALUS-TEE Refresh). TALUS-TEE.Refresh (§6.5) applies the same Shamir share refresh without the CSCP seed step: no pairwise seeds exist in the TEE model. The TEE

coordinator verifies Feldman consistency and the result is a single-round refresh achieving proactive security for TALUS-TEE under the same [HJKY95] mobile adversary model.

C.3 Security Analysis

Definition 9 (Proactive EUF-CMA). A (T, N) -threshold scheme is *proactive EUF-CMA secure* (against a mobile malicious adversary over E epochs) if for every PPT $\mathcal{A}$:

- At the start of epoch e , $\mathcal{A}$ adaptively chooses $C_e \subseteq [N]$ with $|C_e| \leq T-1$ and receives the epoch- e state of all $i \in C_e$.
- Between epochs, Algorithm 1 runs in a recovery window (no party is corrupted); honest parties erase old state.
- $\mathcal{A}$ makes at most q_s signing queries in total.
- $\mathcal{A}$ outputs (m^*, σ^*) not queried and wins if $\text{Verify}(\text{pk}, m^*, \sigma^*) = 1$.

$\text{Adv}_{\text{TALUS-MPC}}^{\text{Pro-EUF-CMA}}(\mathcal{A}, E)$ denotes the winning probability.

Lemma 15 (Share Refresh Independence). *Let $\mathcal{A}$ control C_e during epoch e with $|C_e| \leq T-1$. After one execution of Algorithm 1, for every honest party $j \notin C_e$, the additive update $\Delta_j = \sum_{h=1}^N f_h(j)$ is statistically uniform over $R_q^{n_\ell}$, independent of $\mathcal{A}$'s epoch- e view.*

Proof. At least one honest party $h^* \notin C_e$ contributes $f_{h^*}(j) = \mathbf{a}_{h^*,0} + \sum_{k \geq 1} \mathbf{a}_{h^*,k} j^k = \sum_{k \geq 1} \mathbf{a}_{h^*,k} j^k$ (since $\mathbf{a}_{h^*,0} = \mathbf{0}$), where $\{\mathbf{a}_{h^*,k}\}_{k \geq 1}$ are uniform over $R_q^{n_\ell}$ and independent of $\mathcal{A}$'s view. By Shamir perfect privacy, the $|C_e| \leq T-1$ evaluations $\{f_{h^*}(i) : i \in C_e\}$ are jointly uniform and statistically independent of $f_{h^*}(j)$ for $j \notin C_e$. Hence Δ_j contains $f_{h^*}(j)$ as a statistically independent uniform term; the sum Δ_j is therefore uniform over $R_q^{n_\ell}$. $\square$

Lemma 16 (Seed Refresh Independence). *Under the HJKY95 recovery window assumption, for any pair (i, j) with $j \notin C_e$ (party j is honest and runs the refresh correctly), the updated seed $s'_{ij} = H(\text{"talus-refresh"} \| i \| j \| f_i(j) \| f_j(i))$ is computationally random given $\mathcal{A}$'s view at the start of epoch $e+1$.*

Proof. During the refresh (recovery window), $\mathcal{A}$ does not control any party. Party $j \notin C_e$ honestly samples f_j and sends $f_j(i)$ to i ; after computing s'_{ij} , party j immediately erases $f_j(i)$. At the start of epoch $e+1$, $\mathcal{A}$ corrupts C_{e+1} :

- If $j \notin C_{e+1}$: $\mathcal{A}$ does not learn $f_j(i)$ or s'_{ij} .
- If $j \in C_{e+1}$: $\mathcal{A}$ learns party j 's epoch- $e+1$ state, which includes s'_{ij} but not $f_j(i)$ (erased).

In either case, $f_j(i)$ is unknown to $\mathcal{A}$. Even if $\mathcal{A}$ knows $f_i(j)$ (e.g., from $i \in C_{e+1}$ and its erasure of $f_i(j)$ happening after $\mathcal{A}$ corrupts i), computing the preimage of $H(\cdot \| f_j(i))$ requires querying the random oracle at $f_j(i) \in R_q^{n_\ell}$, a uniformly random point with q^{n_ℓ} possibilities. The probability that any polynomial number of $\mathcal{A}$'s RO queries hits the correct $f_j(i)$ is negligible. Hence s'_{ij} is computationally random. $\square$

Theorem 35 (Proactive EUF-CMA for TALUS-MPC). *Under the hardness assumptions of Theorem 25 and the HJKY95 recovery window model, TALUS-MPC extended with Algorithm 1 achieves proactive EUF-CMA security. For E epochs and q_s total signing queries:*

$$\text{Adv}_{\text{TALUS-MPC}}^{\text{Pro-EUF-CMA}}(\mathcal{A}, E) \leq E \cdot \left(\frac{Q_H^2}{|C|} + \text{Adv}_{\text{Beaver}}^{\text{PRF}}(\mathcal{B}_1) + q_s \cdot R_2^{\text{vec}} \cdot \text{Adv}_{\text{ML-DSA}}^{\text{EUF-CMA}}(\mathcal{B}_2) \right). \quad (5)$$

Proof. By induction on the number of epochs.

Base case ($e = 1$). Theorem 25 applies directly with static corruption set C_1 , yielding the per-epoch bound.

Inductive step. Assume security holds through epoch e . We show epoch $e+1$ starts in a state that satisfies the same conditions as epoch 1.

After Algorithm 1:

- (i) *Share independence.* By Lemma 15, for every honest party $j \notin C_e$, the update Δ_j is statistically uniform and independent of $\mathcal{A}$'s epoch- e view. The erasure of $s_{1,j}$ ensures the adversary sees only $s'_{1,j} = s_{1,j} + \Delta_j$, which is uniformly distributed over the affine coset of valid Shamir shares. An adversary corrupting up to $T-1$ parties in C_{e+1} obtains $T-1$ fresh shares, insufficient to reconstruct $\mathbf{s}_1$ (Shamir threshold).
- (ii) *Seed independence.* By Lemma 16, for every pair (i, j) with at least one honest member, the new seed s'_{ij} is computationally random given $\mathcal{A}$'s epoch- e view. The erasure of old seeds $\{s_{ij}\}$ ensures that epoch- $e+1$ Beaver triples derived from $\{s'_{ij}\}$ are independent of epoch- e protocol transcripts.

Hence epoch $e+1$ begins with fresh shares and seeds satisfying the conditions of the static TALUS-MPC model. Applying Theorem 25 to epoch $e+1$ yields the per-epoch bound. A union bound over E epochs gives equation (5). $\square$

Remark 38 (Adaptive Security with Erasures). The *base* TALUS-MPC protocol (single epoch, no refresh) additionally achieves *adaptive EUF-CMA* against an adversary that corrupts at most $T-1$ parties at any point during protocol execution, provided honest parties erase $\mathbf{y}_i$ immediately upon computing and sending $\mathbf{z}_i$.

Proof sketch. Upon adaptive corruption of party i after online signing, $\mathcal{A}$ learns $(s_{1,i})$ only (since $\mathbf{y}_i$ is erased). The transcript $\mathbf{z}_i = \mathbf{y}_i + c\mathbf{s}_{1,i}$ is already public; obtaining $s_{1,i}$ reveals no new information about other parties' shares. With at most $T-1$ adaptively-obtained shares, $\mathcal{A}$ cannot reconstruct $\mathbf{s}_1$. A standard ROM simulator explains all transcripts consistently by programming the random oracle at challenge queries, following the same technique used in adaptive FROST security [CKGW24].

Combined with Algorithm 1, TALUS-MPC achieves *adaptive EUF-CMA within each epoch* and *proactive security across epoch boundaries*. Full adaptive security without the erasure assumption remains open (it requires equivocable commitments for the HighBits rounding; see §10.4).

D Shift-Invariance Analysis

The Carry Elimination Framework relies on the *shift-invariance* property of HighBits: for a coefficient $w \in \mathbb{Z}_q$, if $|\text{LowBits}(w)| < \gamma_2 - |\delta|$, then $\text{HighBits}(w + \delta) = \text{HighBits}(w)$.

BCC is precisely the condition that shift-invariance holds for all shifts δ with $|\delta| \leq \beta = \tau\eta$. The tables below quantify the fraction of coefficients satisfying shift-invariance for various shift magnitudes.

D.1 Shift-Invariance Probability vs. Shift Magnitude

Table 15: Shift-invariance probability $p(\delta) = 1 - |\delta|/\gamma_2$ for selected shift magnitudes δ . ML-DSA-65: $\gamma_2 = 261,888$, $\beta = 196$.

$ \delta $	$p(\delta)$	Context
196 = β	0.999252	BCC threshold
392 = 2β	0.998504	
200,704 = $\tau \cdot 2^{d-1}$	0.234	ct_0 maximum shift
261,692 = $\gamma_2 - \beta$	0.000748	just inside boundary

D.2 Carry Probability for Distributed $\mathbf{w}_1$

For party count $T \in \{2, 3, 5, 8\}$, the secondary carry probability (probability that $|\text{carry}| \geq 2$, i.e. a 1-bit carry indicator is insufficient) is measured by `experiments/carry_identity.py` over 10^5 trials $\times$ 1,536 coefficients:

In the Lagrange CEF protocol (Protocol 4), since secondary carry probability exceeds 25% for $T \geq 3$, a 1-bit carry indicator is insufficient. In TALUS-MPC's additive masked broadcast

Table 16: Secondary carry probability vs. threshold T . $\sum |\lambda_i|$ is the ℓ_1 -norm of integer Lagrange coefficients for $S = \{1, \dots, T\}$. ML-DSA-65: $\alpha = 523,776$.

T	$\sum \lambda_i $	Carry range	$\Pr[\text{carry} = 0]$	$\Pr[\text{carry} \geq 2]$
2	3	$[-1, 1]$	0.500	$< 0.001\%$
3	7	$[-3, 3]$	0.296	25.9%
5	31	$[-15, 15]$	0.083	75.5%
8	255	$[-121, 120]$	0.012	96.5%

(Section 5.6), this issue does not arise: $\lfloor B/\alpha \rfloor$ is directly public and carries are handled by a single CSCP comparison bit.

E TALUS for All ML-DSA Parameter Sets

TALUS applies to all three ML-DSA parameter sets. Table 17 lists the BCC pass rate and related quantities for each.

Table 17: BCC pass rates and parameters for ML-DSA-44, -65, -87. $p_{\text{coeff}} = 1 - \beta/\gamma_2$, $p_{\text{BCC}} = p_{\text{coeff}}^{nk}$.

Variant	k	ℓ	η	τ	β	γ_2	p_{coeff}	p_{BCC}
ML-DSA-44	4	4	2	39	78	95,232	0.999181	$(0.999181)^{1024}$
ML-DSA-65	6	5	4	49	196	261,888	0.999252	$(0.999252)^{1536}$
ML-DSA-87	8	7	2	60	120	261,888	0.999542	$(0.999542)^{2048}$

ML-DSA-44. $p_{\text{BCC}} \approx e^{-1024 \times 0.000819} \approx e^{-0.838} \approx 0.433$. Higher pass rate than ML-DSA-65 due to the smaller polynomial dimension $nk = 1024$ (ML-DSA-44 actually has a *higher* β/γ_2 ratio: $78/95,232 \approx 0.000819$ vs. $196/261,888 \approx 0.000749$ for ML-DSA-65).

ML-DSA-65. $p_{\text{BCC}} \approx 0.317$ (main body analysis).

ML-DSA-87. $p_{\text{BCC}} \approx e^{-2048 \times 0.000458} \approx e^{-0.937} \approx 0.392$. Despite larger $nk = 2048$, the smaller β/γ_2 ratio gives a higher pass rate than ML-DSA-65.

Security levels. The BCC Safety Theorem (Theorem 2) and its proof hold for all parameter sets, with $\beta = \tau\eta$ substituted accordingly. The security reduction (Theorem 23) likewise holds with loss $Q_H^2/|\mathcal{C}|$ where $|\mathcal{C}| = \binom{256}{\tau} 2^\tau$ in each case.

F FIPS 204 Hint Encoding

TALUS produces signatures $\sigma = (\tilde{c}, \mathbf{z}, \mathbf{h})$ where the hint $\mathbf{h}$ is generally *nonzero*: BCC ensures that $\mathbf{h}$ is computable from public data and satisfies $\text{wt}(\mathbf{h}) \leq \omega = 55$, but does not force $\mathbf{h} = \mathbf{0}$. The expected hint weight is ≈ 38 (Lemma 9).

Computing the hint from public data. After assembling $\mathbf{z}$, the coordinator computes:

$$\mathbf{r} = \mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d \pmod{q}$$

and sets, for each coefficient $j \in [nk]$:

$$h_j = \mathbf{1}[\text{HighBits}(r_j, 2\gamma_2) \neq (w_1)_j].$$

Both $\mathbf{r}$ and $\mathbf{w}_1$ are public, so no secret key material is required.

Why the hint is nonzero in general. The identity $\mathbf{r} = \mathbf{w} - c\mathbf{s}_2 + c\mathbf{t}_0 \pmod{q}$ shows that $\mathbf{r}$ differs from $\mathbf{w} - c\mathbf{s}_2$ by the low-bit term $c\mathbf{t}_0$ (with $\|c\mathbf{t}_0\|_\infty \leq \tau \cdot 2^{d-1} = 200,704$). When $c\mathbf{t}_0$ pushes a coefficient across a modular boundary (a multiple of $\alpha = 2\gamma_2$), $\text{HighBits}(r_j)$ changes, triggering $h_j = 1$. Under BCC, the per-coefficient carry probability is $\approx 2.5\%$ empirically (Lemma 9), giving expected hint weight ≈ 38 and $\Pr[\text{wt}(\mathbf{h}) > 55] < 10^{-3}$.

Encoding (FIPS 204 Algorithm 26). Algorithm 26 (`sigEncode`) encodes $\mathbf{h} \in \{0, 1\}^{nk}$ as $\omega + k$ bytes: for each of the k polynomials, the positions of nonzero entries (at most ω total), followed by a count byte. Since $\text{wt}(\mathbf{h}) \leq \omega = 55$ (ensured by BCC) and $k = 6$, the hint occupies $55 + 6 = 61$ bytes for ML-DSA-65, exactly as in the standard.

Decoding (FIPS 204 Algorithm 27). Algorithm 27 (`sigDecode`) reconstructs $\mathbf{h}$ from the encoding. It rejects any signature where the encoded hint weight exceeds ω ; BCC ensures this check passes.

Verification compatibility. A FIPS 204-compliant verifier processing $\sigma = (\tilde{c}, \mathbf{z}, \mathbf{h})$:

1. Decodes $\mathbf{h}$ via Algorithm 27 (rejects if $\text{wt}(\mathbf{h}) > \omega$).
2. Computes $\mathbf{w}'_1 = \text{UseHint}(\mathbf{h}, \mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d, 2\gamma_2)$.
3. Accepts iff $H(\mu\|\mathbf{w}'_1) = \tilde{c}$ (holds by Theorem 8) and $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$ (checked in step 4 of the signing protocol).

This is the standard FIPS 204 verification path; no special handling of the hint is required. Signatures produced by TALUS are indistinguishable in format from standard single-party ML-DSA-65 signatures.

FIPS 204 cross-check (VERIFY-ACVP). The Rust implementation was verified against the independent `dilithium-py` 1.4.0 ML-DSA-65 verifier [Pop24] via the cross-check script `acvp_crosscheck.py` (`experiments/`). The script generates a TALUS-TEE signature (3-of-5, seed 42) and invokes `ML_DSA_65.verify`: `result = PASS` (confirmed 2026-03-12). During this process, a bug was identified and fixed: the original `compute_challenge_hash` hashed `(pkBytes\|m\|w1Enc)` directly, whereas FIPS 204 requires the two-step $\text{tr} = H(\text{pk}, 64)$, $\mu = H(\text{tr}\|M', 64)$, $\tilde{c} = H(\mu\|w1Enc, \lambda)$. The fix is in `crates/talus-core/src/types.rs`; all 26 unit tests continue to pass after the correction.

G Trilemma Proof and Carry–Privacy Duality

G.1 Proof of Theorem 1

We treat the two parts in turn.

Part 1: no group homomorphism is simultaneously hiding and binding.

Let $f: (R_q^\ell, +) \rightarrow (G, \cdot)$ be a group homomorphism into a finite abelian group G .

Step 1 (CRT reduction). Since q is prime and $2n \mid q - 1$ (for ML-DSA, $q = 8,380,417$ and $n = 256$), the polynomial $X^n + 1$ splits into n distinct linear factors over $\mathbb{Z}_q$. The CRT isomorphism gives $R_q \cong \mathbb{Z}_q^n$, hence $(R_q^\ell, +) \cong (\mathbb{Z}_q^{n\ell}, +)$ as additive groups.

Step 2 (image structure). Every non-zero element of $\mathbb{Z}_q^{n\ell}$ has additive order q . Under a group homomorphism, orders divide, so every element of the image $f(\mathbb{Z}_q^{n\ell})$ has order dividing q . Since q is prime, $f(\mathbb{Z}_q^{n\ell})$ is an elementary abelian q -group, isomorphic to $\mathbb{Z}_q^d$ for some $d \geq 0$.

Step 3 (prime-field linearity). The map $f: \mathbb{Z}_q^{n\ell} \rightarrow \mathbb{Z}_q^d$ is additive by assumption. Since $\mathbb{Z}_q$ is a prime field, scalar multiplication by $a \in \mathbb{Z}_q$ is repeated addition:

$$f(a\mathbf{v}) = f(\underbrace{\mathbf{v} + \cdots + \mathbf{v}}_a) = a \cdot f(\mathbf{v}) \quad \text{for all } a \in \mathbb{Z}_q, \mathbf{v} \in \mathbb{Z}_q^{n\ell}.$$

Therefore f is $\mathbb{Z}_q$ -linear, representable as a matrix $\mathbf{M} \in \mathbb{Z}_q^{d \times n\ell}$. (This step uses the full strength of q being prime; over a non-prime ring the conclusion can fail.)

Step 4 (binding $\Rightarrow$ full rank). If f is binding, then $\ker(\mathbf{M}) = \{\mathbf{0}\}$ (any non-zero kernel element is a binding breach). This forces $\text{rank}(\mathbf{M}) = n\ell$, hence $d \geq n\ell$.

Step 5 (full rank $\Rightarrow$ not hiding). When $\mathbf{M}$ has full column rank over $\mathbb{Z}_q$, the Gram matrix $\mathbf{M}^T \mathbf{M} \in \mathbb{Z}_q^{n\ell \times n\ell}$ is invertible and the left inverse $\mathbf{M}^+ = (\mathbf{M}^T \mathbf{M})^{-1} \mathbf{M}^T$ recovers $\mathbf{y} = \mathbf{M}^+(f(\mathbf{y}))$ in $O((n\ell)^\omega)$ field operations (Gaussian elimination over $\mathbb{Z}_q$). This is *efficient*: for ML-DSA-65, $n\ell = 1280$, so inversion is a routine linear-algebra computation. Thus f reveals $\mathbf{y}$ in polynomial time, violating hiding. $\square$

Part 2 (HighBits is not a group homomorphism). Consider the additive groups $(\mathbb{Z}_q, +)$ and $(\mathbb{Z}_M, +)$ where $M = (q - 1)/\alpha < q$. Let $\phi: \mathbb{Z}_q \rightarrow \mathbb{Z}_M$ be any group homomorphism. The kernel $\ker(\phi)$ is a subgroup of $\mathbb{Z}_q$. Since q is prime, $\mathbb{Z}_q$ is a simple group (no proper non-trivial subgroups). Therefore either $\ker(\phi) = \{0\}$ or $\ker(\phi) = \mathbb{Z}_q$.

If $\ker(\phi) = \{0\}$, then ϕ is injective, so $|\mathbb{Z}_q| \leq |\mathbb{Z}_M|$, i.e., $q \leq M$. But $M = (q - 1)/\alpha < q$ (since $\alpha \geq 2$), a contradiction.

Therefore $\ker(\phi) = \mathbb{Z}_q$ and ϕ is the zero map.

The function $\text{HighBits}: \mathbb{Z}_q \rightarrow \{0, \dots, M-1\}$ is manifestly not the zero map ($\text{HighBits}(\alpha) = 1 \neq 0$), so HighBits is not a group homomorphism from $(\mathbb{Z}_q, +)$ to any group of order at most $M < q$. Consequently, composing an R_q -linear map with coefficient-wise HighBits does not yield a function satisfying (i). $\square$

G.2 Carry–Privacy Duality

The carry δ_j and the secret value $r_{0,j} = \text{LowBits}((\mathbf{A}\mathbf{y})_j)$ are both determined by the same quantity: the LowBits sum $S_j = \sum_{i=1}^T \text{LowBits}(w_{i,j})$. Specifically,

$$\delta_j = \lfloor S_j / \alpha \rfloor, \quad (6)$$

$$r_{0,j} = S_j \bmod \alpha. \quad (7)$$

These are the quotient and remainder of the same integer division.

The vector $\mathbf{r}_0$ is security-critical. From a single valid signature $(\mathbf{z}, \mathbf{h})$ and the public key $(\rho, \mathbf{t}_1)$, one computes $\mathbf{r} = \mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d$ (all public values). Since $\mathbf{r} = \mathbf{A}\mathbf{y} + c(\mathbf{t}_0 - \mathbf{s}_2)$ and $\mathbf{A}\mathbf{y} = \mathbf{w}_1 \cdot \alpha + \mathbf{r}_0$ (where $\mathbf{w}_1$ is determined by the hint), knowledge of $\mathbf{r}_0$ yields

$$c \cdot (\mathbf{t}_0 - \mathbf{s}_2) = \mathbf{r} - \mathbf{w}_1 \cdot \alpha - \mathbf{r}_0.$$

Since c is invertible over R_q with overwhelming probability, one recovers $\mathbf{t}_0 - \mathbf{s}_2 = \mathbf{A}\mathbf{s}_1 - \mathbf{t}_1 \cdot 2^d$ (using $\mathbf{t} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2 = \mathbf{t}_1 \cdot 2^d + \mathbf{t}_0$). Since $\mathbf{t}_1$ is public, this reveals $\mathbf{A}\mathbf{s}_1$. For ML-DSA parameters ($k \geq \ell$), $\mathbf{A}$ has full column rank, so the left inverse recovers $\mathbf{s}_1$ in polynomial time. Thus $\mathbf{r}_0$ enables *full key recovery from a single signature*.

Proposition 3 (Carry–Privacy Duality). *Any protocol that reveals S_j to any party reveals both δ_j (needed for correctness) and $r_{0,j}$ (fatal for security). Therefore, the carry must be computed without revealing $S_j \bmod \alpha$.*

Proof. Equations (6) and (7) are the quotient–remainder decomposition $S_j = \delta_j \cdot \alpha + r_{0,j}$. Knowledge of S_j determines both components uniquely (since $0 \leq r_{0,j} < \alpha$). Knowledge of $\mathbf{r}_0$ together with a single valid signature and the public key reveals $\mathbf{A}\mathbf{s}_1$, hence $\mathbf{s}_1$ via the left inverse, as shown above. $\square$ $\square$

The carry–privacy duality is the central design constraint for any threshold ML-DSA protocol. The carry must be computed from S_j (correctness), yet $S_j \bmod \alpha$ must remain hidden (security). The protocol must evaluate the *quotient* $\lfloor S_j / \alpha \rfloor$ without learning the *remainder* $S_j \bmod \alpha$. The Carry Elimination Framework (Section 5) achieves this via a single-bit distributed comparison per coefficient.

References

- [AFL⁺16] Toshinori Araki, Jun Furukawa, Yehuda Lindell, Ariel Nof, and Kazuma Ohara. High-throughput semi-honest secure three-party computation with an honest majority. In *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, pages 805–817. ACM, 2016.
- [BCdP⁺26] Giacomo Borin, Sofía Celi, Rafael del Pino, Thomas Espitau, Shuichi Katsumata, Guilhem Niot, Thomas Prest, and Kaoru Takemure. Hermine: An efficient lattice-based FROST-like threshold signature. Cryptology ePrint Archive, Paper 2026/419, 2026. Available from World Wide Web: <https://eprint.iacr.org/2026/419>. Lattice-based (not FIPS 204 compatible).
- [BCGI18] Elette Boyle, Geoffroy Couteau, Niv Gilboa, and Yuval Ishai. Compressing vector OLE. In *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, pages 896–912. ACM, 2018.
- [BdCE⁺26] Alexander Bienstock, Leo de Castro, Daniel Escudero, Antigoni Polychroniadou, and Akira Takahashi. Quorus: Efficient, scalable threshold ML-DSA signatures from MPC. Cryptology ePrint Archive, Paper 2025/1163 (revised 2026-01-28), 2026. Available from World Wide Web: <https://eprint.iacr.org/2025/1163>.
- [Bea92] Donald Beaver. Efficient multiparty protocols using circuit randomization. In *Advances in Cryptology – CRYPTO 1991*, volume 576 of *Lecture Notes in Computer Science*, pages 420–432. Springer, 1992.
- [BGI15] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing. In *Advances in Cryptology – EUROCRYPT 2015*, volume 9057 of *Lecture Notes in Computer Science*, pages 337–367. Springer, 2015.
- [BGI16] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing: Improvements and extensions. In *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, pages 1292–1303, 2016.
- [BKL⁺25] Cecilia Boschini, Darya Kaviani, Russell W. F. Lai, Giulio Malavolta, Akira Takahashi, and Mehdi Tibouchi. Ringtail: Practical two-round threshold signatures from learning with errors. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 149–164. IEEE, 2025.
- [BMD⁺17] Ferdinand Brasser, Urs Müller, Alexandra Dmitrienko, Kari Kostiaainen, Srdjan Capkun, and Ahmad-Reza Sadeghi. Software grand exposure: SGX cache attacks are practical. In *11th USENIX Workshop on Offensive Technologies (WOOT 17)*. USENIX Association, 2017.
- [BMV19] Luís T. A. N. Brandão, Nicky Mouha, and Apostol Vassilev. Threshold schemes for cryptographic primitives: Challenges and opportunities in standardization and validation of threshold cryptography. NISTIR 8214, National Institute of Standards and Technology, 2019. Available from World Wide Web: <https://csrc.nist.gov/pubs/ir/8214/final>.
- [BPR12] Abhishek Banerjee, Chris Peikert, and Alon Rosen. Pseudorandom functions and lattices. In *Advances in Cryptology – EUROCRYPT 2012*, volume 7237 of *Lecture Notes in Computer Science*, pages 719–737. Springer, 2012.
- [CdPE⁺26] Sofía Celi, Rafaël del Pino, Thomas Espitau, Guilhem Niot, and Thomas Prest. Efficient threshold ML-DSA. Cryptology ePrint Archive, Paper 2026/013, 2026. Available from World Wide Web: <https://eprint.iacr.org/2026/013>. To appear at USENIX Security ’26.

- [CKGW24] Deirdre Connolly, Chelsea Komlo, Ian Goldberg, and Christopher A. Wood. Two-Round Threshold Schnorr Signatures with FROST. RFC 9591, 2024. Available from World Wide Web: <https://www.rfc-editor.org/rfc/rfc9591.html>.
- [DKL⁺18] Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Dilithium: A lattice-based digital signature scheme. *IACR Trans. Cryptogr. Hardw. Embed. Syst. (TCHES)*, 2018(1):238–268, 2018.
- [DKLS25] Antonín Dufka, Semjon Kravtšenko, Peeter Laud, and Nikita Snetkov. Trilithium: Efficient and universally composable distributed ML-DSA signing. Cryptology ePrint Archive, Paper 2025/675, 2025. Available from World Wide Web: <https://eprint.iacr.org/2025/675>. 2-party protocol; requires trusted Correlated Randomness Provider.
- [DN07] Ivan Damgård and Jesper Buus Nielsen. Scalable and unconditionally secure multi-party computation. In *Advances in Cryptology – CRYPTO 2007*, volume 4622 of *Lecture Notes in Computer Science*, pages 572–590. Springer, 2007.
- [dPKPR24] Rafaël del Pino, Shuichi Katsumata, Thomas Prest, and Mélissa Rossi. Raccoon: A masking-friendly signature proven in the probing model. In *Advances in Cryptology – CRYPTO 2024*, volume 14920 of *Lecture Notes in Computer Science*, pages 409–444. Springer, 2024.
- [dPN25] Rafaël del Pino and Sven Niot. Finally! a compact lattice-based threshold signature. In *Public-Key Cryptography – PKC 2025*, volume 15676 of *Lecture Notes in Computer Science*, pages 169–199. Springer, 2025.
- [DPSZ12] Ivan Damgård, Valerio Pastro, Nigel P. Smart, and Sarah Zakarias. Multiparty computation from somewhat homomorphic encryption. In *Advances in Cryptology – CRYPTO 2012*, volume 7417 of *Lecture Notes in Computer Science*, pages 643–662. Springer, 2012.
- [ENP24] Thomas Espitau, Guilhem Niot, and Thomas Prest. Flood and submerse: Distributed key generation and robust threshold signature from lattices. In *Advances in Cryptology – CRYPTO 2024*, volume 14926 of *Lecture Notes in Computer Science*, pages 425–458. Springer, 2024.
- [Fel87] Paul Feldman. A practical scheme for non-interactive verifiable secret sharing. In *Proc. 28th Annual Symp. Foundations of Computer Science (FOCS)*, pages 427–437. IEEE, 1987.
- [GJKR99] Rosario Gennaro, Stanisław Jarecki, Hugo Krawczyk, and Tal Rabin. Secure distributed key generation for discrete-log based cryptosystems. In *Advances in Cryptology – EUROCRYPT 1999*, volume 1592 of *Lecture Notes in Computer Science*, pages 295–310. Springer, 1999.
- [HA23] Brook Heisler and Jorge Aparicio. Criterion.rs: Statistics-driven microbenchmarking for Rust. <https://github.com/bheisler/criterion.rs>, 2023. Version 0.5.
- [HJKY95] Amir Herzberg, Stanisław Jarecki, Hugo Krawczyk, and Moti Yung. Proactive secret sharing or: How to cope with perpetual leakage. In *Advances in Cryptology – CRYPTO 1995*, volume 963 of *Lecture Notes in Computer Science*, pages 339–352. Springer, 1995.
- [Kao26] Leo Kao. FIPS 204-compatible threshold ML-DSA via Shamir nonce DKG. arXiv preprint arXiv:2601.20917, 2026. Available from World Wide Web: <https://arxiv.org/abs/2601.20917>.

- [Kat07] Jonathan Katz. Universally composable multi-party computation using tamper-proof hardware. In *Advances in Cryptology – EUROCRYPT 2007*, volume 4515 of *Lecture Notes in Computer Science*, pages 115–128. Springer, 2007.
- [KG21] Chelsea Komlo and Ian Goldberg. FROST: Flexible round-optimized Schnorr threshold signatures. In *Selected Areas in Cryptography (SAC 2020)*, volume 12804 of *Lecture Notes in Computer Science*, pages 34–65. Springer, 2021.
- [KLS18] Eike Kiltz, Vadim Lyubashevsky, and Christian Schaffner. A concrete treatment of Fiat–Shamir signatures in the quantum random-oracle model. In *Advances in Cryptology – EUROCRYPT 2018*, volume 10822 of *Lecture Notes in Computer Science*, pages 552–586. Springer, 2018.
- [KRT24] Shuichi Katsumata, Michael Reichle, and Kaoru Takemure. Adaptively secure 5 round threshold signatures from MLWE/MSIS and DL with rewinding. Cryptology ePrint Archive, Paper 2024/1033, 2024. Available from World Wide Web: <https://eprint.iacr.org/2024/1033>.
- [Lin21] Yehuda Lindell. Fast secure two-party ECDSA signing. *J. Cryptology*, 34(4):Art. 44, 2021.
- [Lyu12] Vadim Lyubashevsky. Lattice signatures without trapdoors. In *Advances in Cryptology – EUROCRYPT 2012*, volume 7237 of *Lecture Notes in Computer Science*, pages 738–755. Springer, 2012.
- [Nat24a] National Institute of Standards and Technology. FIPS PUB 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard. Federal Information Processing Standards Publication FIPS PUB 203, National Institute of Standards and Technology (NIST), 2024. Available from World Wide Web: <https://csrc.nist.gov/pubs/fips/203/final>.
- [Nat24b] National Institute of Standards and Technology. FIPS PUB 204: Module-Lattice-Based Digital Signature Standard. Federal Information Processing Standards Publication FIPS PUB 204, National Institute of Standards and Technology (NIST), 2024. Available from World Wide Web: <https://csrc.nist.gov/pubs/fips/204/final>.
- [Pop24] Giacomo Pope. dilithium-py: Pure Python implementation of ML-DSA (FIPS 204). <https://github.com/GiacomoPope/dilithium-py>, 2024. Version 1.4.0.
- [Sha79] Adi Shamir. How to share a secret. *Commun. ACM*, 22(11):612–613, 1979.